

# SciFig-Eval: Source Code Listing

Paul Osemudiamé Oamen  
Department of Computing Science  
University of Aberdeen

2026

This document contains the source code listing for the SciFig-Eval project. The complete source code, including all scripts, dataset, and evaluation outputs, is publicly available at: <https://github.com/WeNLP4Science-Lab/SciFig-Evaluation>

## Contents

|          |                                      |           |
|----------|--------------------------------------|-----------|
| <b>1</b> | <b>Model Abstraction Layer</b>       | <b>3</b>  |
| <b>2</b> | <b>Generation Pipeline</b>           | <b>6</b>  |
| <b>3</b> | <b>MQM Evaluation Pipeline</b>       | <b>11</b> |
| <b>4</b> | <b>Adversarial Experiments</b>       | <b>19</b> |
| <b>5</b> | <b>Results Computation</b>           | <b>31</b> |
| <b>6</b> | <b>Evaluation Experiment Scorers</b> | <b>37</b> |
| <b>7</b> | <b>Plotting Scripts</b>              | <b>48</b> |

## Listings

|    |                                                                                                      |    |
|----|------------------------------------------------------------------------------------------------------|----|
| 1  | scripts/llm_generation/models.py — Model registry and FigureAnnotator classes . . . . .              | 3  |
| 2  | scripts/llm_generation/prompts.py — Prompt template loader . . . . .                                 | 6  |
| 3  | scripts/llm_generation/run.py — Main generation entry point . . . . .                                | 6  |
| 4  | scripts/llm_generation/run_adversarial.py — Adversarial generation runner . . . . .                  | 8  |
| 5  | scripts/evaluation/mqm_evaluator.py — Core MQM scoring and judge prompts . . . . .                   | 11 |
| 6  | scripts/evaluation/evaluate_reference_free.py — Judge A: image-only evaluation . . . . .             | 15 |
| 7  | scripts/evaluation/evaluate_reference_with_image.py — Judge C: image + groundtruth . . . . .         | 17 |
| 8  | scripts/evaluation/judge_backends.py — Backend routing (Azure/OpenRouter/Vertex) . . . . .           | 18 |
| 9  | scripts/experiments/run_transforms.py — Image transform experiment runner . . . . .                  | 19 |
| 10 | scripts/experiments/run_prompt_reverse.py — Prompt reversal experiment . . . . .                     | 21 |
| 11 | scripts/experiments/run_caption_bias.py — Caption bias experiment . . . . .                          | 23 |
| 12 | scripts/experiments/run_questions.py — Capability questions runner . . . . .                         | 25 |
| 13 | scripts/experiments/run_active_admittance.py — Active admittance probes . . . . .                    | 29 |
| 14 | scripts/experiments/run_inductance.py — Inductance reasoning probes . . . . .                        | 30 |
| 15 | scripts/evaluation-results/rq1/compute_leaderboard.py — RQ1 leaderboard with bootstrap CIs . . . . . | 31 |
| 16 | scripts/evaluation-results/rq2/compute_degradation.py — RQ2 transform degradation . . . . .          | 33 |
| 17 | scripts/evaluation-results/rq3/compute_capability.py — RQ3 capability accuracy . . . . .             | 35 |
| 18 | scripts/evaluation-results/rq4/compute_ari.py — RQ4 A-R-I behavioural scores . . . . .               | 36 |
| 19 | scripts/experiments/evaluate_admittance.py — Admittance scoring . . . . .                            | 37 |
| 20 | scripts/experiments/evaluate_capability.py — Capability question scoring . . . . .                   | 40 |
| 21 | scripts/experiments/evaluate_prompt_reverse.py — Prompt reversal scoring . . . . .                   | 43 |
| 22 | scripts/experiments/evaluate_caption_bias.py — Caption bias scoring . . . . .                        | 45 |
| 23 | scripts/evaluation-results/rq1/plot_heatmap.py — Per-language heatmap . . . . .                      | 48 |
| 24 | scripts/evaluation-results/rq2/plot_degradation_slope.py — Degradation slope chart . . . . .         | 49 |
| 25 | scripts/evaluation-results/rq4/plot_ari_dotstrip.py — A-R-I dot-strip profiles . . . . .             | 51 |

|    |                                                                                                  |    |
|----|--------------------------------------------------------------------------------------------------|----|
| 26 | <code>scripts/evaluation-results/discussion/plot_rank_flow.py</code> — Rank flow chart . . . . . | 52 |
|----|--------------------------------------------------------------------------------------------------|----|

## Model Abstraction Layer

```
1 """LLM model abstractions for figure description generation.
2
3 Each model implements the FigureAnnotator interface. To add a new model,
4 subclass FigureAnnotator and implement annotate_figure() and model_name.
5
6 GPT-5.2 routes through OpenRouter; Gemini 3.1 Pro uses Vertex AI directly.
7 """
8
9 import base64
10 import os
11 import time
12 import logging
13 from abc import ABC, abstractmethod
14 from pathlib import Path
15
16 logger = logging.getLogger(__name__)
17
18 OPENROUTER_BASE_URL = "https://openrouter.ai/api/v1"
19
20
21 class FigureAnnotator(ABC):
22     """Abstract base class for LLM-based figure annotators."""
23
24     @property
25     @abstractmethod
26     def model_name(self) -> str:
27         ...
28
29     @property
30     @abstractmethod
31     def router_model_id(self) -> str:
32         ...
33
34     @abstractmethod
35     def annotate_figure(self, prompt: str, image_path: Path, caption: str, paper_title: str = "") -> str:
36         ...
37
38     def __repr__(self) -> str:
39         return f"{self.__class__.__name__}(model_name={self.model_name!r})"
40
41
42 def _encode_image_base64(image_path: Path) -> str:
43     with open(image_path, "rb") as f:
44         return base64.b64encode(f.read()).decode("utf-8")
45
46
47 def _retry(func, max_retries: int = 3, backoff: float = 2.0):
48     delay = 1.0
49     for attempt in range(1, max_retries + 1):
50         try:
51             return func()
52         except Exception as e:
53             if attempt == max_retries:
54                 logger.error(f"Failed after {max_retries} attempts: {e}")
55                 raise
56             logger.warning(f"Attempt {attempt} failed: {e}. Retrying in {delay:.1f}s...")
57             time.sleep(delay)
58             delay *= backoff
59
60
61 def _get_openrouter_client():
62     from openai import OpenAI
63
64     api_key = os.environ.get("OPENROUTER_API_KEY")
65     if not api_key:
66         raise EnvironmentError("OPENROUTER_API_KEY environment variable is not set.")
67     return OpenAI(base_url=OPENROUTER_BASE_URL, api_key=api_key)
68
69
70 def _get_vertex_client():
71     from google import genai
72
73     project = os.environ.get("GOOGLE_CLOUD_PROJECT")
74     location = os.environ.get("GOOGLE_CLOUD_LOCATION", "global")
75     if not project:
76         raise EnvironmentError(
77             "GOOGLE_CLOUD_PROJECT environment variable is not set. "
78             "Set it to your GCP project ID."
79         )
80     return genai.Client(vertexai=True, project=project, location=location)
81
82
83 # -----
84 # GPT-5.2 via OpenRouter
85 # -----
86
87 class GPT52Annotator(FigureAnnotator):
88     """GPT-5.2 via OpenRouter."""
89
90     def __init__(self, max_tokens: int = 2048):
91         self.max_tokens = max_tokens
92
93     @property
94     def model_name(self) -> str:
95         return "gpt-5.2"
96
97     @property
98     def router_model_id(self) -> str:
99         return "openai/gpt-5.2"
100
101     def annotate_figure(self, prompt: str, image_path: Path, caption: str, paper_title: str = "") -> str:
102         client = _get_openrouter_client()
103         b64 = _encode_image_base64(image_path)
104
105         user_text = f"Paper: {paper_title}\nCaption: {caption}" if paper_title else f"Caption: {caption}"
106
107         user_content = [
108             {
109                 "type": "image_url",
110                 "image_url": {"url": f"data:image/png;base64,{b64}", "detail": "high"},
111             },
112             {
113                 "type": "text",
114                 "text": user_text,
115             },
116         ]
117
```

```

118     def _call():
119         response = client.chat.completions.create(
120             model=self.router_model_id,
121             messages=[
122                 {"role": "system", "content": prompt},
123                 {"role": "user", "content": user_content},
124             ],
125             max_tokens=self.max_tokens,
126             temperature=0,
127         )
128         return response.choices[0].message.content.strip()
129
130     return _retry(_call)
131
132 # -----
133 # Azure OpenAI annotator (for GPT and Phi models deployed on Azure)
134 # -----
135
136 class AzureAnnotator(FigureAnnotator):
137     """Annotator for models deployed on Azure OpenAI."""
138
139     def __init__(self, name: str, deployment_name: str, max_tokens: int = 2048):
140         self._model_name = name
141         self._deployment_name = deployment_name
142         self.max_tokens = max_tokens
143
144     @property
145     def model_name(self) -> str:
146         return self._model_name
147
148     @property
149     def router_model_id(self) -> str:
150         return self._deployment_name
151
152     def annotate_figure(self, prompt: str, image_path: Path, caption: str, paper_title: str = "") -> str:
153         from openai import AzureOpenAI
154
155         client = AzureOpenAI(
156             azure_endpoint=os.environ["AZURE_OPENAI_ENDPOINT"],
157             api_key=os.environ["AZURE_OPENAI_API_KEY"],
158             api_version="2024-12-01-preview",
159         )
160         b64 = _encode_image_base64(image_path)
161         user_text = f"Paper: {paper_title}\nCaption: {caption}" if paper_title else f"Caption: {caption}"
162
163         user_content = [
164             {"type": "image_url", "image_url": {"url": f"data:image/png;base64,{b64}", "detail": "high"}},
165             {"type": "text", "text": user_text},
166         ]
167
168         token_param = "max_completion_tokens" if any(v in self._deployment_name for v in ["gpt-5", "gpt-4.1"]) else "max_tokens"
169
170     def _call():
171         response = client.chat.completions.create(
172             model=self._deployment_name,
173             messages=[
174                 {"role": "system", "content": prompt},
175                 {"role": "user", "content": user_content},
176             ],
177             **{token_param: self.max_tokens},
178             temperature=0,
179         )
180         return response.choices[0].message.content.strip()
181
182     return _retry(_call)
183
184 # -----
185 # Llama 4 Maverick via OpenRouter
186 # -----
187
188 class Llama4MaverickAnnotator(FigureAnnotator):
189     """Llama 4 Maverick via OpenRouter."""
190
191     def __init__(self, max_tokens: int = 2048):
192         self.max_tokens = max_tokens
193
194     @property
195     def model_name(self) -> str:
196         return "llama4-maverick"
197
198     @property
199     def router_model_id(self) -> str:
200         return "meta-llama/llama-4-maverick-17b-128e-instruct"
201
202     def annotate_figure(self, prompt: str, image_path: Path, caption: str, paper_title: str = "") -> str:
203         client = _get_openrouter_client()
204         b64 = _encode_image_base64(image_path)
205
206         user_text = f"Paper: {paper_title}\nCaption: {caption}" if paper_title else f"Caption: {caption}"
207
208         user_content = [
209             {
210                 "type": "image_url",
211                 "image_url": {"url": f"data:image/png;base64,{b64}", "detail": "high"},
212             },
213             {
214                 "type": "text",
215                 "text": user_text,
216             },
217         ]
218
219     def _call():
220         response = client.chat.completions.create(
221             model=self.router_model_id,
222             messages=[
223                 {"role": "system", "content": prompt},
224                 {"role": "user", "content": user_content},
225             ],
226             max_tokens=self.max_tokens,
227             temperature=0,
228         )
229         return response.choices[0].message.content.strip()
230
231     return _retry(_call)
232
233 # -----
234 # Generic OpenRouter annotator (for models that follow the standard pattern)
235 # -----

```

```

239
240 class OpenRouterAnnotator(FigureAnnotator):
241     """Generic annotator for any model available via OpenRouter."""
242
243     def __init__(self, name: str, router_id: str, max_tokens: int = 2048):
244         self._model_name = name
245         self._router_model_id = router_id
246         self.max_tokens = max_tokens
247
248     @property
249     def model_name(self) -> str:
250         return self._model_name
251
252     @property
253     def router_model_id(self) -> str:
254         return self._router_model_id
255
256     def annotate_figure(self, prompt: str, image_path: Path, caption: str, paper_title: str = "") -> str:
257         client = _get_openrouter_client()
258         b64 = _encode_image_base64(image_path)
259
260         user_text = f"Paper: {paper_title}\nCaption: {caption}" if paper_title else f"Caption: {caption}"
261
262         user_content = [
263             {
264                 "type": "image_url",
265                 "image_url": {"url": f"data:image/png;base64,{b64}", "detail": "high"},
266             },
267             {
268                 "type": "text",
269                 "text": user_text,
270             },
271         ]
272
273     def _call():
274         response = client.chat.completions.create(
275             model=self.router_model_id,
276             messages=[
277                 {"role": "system", "content": prompt},
278                 {"role": "user", "content": user_content},
279             ],
280             max_tokens=self.max_tokens,
281             temperature=0,
282         )
283         return response.choices[0].message.content.strip()
284
285     return _retry(_call)
286
287
288 def _openrouter_factory(name: str, router_id: str):
289     """Create a factory function for an OpenRouter model."""
290     def factory(**kwargs):
291         return OpenRouterAnnotator(name, router_id, **kwargs)
292     return factory
293
294
295 # -----
296 # Registry
297 # -----
298
299 MODEL_REGISTRY: dict[str, type[FigureAnnotator]] = {
300     "llama4-maverick": Llama4MaverickAnnotator,
301 }
302
303 # Azure-deployed models
304 AZURE_MODELS = {
305     "gpt-5.2": "gpt-5-2",
306     "phi-4-multimodal": "phi-4-multimodal",
307 }
308
309 # OpenRouter models (using generic annotator)
310 OPENROUTER_MODELS = {
311     "gemma3-4b-it": "google/gemma-3-4b-it",
312     "gemma3-12b-it": "google/gemma-3-12b-it",
313     "gemma3-27b-it": "google/gemma-3-27b-it",
314     "qwen3-vl-8b": "qwen/qwen3-vl-8b-instruct",
315     "qwen3-vl-30b-a3b": "qwen/qwen3-vl-30b-a3b-instruct",
316     "qwen3-vl-32b": "qwen/qwen3-vl-32b-instruct",
317     "qwen3-vl-235b-a22b": "qwen/qwen3-vl-235b-a22b-instruct",
318     "llama4-scout": "meta-llama/llama-4-scout",
319     "claude-opus-4.6": "anthropic/claude-opus-4-6",
320 }
321
322
323 def _azure_factory(name: str, deployment_name: str):
324     """Create a factory function for an Azure-deployed model."""
325     def factory(**kwargs):
326         return AzureAnnotator(name, deployment_name, **kwargs)
327     return factory
328
329
330 # Models with custom max_tokens
331 CUSTOM_TOKEN_MODELS = {
332     "gemini-3.1-pro": ("google/gemini-3.1-pro-preview", 16000),
333 }
334
335
336 def _get_all_models() -> dict:
337     """Merge all model registries."""
338     all_models = dict(MODEL_REGISTRY)
339     # Add models with custom max_tokens
340     for name, (router_id, max_tok) in CUSTOM_TOKEN_MODELS.items():
341         all_models[name] = _openrouter_factory(name, router_id)
342         # Override default max_tokens via a wrapper
343         base_factory = all_models[name]
344         def _custom_factory(bf=base_factory, mt=max_tok):
345             def factory(**kwargs):
346                 kwargs.setdefault('max_tokens', mt)
347                 return bf(**kwargs)
348             return factory
349         all_models[name] = _custom_factory()
350     # Add Azure-deployed models
351     for name, deployment in AZURE_MODELS.items():
352         all_models[name] = _azure_factory(name, deployment)
353     # Add generic OpenRouter models
354     for name, router_id in OPENROUTER_MODELS.items():
355         all_models[name] = _openrouter_factory(name, router_id)
356     return all_models
357
358
359 def get_annotator(model_name: str, **kwargs) -> FigureAnnotator:

```

```

360 all_models = _get_all_models()
361 if model_name not in all_models:
362     available = ", ".join(sorted(all_models.keys()))
363     raise ValueError(f"Unknown model {model_name!r}. Available: {available}")
364 return all_models[model_name](**kwargs)

```

Listing 1: scripts/llm\_generation/models.py — Model registry and FigureAnnotator classes

```

1 """Prompt loading and selection for figure description generation."""
2
3 from pathlib import Path
4
5 PROMPTS_DIR = Path(__file__).resolve().parent / "prompts"
6
7 # Maps figure_type values (from groundtruth) to prompt filenames
8 FIGURE_TYPE_TO_FILE = {
9     "Line Plot": "line_plot.txt",
10    "Bar Chart": "bar_chart.txt",
11    "Pie Chart": "pie_chart.txt",
12 }
13
14 DEFAULT_PROMPT_FILE = "default.txt"
15
16 # Maps annotation_language / paper_language to translated prompt subdirectory
17 LANGUAGE_TO_DIR = {
18     "Bulgarian": "translated/bulgarian",
19     "Chinese": "translated/chinese",
20     "German": "translated/german",
21 }
22
23
24 def load_prompts() -> dict[tuple[str, str], str]:
25     """Load all prompts into a (language, figure_type) -> prompt_text mapping.
26
27     English prompts are loaded from the top-level prompts/ directory.
28     Translated prompts are loaded from prompts/translated/<language>/.
29     Falls back gracefully if a translated prompt file is missing.
30
31     Returns:
32         Dictionary mapping (language, figure_type) to prompt string.
33     """
34     prompts = {}
35
36     # Load English prompts (top-level)
37     for figure_type, filename in FIGURE_TYPE_TO_FILE.items():
38         path = PROMPTS_DIR / filename
39         if path.exists():
40             prompts[("English", figure_type)] = path.read_text(encoding="utf-8")
41
42     # English default
43     default_path = PROMPTS_DIR / DEFAULT_PROMPT_FILE
44     if default_path.exists():
45         prompts[("English", "Default")] = default_path.read_text(encoding="utf-8")
46
47     # Load translated prompts
48     for language, subdir in LANGUAGE_TO_DIR.items():
49         lang_dir = PROMPTS_DIR / subdir
50         if not lang_dir.exists():
51             continue
52         for figure_type, filename in FIGURE_TYPE_TO_FILE.items():
53             path = lang_dir / filename
54             if path.exists():
55                 prompts[(language, figure_type)] = path.read_text(encoding="utf-8")
56         default_path = lang_dir / DEFAULT_PROMPT_FILE
57         if default_path.exists():
58             prompts[(language, "Default")] = default_path.read_text(encoding="utf-8")
59
60     return prompts
61
62
63 def get_prompt(
64     prompts: dict[tuple[str, str], str],
65     language: str,
66     figure_type: str,
67 ) -> str:
68     """Select the best prompt for a given language and figure type.
69
70     Fallback chain:
71     1. (language, figure_type)      exact match
72     2. (language, "Default")        language-specific default
73     3. ("English", figure_type)     English prompt for that figure type
74     4. ("English", "Default")       ultimate fallback
75
76     Raises:
77         ValueError: If no prompt can be found at all (shouldn't happen if
78                     English prompts are present).
79     """
80     for key in [
81         (language, figure_type),
82         (language, "Default"),
83         ("English", figure_type),
84         ("English", "Default"),
85     ]:
86         if key in prompts:
87             return prompts[key]
88
89     raise ValueError(
90         f"No prompt found for language={language!r}, figure_type={figure_type!r}. "
91         "Ensure at least the English default prompt exists."
92     )

```

Listing 2: scripts/llm\_generation/prompts.py — Prompt template loader

## Generation Pipeline

```

1 """Runner script for LLM figure description generation.
2
3 Usage:
4     export OPENROUTER_API_KEY=sk-or-...
5     python3 scripts/llm_generation/run.py <model_name> [--workers N]
6 """

```

```

7
8 import argparse
9 import json
10 import logging
11 import sys
12 import threading
13 from concurrent.futures import ThreadPoolExecutor, as_completed
14 from pathlib import Path
15
16 # Paths
17 ROOT = Path(__file__).resolve().parent.parent.parent
18 GROUNDTRUTH_DIR = ROOT / "Dataset" / "groundtruth"
19 FIGURES_DIR = ROOT / "Dataset" / "figures"
20 OUTPUT_DIR = ROOT / "output" / "generation"
21
22 sys.path.insert(0, str(ROOT / "scripts"))
23
24 from llm_generation.models import get_annotator
25 from llm_generation.prompts import load_prompts, get_prompt
26
27 logging.basicConfig(
28     level=logging.INFO,
29     format="%(asctime)s %(levelname)s %(message)s",
30     datefmt="%H:%M:%S",
31 )
32 logger = logging.getLogger(__name__)
33
34 SUBFOLDERS = [
35     "bulgarian_only",
36     "chinese_only",
37     "english_only",
38     "german_only",
39     "multi_language",
40 ]
41
42
43 def _discover_figures(subfolder: str) -> list[str]:
44     """Discover all figure keys in a groundtruth subfolder."""
45     gt_folder = GROUNDTRUTH_DIR / subfolder
46     keys = sorted(p.stem for p in gt_folder.glob("*.json"))
47     return keys
48
49
50 def _process_figure(annotator, prompts, subfolder, fig_key, model_output_dir):
51     """Process a single figure. Returns (success: bool, fig_key: str)."""
52     gt_folder = GROUNDTRUTH_DIR / subfolder
53     fig_folder = FIGURES_DIR / subfolder
54     out_folder = model_output_dir / subfolder
55
56     gt_path = gt_folder / f"{fig_key}.json"
57     fig_path = fig_folder / f"{fig_key}.png"
58     out_path = out_folder / f"{fig_key}.json"
59
60     # Re-check skip (race-safe: unique file per figure)
61     if out_path.exists():
62         return True, fig_key, "skip"
63
64     with open(gt_path) as f:
65         gt = json.load(f)
66
67     if subfolder == "multi_language":
68         languages = sorted(set(
69             a["annotation_language"] for a in gt["annotations"]
70         ))
71     else:
72         languages = [gt["paper_language"]]
73
74     logger.info(f" [{subfolder}] {fig_key} ({gt['figure_type']}) languages: {languages}")
75
76     if len(languages) == 1:
77         prompt_text = get_prompt(prompts, languages[0], gt["figure_type"])
78         try:
79             description = annotator.annotate_figure(
80                 prompt=prompt_text,
81                 image_path=fig_path,
82                 caption=gt["caption"],
83                 paper_title=gt.get("paper_title", ""),
84             )
85         except Exception as e:
86             logger.error(f" FAIL {fig_key}: {e}")
87             return False, fig_key, str(e)
88
89         result = {
90             "figure_key": fig_key,
91             "task_id": gt["task_id"],
92             "arxiv_id": gt.get("arxiv_id"),
93             "paper_title": gt.get("paper_title", ""),
94             "paper_language": gt["paper_language"],
95             "figure_type": gt["figure_type"],
96             "caption": gt["caption"],
97             "model_name": annotator.model_name,
98             "model_annotation": description,
99         }
100         logger.info(f" OK {fig_key} ({len(description)} chars)")
101
102     else:
103         annotations_by_lang = {}
104         for lang in languages:
105             prompt_text = get_prompt(prompts, lang, gt["figure_type"])
106             logger.info(f" generating {lang}...")
107             try:
108                 desc = annotator.annotate_figure(
109                     prompt=prompt_text,
110                     image_path=fig_path,
111                     caption=gt["caption"],
112                     paper_title=gt.get("paper_title", ""),
113                 )
114                 annotations_by_lang[lang] = desc
115                 logger.info(f" {lang}: {len(desc)} chars")
116             except Exception as e:
117                 logger.error(f" FAIL {fig_key} ({lang}): {e}")
118                 return False, fig_key, str(e)
119
120         result = {
121             "figure_key": fig_key,
122             "task_id": gt["task_id"],
123             "arxiv_id": gt.get("arxiv_id"),
124             "paper_title": gt.get("paper_title", ""),
125             "paper_language": gt["paper_language"],
126             "figure_type": gt["figure_type"],
127             "caption": gt["caption"],

```

```

128         "model_name": annotator.model_name,
129         "model_annotations": annotations_by_lang,
130     }
131     logger.info(f" OK {fig_key} ({len(annotations_by_lang)} languages)")
132
133     with open(out_path, "w") as f:
134         json.dump(result, f, indent=2, ensure_ascii=False)
135
136     return True, fig_key, "done"
137
138
139 def run(model_name: str = "gpt-5.2", workers: int = 1, subfolder_filter: str | None = None):
140     annotator = get_annotator(model_name)
141     prompts = load_prompts()
142     model_output_dir = OUTPUT_DIR / annotator.model_name
143
144     subfolders = [subfolder_filter] if subfolder_filter else SUBFOLDERS
145
146     logger.info(f"Model: {annotator.model_name}")
147     logger.info(f"Output: {model_output_dir}")
148     logger.info(f"Workers: {workers}")
149     logger.info(f"Subfolders: {subfolders}")
150     logger.info(f"Loaded {len(prompts)} prompts")
151     print()
152
153     # Collect all work items, filtering out already-done and missing files
154     work_items = []
155     skipped = 0
156     for subfolder in subfolders:
157         gt_folder = GROUNDTRUTH_DIR / subfolder
158         fig_folder = FIGURES_DIR / subfolder
159         out_folder = model_output_dir / subfolder
160         out_folder.mkdir(parents=True, exist_ok=True)
161
162         for fig_key in discover_figures(subfolder):
163             gt_path = gt_folder / f"{fig_key}.json"
164             fig_path = fig_folder / f"{fig_key}.png"
165             out_path = out_folder / f"{fig_key}.json"
166
167             if not gt_path.exists() or not fig_path.exists():
168                 logger.warning(f" SKIP {fig_key} missing groundtruth or image")
169                 continue
170             if out_path.exists():
171                 skipped += 1
172                 continue
173
174             work_items.append((subfolder, fig_key))
175
176     total = len(work_items) + skipped
177     logger.info(f"Total figures: {total} ({skipped} already done, {len(work_items)} to process)")
178     print()
179
180     if not work_items:
181         logger.info("Nothing to do all figures already generated.")
182         return
183
184     success = skipped
185     errors = 0
186     lock = threading.Lock()
187
188     def _on_complete(ok, fig_key, status):
189         nonlocal success, errors
190         with lock:
191             if ok:
192                 success += 1
193             else:
194                 errors += 1
195             done = success + errors
196             if done % 10 == 0 or not ok:
197                 logger.info(f" Progress: {done - skipped}/{len(work_items)} "
198                             f"(success={success}, errors={errors})")
199
200     if workers == 1:
201         for subfolder, fig_key in work_items:
202             ok, key, status = _process_figure(
203                 annotator, prompts, subfolder, fig_key, model_output_dir
204             )
205             _on_complete(ok, key, status)
206     else:
207         with ThreadPoolExecutor(max_workers=workers) as executor:
208             futures = []
209             executor.submit(
210                 _process_figure, annotator, prompts, subfolder, fig_key, model_output_dir
211             ): fig_key
212             for subfolder, fig_key in work_items
213         }
214         for future in as_completed(futures):
215             try:
216                 ok, key, status = future.result()
217                 _on_complete(ok, key, status)
218             except Exception as e:
219                 fig_key = futures[future]
220                 logger.error(f" UNEXPECTED {fig_key}: {e}")
221                 with lock:
222                     errors += 1
223
224     print()
225     logger.info(f"Done. {success}/{total} succeeded, {errors} errors.")
226
227
228 if __name__ == "__main__":
229     parser = argparse.ArgumentParser(description="Generate figure descriptions with LLMs")
230     parser.add_argument("model", nargs="?", default="gpt-5.2", help="Model name")
231     parser.add_argument("--workers", type=int, default=1, help="Number of parallel workers")
232     parser.add_argument("--subfolder", type=str, default=None, help="Process only this subfolder")
233     args = parser.parse_args()
234     run(args.model, args.workers, args.subfolder)

```

Listing 3: scripts/llm\_generation/run.py — Main generation entry point

```

1 """Generate figure descriptions for adversarial-transformed images.
2
3 Runs a generator model on transformed versions of selected figures.
4 Uses Azure OpenAI for GPT models.
5
6 Output: output/generation_adversarial/<model>/<transform>/<subfolder>/<fig_key>.json
7
8 Usage:

```



```

9     python3 scripts/llm_generation/run_adversarial.py --model gpt-5.2 --workers 4
10    python3 scripts/llm_generation/run_adversarial.py --model gpt-5.2 --transform jpeg_compression
11    """
12
13    import argparse
14    import base64
15    import json
16    import logging
17    import os
18    import sys
19    import time
20    import threading
21    from concurrent.futures import ThreadPoolExecutor, as_completed
22    from pathlib import Path
23
24    from dotenv import load_dotenv
25
26    ROOT = Path(__file__).resolve().parent.parent.parent
27    load_dotenv(ROOT / ".env")
28
29    sys.path.insert(0, str(ROOT / "scripts"))
30    from llm_generation.prompts import load_prompts, get_prompt
31
32    TRANSFORMS_DIR = ROOT / "transforms"
33    GROUNDTRUTH_DIR = ROOT / "Dataset" / "groundtruth"
34    OUTPUT_DIR = ROOT / "output" / "generation_adversarial"
35
36    logging.basicConfig(level=logging.INFO, format="%(asctime)s %(levelname)s %(message)s", datefmt="%H:%M:%S")
37    logger = logging.getLogger(__name__)
38
39
40    def _get_azure_client():
41        from openai import AzureOpenAI
42        return AzureOpenAI(
43            azure_endpoint=os.environ["AZURE_OPENAI_ENDPOINT"],
44            api_key=os.environ["AZURE_OPENAI_API_KEY"],
45            api_version="2024-12-01-preview",
46        )
47
48
49    def _encode_image(path: Path) -> str:
50        with open(path, "rb") as f:
51            return base64.b64encode(f.read()).decode("utf-8")
52
53
54    def _retry(func, max_retries=3, backoff=2.0):
55        delay = 1.0
56        for attempt in range(1, max_retries + 1):
57            try:
58                return func()
59            except Exception as e:
60                if attempt == max_retries:
61                    raise
62                logger.warning(f"Attempt {attempt} failed: {e}. Retrying in {delay:.1f}s...")
63                time.sleep(delay)
64                delay *= backoff
65
66
67    # Azure deployment name mapping
68    AZURE_DEPLOYMENTS = {
69        "gpt-5.2": "gpt-5.2",
70        "gpt-4o": "gpt-4o",
71        "gpt-5.4-mini": "gpt-5.4-mini",
72    }
73
74
75    def generate_description(client, deployment_name, prompt, image_path, caption, paper_title=""):
76        """Generate a figure description using Azure OpenAI."""
77        b64 = _encode_image(image_path)
78        user_text = f"Paper: {paper_title}\nCaption: {caption}" if paper_title else f"Caption: {caption}"
79
80        user_content = [
81            {"type": "image_url", "image_url": {"url": f"data:image/png;base64,{b64}", "detail": "high"}},
82            {"type": "text", "text": user_text},
83        ]
84
85        # Newer models use max_completion_tokens
86        token_param = "max_completion_tokens" if any(v in deployment_name for v in ["gpt-5", "gpt-4.1"]) else "max_tokens"
87
88        def _call():
89            response = client.chat.completions.create(
90                model=deployment_name,
91                messages=[
92                    {"role": "system", "content": prompt},
93                    {"role": "user", "content": user_content},
94                ],
95                **{token_param: 2048},
96            )
97            return response.choices[0].message.content.strip()
98
99        return _retry(_call)
100
101
102    def _process_figure(client, deployment_name, prompts, transform, subfolder, fig_key, model_name):
103        """Process a single transformed figure."""
104        fig_path = TRANSFORMS_DIR / transform / "figures" / subfolder / f"{fig_key}.png"
105        gt_path = GROUNDTRUTH_DIR / subfolder / f"{fig_key}.json"
106        out_path = OUTPUT_DIR / model_name / transform / subfolder / f"{fig_key}.json"
107
108        if out_path.exists():
109            return True, fig_key, "skip"
110        if not fig_path.exists() or not gt_path.exists():
111            return True, fig_key, "skip"
112
113        with open(gt_path) as f:
114            gt = json.load(f)
115
116        figure_type = gt["figure_type"]
117
118        if subfolder == "multi_language":
119            languages = sorted(set(a["annotation_language"] for a in gt["annotations"]))
120        else:
121            languages = [gt["paper_language"]]
122
123        try:
124            if len(languages) == 1:
125                prompt_text = get_prompt(prompts, languages[0], figure_type)
126                description = generate_description(client, deployment_name, prompt_text, fig_path, gt["caption"], gt.get("paper_title", ""))
127
128                result = {
129                    "figure_key": fig_key,

```

```

130         "task_id": gt["task_id"],
131         "arxiv_id": gt.get("arxiv_id"),
132         "paper_title": gt.get("paper_title", ""),
133         "paper_language": gt["paper_language"],
134         "figure_type": figure_type,
135         "caption": gt["caption"],
136         "model_name": model_name,
137         "transform": transform,
138         "model_annotation": description,
139     }
140 else:
141     annotations_by_lang = {}
142     for lang in languages:
143         prompt_text = get_prompt(prompts, lang, figure_type)
144         desc = generate_description(client, deployment_name, prompt_text, fig_path, gt["caption"], gt.get("paper_title", ""))
145         annotations_by_lang[lang] = desc
146
147     result = {
148         "figure_key": fig_key,
149         "task_id": gt["task_id"],
150         "arxiv_id": gt.get("arxiv_id"),
151         "paper_title": gt.get("paper_title", ""),
152         "paper_language": gt["paper_language"],
153         "figure_type": figure_type,
154         "caption": gt["caption"],
155         "model_name": model_name,
156         "transform": transform,
157         "model_annotations": annotations_by_lang,
158     }
159
160     out_path.parent.mkdir(parents=True, exist_ok=True)
161     with open(out_path, "w") as f:
162         json.dump(result, f, indent=2, ensure_ascii=False)
163
164     logger.info(f" OK {transform}/{subfolder}/{fig_key}")
165     return True, fig_key, "done"
166
167 except Exception as e:
168     logger.error(f" FAIL {transform}/{subfolder}/{fig_key}: {e}")
169     return False, fig_key, str(e)
170
171
172 def run(model_name="gpt-5.2", workers=4, transform_filter=None, max_figures=None):
173     # Load adversarial samples
174     with open(ROOT / "adversarial_samples.json") as f:
175         adv = json.load(f)
176
177     transforms = adv["transforms"]
178     if transform_filter:
179         transforms = [t for t in transforms if t == transform_filter]
180
181     deployment_name = AZURE_DEPLOYMENTS.get(model_name, model_name)
182     client = _get_azure_client()
183     prompts = load_prompts()
184
185     # Build work items
186     work_items = []
187     skipped = 0
188     for transform in transforms:
189         for subfolder, fig_keys in adv["samples"].items():
190             keys = fig_keys[:max_figures] if max_figures else fig_keys
191             for fig_key in keys:
192                 out_path = OUTPUT_DIR / model_name / transform / subfolder / f"{fig_key}.json"
193                 fig_path = TRANSFORMS_DIR / transform / "figures" / subfolder / f"{fig_key}.png"
194                 if out_path.exists():
195                     skipped += 1
196                 elif fig_path.exists():
197                     work_items.append((transform, subfolder, fig_key))
198
199     logger.info(f"Adversarial generation | model={model_name} | deployment={deployment_name}")
200     logger.info(f"Transforms: {transforms}")
201     logger.info(f"Total: {len(work_items) + skipped} ({skipped} done, {len(work_items)} to generate)")
202
203     if not work_items:
204         logger.info("Nothing to do.")
205         return
206
207     success, errors = skipped, 0
208     lock = threading.Lock()
209
210     def _on_done(ok, k, s):
211         nonlocal success, errors
212         with lock:
213             if ok:
214                 success += 1
215             else:
216                 errors += 1
217             d = success + errors - skipped
218             if d % 10 == 0 or not ok:
219                 logger.info(f" Progress: {d}/{len(work_items)} (errors={errors})")
220
221     if workers == 1:
222         for t, sub, fk in work_items:
223             ok, k, s = _process_figure(client, deployment_name, prompts, t, sub, fk, model_name)
224             _on_done(ok, k, s)
225     else:
226         with ThreadPoolExecutor(max_workers=workers) as ex:
227             futs = {
228                 ex.submit(_process_figure, client, deployment_name, prompts, t, sub, fk, model_name): fk
229                 for t, sub, fk in work_items
230             }
231             for f in as_completed(futs):
232                 try:
233                     ok, k, s = f.result()
234                     _on_done(ok, k, s)
235                 except Exception as e:
236                     logger.error(f" UNEXPECTED {futs[f]}: {e}")
237                     with lock:
238                         errors += 1
239
240     logger.info(f"Done. {success}/{len(work_items) + skipped} generated, {errors} failures.")
241
242
243 if __name__ == "__main__":
244     p = argparse.ArgumentParser(description="Generate descriptions for adversarial-transformed figures")
245     p.add_argument("--model", default="gpt-5.2", help="Model name")
246     p.add_argument("--workers", type=int, default=4)
247     p.add_argument("--transform", default=None, help="Filter to specific transform")
248     p.add_argument("--max-figures", type=int, default=None, help="Limit figures per subfolder (for testing)")
249     a = p.parse_args()
250     run(a.model, a.workers, a.transform, a.max_figures)

```

## MQM Evaluation Pipeline

```

1 """Core MQM evaluation logic      LLM-as-judge for figure annotation quality.
2
3 Uses the MQM (Multidimensional Quality Metrics) framework from the project's
4 human evaluation guidelines. Supports three judge configurations:
5
6 Reference-free: image + generation prompt + caption + model output (no groundtruth)
7 Reference-only: groundtruth annotation + model output (no image)
8 Reference-with-image: image + groundtruth annotation + model output
9 """
10
11 import base64
12 import json
13 import os
14 import re
15 import time
16 import logging
17 from pathlib import Path
18 from dotenv import load_dotenv
19
20 load_dotenv(Path(__file__).resolve().parent.parent.parent / ".env")
21
22 from evaluation.judge_backends import resolve_backend, get_openai_compat_client, get_vertex_client
23
24 logger = logging.getLogger(__name__)
25
26 # MQM weights from the guidelines (Table 2)
27 MQM_WEIGHTS = {
28     ("Accuracy", "Major"): 5.0,
29     ("Accuracy", "Minor"): 2.0,
30     ("Completeness", "Major"): 3.5,
31     ("Completeness", "Minor"): 1.5,
32     ("Clarity and Readability", "Major"): 2.0,
33     ("Clarity and Readability", "Minor"): 1.0,
34 }
35
36 VALID_CATEGORIES = {"Accuracy", "Completeness", "Clarity and Readability"}
37 VALID_SEVERITIES = {"Major", "Minor"}
38 VALID_SUB_TYPES = {
39     "Accuracy": {
40         "Incorrect Numerical Value",
41         "Incorrect Axis or Legend Interpretation",
42         "Incorrect Visual Attribute Mapping",
43         "Incorrect Structural Description",
44     },
45     "Completeness": {
46         "Missing Chart Purpose",
47         "Missing Axis Description",
48         "Missing Visual Features",
49         "Hallucinated Content",
50         "Unwanted Interpretation",
51     },
52     "Clarity and Readability": {
53         "Ambiguous Description",
54         "Over-Generalization",
55         "Overly Verbose Description",
56         "Poor Sentence Structure",
57     },
58 }
59
60 _MQM_ERROR_CATEGORIES = """\
61 ## MQM Error Categories
62
63 ### Accuracy Errors
64 - **Incorrect Numerical Value**: Reports incorrect numbers, percentages, or quantities. Example: Reporting "60%" when the plot shows approximately "45%".
65 - **Incorrect Axis or Legend Interpretation**: Incorrectly describes axis labels, units, or legend information. Example: Stating that the x-axis represents
66   temperature when it represents time.
67 - **Incorrect Visual Attribute Mapping**: Assigns incorrect category names, series, values, marker shapes, or labels to visual elements such as bars, lines,
68   pie segments, or datasets. Example: Assigning the blue bar to Method A when it corresponds to Method B.
69 - **Incorrect Structural Description**: Misrepresents the structural organization of the chart. Example: Describing grouped bars as stacked, reporting 5
70   slices when 4 exist, or misstating axis orientation.
71
72 ### Completeness Errors
73 - **Missing Chart Purpose**: The description does not state what the chart represents based on visible labels or title.
74 - **Missing Axis Description**: Omits essential axis information such as axis labels, units, scale type, value range, or tick intervals.
75 - **Missing Visual Features**: Misses visual and stylistic features such as grouped/stacked structure, legend presence, sorting or ordering, or visual
76   emphasis (highlighted bar, exploded slice).
77 - **Hallucinated Content**: Introduces information or visual elements that do not exist in the figure. Example: Stating that the chart contains five
78   categories when only four are present.
79 - **Unwanted Interpretation**: Includes interpretation or inference beyond what is explicitly shown in the figure, introduces comparisons not explicitly
80   indicated in the chart, or uses subjective or evaluative language (e.g., "significant increase").
81
82 ### Clarity and Readability Errors
83 - **Ambiguous Description**: Contains vague or unclear references. Example: Using phrases such as "this increases significantly" without specifying the
84   subject.
85 - **Over-Generalization**: Oversimplifies or exaggerates visual information. Example: Claiming that all methods perform similarly despite visible variation.
86 - **Overly Verbose Description**: Contains unnecessary repetition or excessive detail that does not add informational value.
87 - **Poor Sentence Structure**: Contains grammatical errors or awkward phrasing that reduces readability.
88
89 ## Severity Levels
90 - **Major**: Errors that make the description factually unusable or misleading, or significantly degrade usability or scientific correctness.
91 - **Minor**: Stylistic or readability issues that do not affect the core factual meaning of the annotation."""
92
93 # Sub-types where the error refers to MISSING information (no text span to highlight)
94 NON_ANNOTATABLE_SUB_TYPES = {
95     "Missing Chart Purpose",
96     "Missing Axis Description",
97     "Missing Visual Features",
98 }
99
100 _MQM_OUTPUT_FORMAT = """\
101 ## Output Format
102 Return a JSON object with this exact structure:
103 {
104     "errors": [
105         {
106             "category": "Accuracy",
107             "sub_type": "Incorrect Numerical Value",
108             "severity": "Major",

```

```

102     "text_span": "with a final value of 60%",
103     "evidence": "Description says 60% but figure shows approximately 45%"
104 },
105 {
106     "category": "Completeness",
107     "sub_type": "Missing Axis Description",
108     "severity": "Major",
109     "text_span": null,
110     "evidence": "The description does not mention the y-axis units (percentage)"
111 }
112 ]
113 }
114
115 ## text_span rules
116 - For errors about INCORRECT or UNWANTED content (Accuracy errors, Hallucinated Content, Unwanted Interpretation, all Clarity errors): set "text_span" to
  the EXACT substring from the machine-generated description that contains the error. Copy it verbatim do not paraphrase.
117 - For errors about MISSING content (Missing Chart Purpose, Missing Axis Description, Missing Visual Features): set "text_span" to null, since there is no
  text to highlight.
118
119 If there are no errors, return: {"errors": []}
120 Return ONLY the JSON object. No extra text.
121
122 # -----
123 # Reference-free (image + generation prompt + caption + model output, no groundtruth)
124 # -----
125 JUDGE_A_PROMPT = f"""
126 You are an expert evaluator assessing the quality of a machine-generated scientific figure description.
127
128 ## Your Task
129 Examine the provided figure image carefully. You are also given the GENERATION PROMPT that was used to instruct the model this defines what the
  description should cover. Evaluate the machine-generated description against WHAT YOU ACTUALLY SEE in the figure AND whether it fulfills the
  requirements of the generation prompt. Identify ALL errors.
130
131 {_MQM_ERROR_CATEGORIES}
132
133 ## Instructions
134 1. Examine the figure image thoroughly note axes, labels, data series, values, colors, legends, trends.
135 2. Read the generation prompt to understand what the model was asked to describe.
136 3. Read the machine-generated description.
137 4. Compare each claim in the description against what you see in the figure (for Accuracy).
138 5. Check whether the description covers everything the generation prompt requested (for Completeness). If the prompt asks for specific elements (e.g., axes,
  legend, colors, values) and the description omits them, flag as a Completeness error.
139 6. Identify ALL errors. Be thorough but fair minor approximations in numerical values are acceptable.
140 7. For each error, classify by category, sub_type, and severity, and provide specific evidence.
141
142 {_MQM_OUTPUT_FORMAT}"""
143
144 # -----
145 # Reference-only (groundtruth + model output, no image)
146 # -----
147 JUDGE_B_PROMPT = f"""
148 You are an expert evaluator assessing the quality of a machine-generated scientific figure description.
149
150 ## Your Task
151 You are given a REFERENCE description written by a human expert and a MACHINE-GENERATED description of the same scientific figure. Compare the machine-
  generated description against the reference to identify errors.
152
153 Use the reference as the source of truth for what the figure contains, since you do not have access to the image.
154
155 {_MQM_ERROR_CATEGORIES}
156
157 ## Instructions
158 1. Read the reference description carefully it tells you what the figure actually shows.
159 2. Read the machine-generated description.
160 3. Compare each claim in the machine-generated description against the reference.
161 4. Any claim that contradicts the reference is an Accuracy error.
162 5. Any important information present in the reference but missing from the machine output is a Completeness error.
163 6. Any information in the machine output that is NOT mentioned in the reference should be flagged as Hallucinated Content (it may be fabricated since you
  cannot verify it visually).
164 7. Assess clarity and readability of the machine-generated description on its own merits.
165 8. For each error, classify by category, sub_type, and severity, and provide specific evidence.
166
167 {_MQM_OUTPUT_FORMAT}"""
168
169 # -----
170 # Reference-with-image (image + groundtruth + model output)
171 # -----
172 JUDGE_C_PROMPT = f"""
173 You are an expert evaluator assessing the quality of a machine-generated scientific figure description.
174
175 ## Your Task
176 You are given the FIGURE IMAGE, a REFERENCE description written by a human expert, and a MACHINE-GENERATED description. Use BOTH the image and the reference
  to evaluate the machine-generated description. Identify ALL errors.
177
178 The image is the primary source of truth. The reference description provides additional context about what a human expert considered important to mention.
179
180 {_MQM_ERROR_CATEGORIES}
181
182 ## Instructions
183 1. Examine the figure image thoroughly note axes, labels, data series, values, colors, legends, trends.
184 2. Read the reference description to understand what a human expert highlighted.
185 3. Read the machine-generated description.
186 4. Compare each claim in the machine-generated description against BOTH the image and the reference.
187 5. Identify ALL errors. Be thorough but fair minor approximations in numerical values should be flagged as Minor, not Major. Reserve Major for values
  that are clearly wrong or misleading.
188 6. Use the image to verify factual claims.
189 7. **Completeness check**: Go through the reference description sentence by sentence. For each piece of information in the reference, check whether the
  machine-generated description covers it. If not, flag it as a Completeness error with the appropriate sub-type:
190 - **Missing Chart Purpose**: if the reference states what the chart represents or illustrates and the model omits this.
191 - **Missing Axis Description**: if the reference describes axis labels, units, scale type, value ranges, or tick intervals and the model omits any of
  these.
192 - **Missing Visual Features**: if the reference mentions specific data series, colors, line styles, markers, legend items, annotations, values, groupings
  , or structural elements and the model omits any of these.
193 Flag each missing element as a separate error. Do not combine multiple omissions into one error.
194 8. For each error, classify by category, sub_type, and severity, and provide specific evidence.
195
196 {_MQM_OUTPUT_FORMAT}"""
197
198 # Keep backward compatibility
199 MQM_JUDGE_PROMPT = JUDGE_A_PROMPT
200
201
202
203
204 def _encode_image(path: Path) -> str:
205     with open(path, "rb") as f:
206         return base64.b64encode(f.read()).decode("utf-8")
207
208
209 def _extract_json(text: str) -> dict:
210     text = text.strip()

```

```

211     try:
212         return json.loads(text)
213     except json.JSONDecodeError:
214         pass
215     match = re.search(r"```(?:json)?\s*\n?(.*)\n?\s*```", text, re.DOTALL)
216     if match:
217         try:
218             return json.loads(match.group(1).strip())
219         except json.JSONDecodeError:
220             pass
221     raise ValueError(f"Could not parse JSON from judge response: {text[:300]}...")
222
223
224 def _retry(func, max_retries: int = 3, backoff: float = 2.0):
225     delay = 1.0
226     for attempt in range(1, max_retries + 1):
227         try:
228             return func()
229         except Exception as e:
230             if attempt == max_retries:
231                 raise
232             logger.warning(f"Attempt {attempt} failed: {e}. Retrying in {delay:.1f}s...")
233             time.sleep(delay)
234             delay *= backoff
235
236
237 def compute_mqm_score(errors: list[dict]) -> tuple[float, float]:
238     """Compute MQM score from a list of errors.
239
240     Returns:
241         (mqm_score, total_penalty) where mqm_score = max(0, 100 - total_penalty)
242     """
243     total_penalty = 0.0
244     for err in errors:
245         cat = err.get("category", "")
246         sev = err.get("severity", "")
247         weight = MQM_WEIGHTS.get((cat, sev), 0.0)
248         total_penalty += weight
249     mqm_score = max(0.0, 100.0 - total_penalty)
250     return mqm_score, total_penalty
251
252
253 def validate_errors(errors: list[dict]) -> list[dict]:
254     """Normalize and validate error entries from the LLM judge."""
255     validated = []
256     for err in errors:
257         cat = err.get("category", "")
258         sub = err.get("sub_type", "")
259         sev = err.get("severity", "")
260
261         # Normalize category
262         if cat not in VALID_CATEGORIES:
263             for valid_cat in VALID_CATEGORIES:
264                 if valid_cat.lower() in cat.lower():
265                     cat = valid_cat
266                     break
267
268         # Normalize severity
269         if sev not in VALID_SEVERITIES:
270             sev = "Minor" # default to Minor if unclear
271
272         # Handle text_span
273         text_span = err.get("text_span")
274         if text_span is not None:
275             text_span = str(text_span).strip() if text_span else None
276         # Force null for non-annotatable sub-types
277         if sub in NON_ANNOTATABLE_SUB_TYPES:
278             text_span = None
279
280         validated.append({
281             "category": cat,
282             "sub_type": sub,
283             "severity": sev,
284             "weight": MQM_WEIGHTS.get((cat, sev), 0.0),
285             "text_span": text_span,
286             "evidence": err.get("evidence", ""),
287         })
288     return validated
289
290
291 def _build_context_text(annotation: str, caption: str = "", paper_title: str = "",
292                        figure_type: str = "", reference: str = "",
293                        generation_prompt: str = "") -> str:
294     """Build the user-facing context text for the judge."""
295     parts = []
296     if paper_title:
297         parts.append(f"Paper: {paper_title}")
298     if caption:
299         parts.append(f"Caption: {caption}")
300     if figure_type:
301         parts.append(f"Figure type: {figure_type}")
302     if generation_prompt:
303         parts.append(f"\n--- Generation prompt (given to the model) ---\n{generation_prompt}")
304     if reference:
305         parts.append(f"\n--- Reference description (human expert) ---\n{reference}")
306     parts.append(f"\n--- Machine-generated description to evaluate ---\n{annotation}")
307     return "\n".join(parts)
308
309
310 def _run_judge(client, judge_model: str, system_prompt: str,
311               user_content, max_tokens: int = 2048) -> dict:
312     """Call the judge LLM and return validated MQM result."""
313     def _call():
314         # Newer models (GPT-5.x+) use max_completion_tokens instead of max_tokens
315         token_param = "max_completion_tokens" if any(v in judge_model for v in ["gpt-5", "gpt-4.1"]) else "max_tokens"
316         response = client.chat.completions.create(
317             model=judge_model,
318             messages=[
319                 {"role": "system", "content": system_prompt},
320                 {"role": "user", "content": user_content},
321             ],
322             **{token_param: max_tokens},
323             temperature=0,
324             response_format={"type": "json_object"},
325         )
326         raw = response.choices[0].message.content.strip()
327         return _extract_json(raw)
328
329     result = _retry(_call)
330     errors = validate_errors(result.get("errors", []))
331     mqm_score, total_penalty = compute_mqm_score(errors)

```

```

332
333     return {
334         "errors": errors,
335         "mqm_score": round(mqm_score, 2),
336         "total_penalty": round(total_penalty, 2),
337         "error_count": len(errors),
338         "judge_model": judge_model,
339     }
340
341
342 def _run_judge_gemini(judge_model: str, system_prompt: str,
343                      user_content_parts, backend_config: dict,
344                      max_tokens: int = 32768) -> dict:
345     """Call a Gemini model via Vertex AI and return validated MQM result."""
346     from google.genai import types
347
348     client = get_vertex_client(backend_config, judge_model)
349
350     def _call():
351         config_kwargs = {
352             "system_instruction": system_prompt,
353             "max_output_tokens": max_tokens,
354             "response_mime_type": "application/json",
355             "temperature": 0,
356         }
357         # Thinking models (2.5 Pro, etc.) need a thinking config to avoid truncation
358         if "2.5" in judge_model or "2-5" in judge_model:
359             config_kwargs["thinking_config"] = types.ThinkingConfig(
360                 thinking_budget=4096,
361             )
362         response = client.models.generate_content(
363             model=judge_model,
364             contents=user_content_parts,
365             config=types.GenerateContentConfig(**config_kwargs),
366         )
367         raw = response.text.strip()
368         return _extract_json(raw)
369
370     result = _retry(_call)
371     errors = validate_errors(result.get("errors", []))
372     mqm_score, total_penalty = compute_mqm_score(errors)
373
374     return {
375         "errors": errors,
376         "mqm_score": round(mqm_score, 2),
377         "total_penalty": round(total_penalty, 2),
378         "error_count": len(errors),
379         "judge_model": judge_model,
380     }
381
382
383 def _get_judge_result(judge_model: str, system_prompt: str, text_content: str,
384                     image_path=None, max_tokens: int = 2048) -> dict:
385     """Unified judge dispatcher. Routes to the correct backend."""
386     backend_type, backend_config, model_id = resolve_backend(judge_model)
387
388     if backend_type == "openai_compat":
389         client = get_openai_compat_client(backend_config)
390         user_content = []
391         if image_path:
392             b64 = _encode_image(image_path)
393             user_content.append({
394                 "type": "image_url",
395                 "image_url": {"url": f"data:image/png;base64,{b64}", "detail": "high"},
396             })
397         user_content.append({"type": "text", "text": text_content})
398         return _run_judge(client, model_id, system_prompt, user_content, max_tokens)
399
400     elif backend_type == "vertex_ai":
401         from google.genai import types
402         parts = []
403         if image_path:
404             parts.append(types.Part.from_bytes(
405                 data=Path(image_path).read_bytes(), mime_type="image/png",
406             ))
407         parts.append(text_content)
408         # Vertex AI / Gemini needs higher token budget (thinking tokens are separate)
409         return _run_judge_gemini(model_id, system_prompt, parts, backend_config, max_tokens=8192)
410
411     else:
412         raise ValueError(f"Unknown backend type: {backend_type}")
413
414
415 # -----
416 # Reference-free
417 # -----
418
419 def evaluate_annotation(
420     image_path: Path,
421     annotation: str,
422     caption: str,
423     paper_title: str = "",
424     figure_type: str = "",
425     generation_prompt: str = "",
426     judge_model: str = "openai/gpt-4o-mini",
427     max_tokens: int = 2048,
428 ) -> dict:
429     """Reference-free evaluate annotation against the figure image + generation prompt."""
430     user_text = _build_context_text(annotation, caption, paper_title, figure_type,
431                                     generation_prompt=generation_prompt)
432     return _get_judge_result(judge_model, JUDGE_A_PROMPT, user_text,
433                             image_path=image_path, max_tokens=max_tokens)
434
435
436 # Alias for backward compatibility
437 evaluate_judge_a = evaluate_annotation
438
439
440 # -----
441 # Reference-only (no image)
442 # -----
443
444 def evaluate_judge_b(
445     annotation: str,
446     reference: str,
447     caption: str,
448     paper_title: str = "",
449     figure_type: str = "",
450     judge_model: str = "openai/gpt-4o-mini",
451     max_tokens: int = 2048,
452 ) -> dict:

```

```

453 """Reference-only evaluate annotation against the reference (no image)."""
454 user_text = _build_context_text(annotation, caption, paper_title, figure_type, reference)
455 return _get_judge_result(judge_model, JUDGE_B_PROMPT, user_text,
456                          max_tokens=max_tokens)
457
458 -----
459 #
460 # Reference-with-image (image + reference)
461 # -----
462
463 def evaluate_judge_c(
464     image_path: Path,
465     annotation: str,
466     reference: str,
467     caption: str,
468     paper_title: str = "",
469     figure_type: str = "",
470     judge_model: str = "openai/gpt-4o-mini",
471     max_tokens: int = 2048,
472 ) -> dict:
473     """Reference-with-image evaluate annotation against both the image and reference."""
474     user_text = _build_context_text(annotation, caption, paper_title, figure_type, reference)
475     return _get_judge_result(judge_model, JUDGE_C_PROMPT, user_text,
476                             image_path=image_path, max_tokens=max_tokens)

```

Listing 5: scripts/evaluation/mqm\_evaluator.py — Core MQM scoring and judge prompts

```

1 """Reference-free evaluation (image + generation prompt + model output, no groundtruth).
2
3 The judge examines the figure image and the generation prompt that was given
4 to the model. It evaluates accuracy against the image and completeness against
5 what the prompt requested. No human reference annotation is used.
6
7 Usage:
8     python3 scripts/evaluation/evaluate_reference_free.py <model_name> [--judge MODEL] [--workers N] [--subfolder X]
9 """
10
11 import argparse
12 import json
13 import logging
14 import sys
15 import threading
16 from concurrent.futures import ThreadPoolExecutor, as_completed
17 from pathlib import Path
18
19 ROOT = Path(__file__).resolve().parent.parent.parent
20 sys.path.insert(0, str(ROOT / "scripts"))
21
22 from evaluation.mqm_evaluator import evaluate_judge_a
23 from llm_generation.prompts import load_prompts, get_prompt
24
25 GENERATION_DIR = ROOT / "output" / "generation"
26 FIGURES_DIR = ROOT / "Dataset" / "figures"
27
28 logging.basicConfig(level=logging.INFO, format="%(asctime)s %(levelname)s %(message)s", datefmt="%H:%M:%S")
29 logger = logging.getLogger(__name__)
30
31 ALL_SUBFOLDERS = ["bulgarian_only", "chinese_only", "english_only", "german_only", "multi_language"]
32
33 SUBFOLDER_TO_LANGUAGE = {
34     "bulgarian_only": "Bulgarian",
35     "chinese_only": "Chinese",
36     "english_only": "English",
37     "german_only": "German",
38 }
39
40 # Load all prompts once at module level
41 _PROMPTS = load_prompts()
42
43
44 def _judge_slug(judge_model):
45     """Convert judge model ID to a filesystem-safe slug. e.g. 'openai/gpt-4o-mini' -> 'gpt-4o-mini'."""
46     return judge_model.split("/")[-1]
47
48
49 def _output_dir(judge_model):
50     return ROOT / "output" / "evaluation" / "reference_free" / _judge_slug(judge_model)
51
52
53 def _load_samples(samples_path):
54     """Load samples.json and return a dict of {subfolder: set(fig_keys)}."""
55     with open(samples_path) as f:
56         data = json.load(f)
57     result = {}
58     for subfolder, types in data["samples"].items():
59         keys = set()
60         for fig_type, figs in types.items():
61             keys.update(figs)
62         result[subfolder] = keys
63     return result
64
65
66 def _discover(subfolder, model_name, sample_keys=None):
67     folder = GENERATION_DIR / model_name / subfolder
68     keys = sorted(p.stem for p in folder.glob("*.json")) if folder.exists() else []
69     if sample_keys is not None and subfolder in sample_keys:
70         keys = [k for k in keys if k in sample_keys[subfolder]]
71     return keys
72
73
74 def _get_generation_prompt(language, figure_type):
75     """Reconstruct the generation prompt that was used for a given language + figure type."""
76     try:
77         return get_prompt(_PROMPTS, language, figure_type)
78     except ValueError:
79         logger.warning(f" No prompt found for {language}/{figure_type}, using empty")
80         return ""
81
82
83 def _process(fig_key, subfolder, model_name, judge_model):
84     gen_path = GENERATION_DIR / model_name / subfolder / f"{fig_key}.json"
85     fig_path = FIGURES_DIR / subfolder / f"{fig_key}.png"
86     out_path = _output_dir(judge_model) / model_name / subfolder / f"{fig_key}.json"
87
88     if out_path.exists():
89         return True, fig_key, "skip"
90     if not gen_path.exists() or not fig_path.exists():
91         return True, fig_key, "skip"

```

```

92
93 with open(gen_path) as f:
94     gen = json.load(f)
95
96 figure_type = gen.get("figure_type", "")
97
98 if "model_annotations" in gen:
99     mqm_by_lang = {}
100     for lang, annotation in sorted(gen["model_annotations"].items()):
101         gen_prompt = _get_generation_prompt(lang, figure_type)
102         try:
103             result = evaluate_judge_a(
104                 image_path=fig_path, annotation=annotation,
105                 caption=gen.get("caption", ""), paper_title=gen.get("paper_title", ""),
106                 figure_type=figure_type, generation_prompt=gen_prompt,
107                 judge_model=judge_model,
108             )
109             mqm_by_lang[lang] = result
110         except Exception as e:
111             logger.error(f" FAIL {fig_key} ({lang}): {e}")
112             return False, fig_key, str(e)
113
114 avg_mqm = sum(r["mqm_score"] for r in mqm_by_lang.values()) / len(mqm_by_lang)
115 eval_result = {
116     "figure_key": fig_key, "model_name": gen.get("model_name", model_name),
117     "figure_type": figure_type, "judge_type": "reference_free",
118     "judge_model": judge_model,
119     "mqm_by_language": mqm_by_lang, "mqm_score_avg": round(avg_mqm, 2),
120 }
121 logger.info(f" OK {fig_key}: avg MQM={avg_mqm:.1f} ({len(mqm_by_lang)} langs)")
122 else:
123     annotation = gen.get("model_annotation", "")
124     lang = SUBFOLDER_TO_LANGUAGE.get(subfolder, gen.get("paper_language", "English"))
125     gen_prompt = _get_generation_prompt(lang, figure_type)
126     try:
127         result = evaluate_judge_a(
128             image_path=fig_path, annotation=annotation,
129             caption=gen.get("caption", ""), paper_title=gen.get("paper_title", ""),
130             figure_type=figure_type, generation_prompt=gen_prompt,
131             judge_model=judge_model,
132         )
133     except Exception as e:
134         logger.error(f" FAIL {fig_key}: {e}")
135         return False, fig_key, str(e)
136
137 eval_result = {
138     "figure_key": fig_key, "model_name": gen.get("model_name", model_name),
139     "figure_type": figure_type, "judge_type": "reference_free",
140     "judge_model": judge_model,
141     **result,
142 }
143 logger.info(f" OK {fig_key}: MQM={result['mqm_score']}, {result['error_count']} errors")
144
145 out_path.parent.mkdir(parents=True, exist_ok=True)
146 with open(out_path, "w") as f:
147     json.dump(eval_result, f, indent=2, ensure_ascii=False)
148 return True, fig_key, "done"
149
150
151 def run(model_name, judge_model="openai/gpt-4o-mini", workers=1, subfolder_filter=None, samples_path=None):
152     subfolders = [subfolder_filter] if subfolder_filter else ALL_SUBFOLDERS
153     sample_keys = _load_samples(samples_path) if samples_path else None
154     out_base = _output_dir(judge_model)
155     logger.info(f"Reference-free | model={model_name} | judge={judge_model} | workers={workers}")
156     if samples_path:
157         logger.info(f"Samples filter: {samples_path}")
158     logger.info(f"Output: {out_base / model_name}")
159
160     work_items, skipped = [], 0
161     for sub in subfolders:
162         (out_base / model_name / sub).mkdir(parents=True, exist_ok=True)
163         for fig_key in _discover(sub, model_name, sample_keys):
164             if (out_base / model_name / sub / f"{fig_key}.json").exists():
165                 skipped += 1
166             elif (FIGURES_DIR / sub / f"{fig_key}.png").exists():
167                 work_items.append((fig_key, sub))
168
169     logger.info(f"Total: {len(work_items) + skipped} ({skipped} done, {len(work_items)} to evaluate)")
170     if not work_items:
171         logger.info("Nothing to do.")
172     return
173
174 success, errors, lock = skipped, 0, threading.Lock()
175
176 def _on_done(ok, k, s):
177     nonlocal success, errors
178     with lock:
179         if ok: success += 1
180         else: errors += 1
181         d = success + errors - skipped
182         if d % 10 == 0 or not ok:
183             logger.info(f" Progress: {d}/{len(work_items)} (errors={errors})")
184
185 if workers == 1:
186     for fk, sub in work_items:
187         ok, k, s = _process(fk, sub, model_name, judge_model)
188         _on_done(ok, k, s)
189 else:
190     with ThreadPoolExecutor(max_workers=workers) as ex:
191         futs = [ex.submit(_process, fk, sub, model_name, judge_model) for fk, sub in work_items]
192         for f in as_completed(futs):
193             try:
194                 ok, k, s = f.result()
195                 _on_done(ok, k, s)
196             except Exception as e:
197                 logger.error(f" UNEXPECTED {futs[f]}: {e}")
198                 with lock: errors += 1
199
200 logger.info(f"Done. {success}/{len(work_items)+skipped} evaluated, {errors} failures.")
201
202
203 if __name__ == "__main__":
204     p = argparse.ArgumentParser(description="Reference-free MQM evaluation (image + generation prompt, no groundtruth)")
205     p.add_argument("model", help="Model name to evaluate")
206     p.add_argument("--judge", default="openai/gpt-4o-mini", help="Judge model")
207     p.add_argument("--workers", type=int, default=1)
208     p.add_argument("--subfolder", default=None)
209     p.add_argument("--samples", default=None, help="Path to samples.json to filter figures")
210     a = p.parse_args()
211     run(a.model, a.judge, a.workers, a.subfolder, a.samples)

```



## Listing 6: scripts/evaluation/evaluate\_reference\_free.py — Judge A: image-only evaluation

```

1  """Reference-with-image evaluation (image + groundtruth + model output).
2
3  The judge has access to both the figure image and the human expert reference
4  annotation, providing the most comprehensive evaluation context.
5
6  Usage:
7  python3 scripts/evaluation/evaluate_reference_with_image.py <model_name> [--judge MODEL] [--workers N] [--subfolder X]
8  """
9
10 import argparse
11 import json
12 import logging
13 import sys
14 import threading
15 from concurrent.futures import ThreadPoolExecutor, as_completed
16 from pathlib import Path
17
18 ROOT = Path(__file__).resolve().parent.parent.parent
19 sys.path.insert(0, str(ROOT / "scripts"))
20
21 from evaluation.mqgm_evaluator import evaluate_judge_c
22
23 GENERATION_DIR = ROOT / "output" / "generation"
24 GROUNDTRUTH_DIR = ROOT / "Dataset" / "groundtruth"
25 FIGURES_DIR = ROOT / "Dataset" / "figures"
26
27 logging.basicConfig(level=logging.INFO, format="%(asctime)s %(levelname)s %(message)s", datefmt="%H:%M:%S")
28 logger = logging.getLogger(__name__)
29
30 ALL_SUBFOLDERS = ["bulgarian_only", "chinese_only", "english_only", "german_only", "multi_language"]
31
32 SUBFOLDER_TO_LANGUAGE = {
33     "bulgarian_only": "Bulgarian",
34     "chinese_only": "Chinese",
35     "english_only": "English",
36     "german_only": "German",
37 }
38
39
40 def _judge_slug(judge_model):
41     """Convert judge model ID to a filesystem-safe slug."""
42     return judge_model.split("/")[-1]
43
44
45 def _output_dir(judge_model):
46     return ROOT / "output" / "evaluation" / "reference_with_image" / _judge_slug(judge_model)
47
48
49 def _load_samples(samples_path):
50     """Load samples.json and return a dict of {subfolder: set(fig_keys)}."""
51     with open(samples_path) as f:
52         data = json.load(f)
53     result = {}
54     for subfolder, types in data["samples"].items():
55         keys = set()
56         for fig_type, figs in types.items():
57             keys.update(figs)
58     result[subfolder] = keys
59     return result
60
61
62 def _discover(subfolder, model_name, sample_keys=None):
63     folder = GENERATION_DIR / model_name / subfolder
64     keys = sorted(p.stem for p in folder.glob("*.json")) if folder.exists() else []
65     if sample_keys is not None and subfolder in sample_keys:
66         keys = [k for k in keys if k in sample_keys[subfolder]]
67     return keys
68
69
70 def _get_reference(gt, lang):
71     """Extract reference annotation for a given language from groundtruth."""
72     for a in gt.get("annotations", []):
73         if a.get("annotation_language") == lang:
74             return a.get("annotation", "")
75     return ""
76
77
78 def _process(fig_key, subfolder, model_name, judge_model):
79     gen_path = GENERATION_DIR / model_name / subfolder / f"{fig_key}.json"
80     gt_path = GROUNDTRUTH_DIR / subfolder / f"{fig_key}.json"
81     fig_path = FIGURES_DIR / subfolder / f"{fig_key}.png"
82     out_path = _output_dir(judge_model) / model_name / subfolder / f"{fig_key}.json"
83
84     if out_path.exists():
85         return True, fig_key, "skip"
86     if not gen_path.exists() or not gt_path.exists() or not fig_path.exists():
87         return True, fig_key, "skip"
88
89     with open(gen_path) as f:
90         gen = json.load(f)
91     with open(gt_path) as f:
92         gt = json.load(f)
93
94     if "model_annotations" in gen:
95         mqgm_by_lang = {}
96         for lang, annotation in sorted(gen["model_annotations"].items()):
97             reference = _get_reference(gt, lang)
98             if not reference:
99                 logger.warning(f" SKIP {fig_key} ({lang}) no reference annotation")
100                 continue
101             try:
102                 result = evaluate_judge_c(
103                     image_path=fig_path, annotation=annotation, reference=reference,
104                     caption=gen.get("caption", ""), paper_title=gen.get("paper_title", ""),
105                     figure_type=gen.get("figure_type", ""), judge_model=judge_model,
106                 )
107                 mqgm_by_lang[lang] = result
108             except Exception as e:
109                 logger.error(f" FAIL {fig_key} ({lang}): {e}")
110                 return False, fig_key, str(e)
111
112     if not mqgm_by_lang:
113         return True, fig_key, "skip"
114
115     avg_mqgm = sum(r["mqgm_score"] for r in mqgm_by_lang.values()) / len(mqgm_by_lang)

```

```

116     eval_result = {
117         "figure_key": fig_key, "model_name": gen.get("model_name", model_name),
118         "figure_type": gen.get("figure_type", ""), "judge_type": "reference_with_image",
119         "judge_model": judge_model,
120         "mqm_by_language": mqm_by_lang, "mqm_score_avg": round(avg_mqm, 2),
121     }
122     logger.info(f" OK   {fig_key}: avg MQM={avg_mqm:.1f} ({len(mqm_by_lang)} langs)")
123 else:
124     annotation = gen.get("model_annotation", "")
125     lang = SUBFOLDER_TO_LANGUAGE.get(subfolder, gt.get("paper_language", "English"))
126     reference = _get_reference(gt, lang)
127     if not reference:
128         return True, fig_key, "skip"
129
130     try:
131         result = evaluate_judge_c(
132             image_path=fig_path, annotation=annotation, reference=reference,
133             caption=gen.get("caption", ""), paper_title=gen.get("paper_title", ""),
134             figure_type=gen.get("figure_type", ""), judge_model=judge_model,
135         )
136     except Exception as e:
137         logger.error(f" FAIL {fig_key}: {e}")
138         return False, fig_key, str(e)
139
140     eval_result = {
141         "figure_key": fig_key, "model_name": gen.get("model_name", model_name),
142         "figure_type": gen.get("figure_type", ""), "judge_type": "reference_with_image",
143         "judge_model": judge_model,
144         **result,
145     }
146     logger.info(f" OK   {fig_key}: MQM={result['mqm_score']}, {result['error_count']} errors")
147
148 out_path.parent.mkdir(parents=True, exist_ok=True)
149 with open(out_path, "w") as f:
150     json.dump(eval_result, f, indent=2, ensure_ascii=False)
151 return True, fig_key, "done"
152
153
154 def run(model_name, judge_model="openai/gpt-4o-mini", workers=1, subfolder_filter=None, samples_path=None):
155     subfolders = [subfolder_filter] if subfolder_filter else ALL_SUBFOLDERS
156     sample_keys = _load_samples(samples_path) if samples_path else None
157     out_base = _output_dir(judge_model)
158     logger.info(f"Reference-with-image | model={model_name} | judge={judge_model} | workers={workers}")
159     if samples_path:
160         logger.info(f"Samples filter: {samples_path}")
161     logger.info(f"Output: {out_base / model_name}")
162
163     work_items, skipped = [], 0
164     for sub in subfolders:
165         (out_base / model_name / sub).mkdir(parents=True, exist_ok=True)
166         for fig_key in _discover(sub, model_name, sample_keys):
167             out_path = out_base / model_name / sub / f"{fig_key}.json"
168             gt_path = GROUNDTRUTH_DIR / sub / f"{fig_key}.json"
169             fig_path = FIGURES_DIR / sub / f"{fig_key}.png"
170             if out_path.exists():
171                 skipped += 1
172             elif gt_path.exists() and fig_path.exists():
173                 work_items.append((fig_key, sub))
174
175     logger.info(f"Total: {len(work_items) + skipped} ({skipped} done, {len(work_items)} to evaluate)")
176     if not work_items:
177         logger.info("Nothing to do.")
178         return
179
180     success, errors, lock = skipped, 0, threading.Lock()
181
182     def _on_done(ok, k, s):
183         nonlocal success, errors
184         with lock:
185             if ok: success += 1
186             else: errors += 1
187             d = success + errors - skipped
188             if d % 10 == 0 or not ok:
189                 logger.info(f" Progress: {d}/{len(work_items)} (errors={errors})")
190
191     if workers == 1:
192         for fk, sub in work_items:
193             ok, k, s = _process(fk, sub, model_name, judge_model)
194             _on_done(ok, k, s)
195     else:
196         with ThreadPoolExecutor(max_workers=workers) as ex:
197             futs = [ex.submit(_process, fk, sub, model_name, judge_model) for fk, sub in work_items]
198             for f in as_completed(futs):
199                 try:
200                     ok, k, s = f.result()
201                     _on_done(ok, k, s)
202                 except Exception as e:
203                     logger.error(f" UNEXPECTED {futs[f]}: {e}")
204                     with lock: errors += 1
205
206     logger.info(f"Done. {success}/{len(work_items)+skipped} evaluated, {errors} failures.")
207
208
209 if __name__ == "__main__":
210     p = argparse.ArgumentParser(description="Reference-with-image MQM evaluation (image + groundtruth)")
211     p.add_argument("model", help="Model name to evaluate")
212     p.add_argument("--judge", default="openai/gpt-4o-mini", help="Judge model")
213     p.add_argument("--workers", type=int, default=1)
214     p.add_argument("--subfolder", default=None)
215     p.add_argument("--samples", default=None, help="Path to samples.json to filter figures")
216     a = p.parse_args()
217     run(a.model, a.judge, a.workers, a.subfolder, a.samples)

```

Listing 7: scripts/evaluation/evaluate\_reference\_with\_image.py — Judge C: image + groundtruth

```

1 """Backend routing for LLM judge models.
2
3 Resolves a judge model identifier (e.g. "openai/gpt-4o-mini", "azure/gpt-4o",
4 "gemini-3.1-pro") to the correct backend and client configuration.
5
6 Configuration is loaded from judge_backends.json in the same directory.
7 """
8
9 import json
10 import os
11 import logging
12 from pathlib import Path
13

```

```

14 logger = logging.getLogger(__name__)
15
16 _CONFIG_PATH = Path(__file__).parent / "judge_backends.json"
17 _config = None
18
19
20 def _load_config():
21     global _config
22     if _config is None:
23         with open(_CONFIG_PATH) as f:
24             _config = json.load(f)
25     return _config
26
27
28 def resolve_backend(judge_model: str) -> tuple:
29     """Resolve a judge model string to (backend_type, backend_config, model_id).
30
31     Routes are matched by prefix in order. The first match wins.
32     If strip_prefix is true, the prefix is removed from the model_id
33     passed to the API.
34     """
35     config = _load_config()
36
37     for route in config["routes"]:
38         prefix = route["prefix"]
39         if prefix == "*" or judge_model.startswith(prefix):
40             backend_name = route["backend"]
41             backend_config = config["backends"][backend_name]
42             model_id = judge_model[len(prefix):] if route.get("strip_prefix") else judge_model
43             return backend_config["type"], backend_config, model_id
44
45     raise ValueError(f"No backend route matched for judge model: {judge_model}")
46
47
48 def get_openai_compat_client(backend_config: dict):
49     """Create an OpenAI-compatible client (works for OpenRouter and Azure)."""
50     api_key = os.environ.get(backend_config["api_key_env"])
51     if not api_key:
52         raise EnvironmentError(
53             f"{backend_config['api_key_env']} environment variable is not set."
54         )
55
56     if backend_config.get("azure"):
57         from openai import AzureOpenAI
58         endpoint = os.environ.get(backend_config["endpoint_env"])
59         if not endpoint:
60             raise EnvironmentError(
61                 f"{backend_config['endpoint_env']} environment variable is not set."
62             )
63         return AzureOpenAI(
64             azure_endpoint=endpoint,
65             api_key=api_key,
66             api_version=backend_config.get("api_version", "2024-12-01-preview"),
67         )
68     else:
69         from openai import OpenAI
70         return OpenAI(
71             base_url=backend_config["base_url"],
72             api_key=api_key,
73         )
74
75
76 def get_vertex_client(backend_config: dict, model: str = ""):
77     """Create a Google GenAI / Vertex AI client."""
78     from google import genai
79     project = os.environ.get(
80         backend_config.get("project_env", ""),
81         backend_config.get("project_default", ""),
82     )
83     location = backend_config.get("location", "us-central1")
84     # Gemini 3+ preview models require location='global'
85     if "preview" in model or "gemini-3" in model:
86         location = "global"
87     return genai.Client(vertexai=True, project=project, location=location)

```

Listing 8: scripts/evaluation/judge\_backends.py — Backend routing (Azure/OpenRouter/Vertex)

## Adversarial Experiments

```

1 """Generate figure descriptions for adversarial-transformed images.
2
3 Runs any registered model on transformed versions of the 45 adversarial figures.
4 Uses the model registry from models.py (OpenRouter, Vertex AI, Azure, etc.).
5
6 Output: output/experiments/transforms/<model>/<transform>/<subfolder>/<fig_key>.json
7
8 Usage:
9 python3 scripts/experiments/run_transforms.py gpt-5.2 --workers 4
10 python3 scripts/experiments/run_transforms.py llama4-maverick --transform selective_blur
11 python3 scripts/experiments/run_transforms.py --list-models
12 """
13
14 import argparse
15 import json
16 import logging
17 import sys
18 import threading
19 from concurrent.futures import ThreadPoolExecutor, as_completed
20 from pathlib import Path
21
22 from dotenv import load_dotenv
23
24 ROOT = Path(__file__).resolve().parent.parent.parent
25 load_dotenv(ROOT / ".env")
26 sys.path.insert(0, str(ROOT / "scripts"))
27
28 from llm_generation.models import get_annotator, _get_all_models
29 from llm_generation.prompts import load_prompts, get_prompt
30
31 ADVERSARIAL_DIR = ROOT / "adversarial_experiments"
32 SAMPLES_FILE = ADVERSARIAL_DIR / "samples.json"
33 FIGURES_DIR = ADVERSARIAL_DIR / "figures"
34 GROUNDTRUTH_DIR = ROOT / "Dataset" / "groundtruth"

```

```

35 OUTPUT_DIR = ROOT / "output" / "experiments" / "transforms"
36
37 ALL_TRANSFORMS = [
38     "original",
39     "jpeg_compression",
40     "noise",
41     "aspect_ratio",
42     "low_contrast",
43     "rotation",
44     "axis_blurred",
45     "selective_blur",
46     "blurred_in_paper",
47 ]
48
49 logging.basicConfig(level=logging.INFO, format="%(asctime)s %(levelname)s %(message)s", datefmt="%H:%M:%S")
50 logger = logging.getLogger(__name__)
51
52
53 def _get_transform_image_path(subfolder, fig_key, transform):
54     """Get the path to a transformed image."""
55     if transform == "original":
56         return FIGURES_DIR / subfolder / fig_key / "original.png"
57     return FIGURES_DIR / subfolder / fig_key / "transforms" / f"{transform}.png"
58
59
60 def _process_figure(annotator, prompts, transform, subfolder, fig_key):
61     """Process a single transformed figure."""
62     fig_path = _get_transform_image_path(subfolder, fig_key, transform)
63     gt_path = GROUNDTRUTH_DIR / subfolder / f"{fig_key}.json"
64     out_path = OUTPUT_DIR / annotator.model_name / transform / subfolder / f"{fig_key}.json"
65
66     if out_path.exists():
67         return True, fig_key, "skip"
68
69     if not fig_path.exists():
70         logger.warning(f" SKIP {transform}/{subfolder}/{fig_key} no transformed image")
71         return True, fig_key, "skip"
72     if not gt_path.exists():
73         logger.warning(f" SKIP {transform}/{subfolder}/{fig_key} no groundtruth")
74         return True, fig_key, "skip"
75
76     with open(gt_path) as f:
77         gt = json.load(f)
78
79     figure_type = gt["figure_type"]
80
81     if subfolder == "multi_language":
82         languages = sorted(set(a["annotation_language"] for a in gt["annotations"]))
83     else:
84         languages = [gt["paper_language"]]
85
86     try:
87         if len(languages) == 1:
88             prompt_text = get_prompt(prompts, languages[0], figure_type)
89             description = annotator.annotate_figure(
90                 prompt=prompt_text,
91                 image_path=fig_path,
92                 caption="",
93                 paper_title="",
94             )
95
96             result = {
97                 "figure_key": fig_key,
98                 "task_id": gt["task_id"],
99                 "arxiv_id": gt.get("arxiv_id"),
100                 "paper_language": gt["paper_language"],
101                 "figure_type": figure_type,
102                 "model_name": annotator.model_name,
103                 "transform": transform,
104                 "condition": "image_only",
105                 "model_annotation": description,
106             }
107         else:
108             annotations_by_lang = {}
109             for lang in languages:
110                 prompt_text = get_prompt(prompts, lang, figure_type)
111                 desc = annotator.annotate_figure(
112                     prompt=prompt_text,
113                     image_path=fig_path,
114                     caption="",
115                     paper_title="",
116                 )
117                 annotations_by_lang[lang] = desc
118
119             result = {
120                 "figure_key": fig_key,
121                 "task_id": gt["task_id"],
122                 "arxiv_id": gt.get("arxiv_id"),
123                 "paper_language": gt["paper_language"],
124                 "figure_type": figure_type,
125                 "model_name": annotator.model_name,
126                 "transform": transform,
127                 "condition": "image_only",
128                 "model_annotations": annotations_by_lang,
129             }
130
131     out_path.parent.mkdir(parents=True, exist_ok=True)
132     with open(out_path, "w") as f:
133         json.dump(result, f, indent=2, ensure_ascii=False)
134
135     logger.info(f" OK {transform}/{subfolder}/{fig_key}")
136     return True, fig_key, "done"
137
138 except Exception as e:
139     logger.error(f" FAIL {transform}/{subfolder}/{fig_key}: {e}")
140     return False, fig_key, str(e)
141
142
143 def run(model_name="gpt-5.2", workers=4, transform_filter=None, subfolder_filter=None):
144     with open(SAMPLES_FILE) as f:
145         adv = json.load(f)
146
147     transforms = list(ALL_TRANSFORMS)
148     if transform_filter:
149         transforms = [t for t in transforms if t == transform_filter]
150     if not transforms:
151         logger.error(f"Unknown transform: {transform_filter}. Available: {ALL_TRANSFORMS}")
152         return
153
154     annotator = get_annotator(model_name)
155     prompts = load_prompts()

```

```

156
157 # Build work items
158 work_items = []
159 skipped = 0
160 for transform in transforms:
161     for subfolder, fig_keys in adv["samples"].items():
162         if subfolder_filter and subfolder != subfolder_filter:
163             continue
164         for fig_key in fig_keys:
165             out_path = OUTPUT_DIR / annotator.model_name / transform / subfolder / f"{fig_key}.json"
166             fig_path = _get_transform_image_path(subfolder, fig_key, transform)
167             if out_path.exists():
168                 skipped += 1
169             elif fig_path.exists():
170                 work_items.append((transform, subfolder, fig_key))
171
172 total = len(work_items) + skipped
173 logger.info(f"Transform generation | model={annotator.model_name}")
174 logger.info(f"Transforms: {transforms}")
175 logger.info(f"Total: {total} ({skipped} done, {len(work_items)} to generate)")
176
177 if not work_items:
178     logger.info("Nothing to do.")
179     return
180
181 success, errors = skipped, 0
182 lock = threading.Lock()
183
184 def _on_done(ok, k, s):
185     nonlocal success, errors
186     with lock:
187         if ok:
188             success += 1
189         else:
190             errors += 1
191     d = success + errors - skipped
192     if d % 10 == 0 or not ok:
193         logger.info(f" Progress: {d}/{len(work_items)} (errors={errors})")
194
195 if workers == 1:
196     for t, sub, fk in work_items:
197         ok, k, s = _process_figure(annotator, prompts, t, sub, fk)
198         _on_done(ok, k, s)
199 else:
200     with ThreadPoolExecutor(max_workers=workers) as ex:
201         futs = {
202             ex.submit(_process_figure, annotator, prompts, t, sub, fk): fk
203             for t, sub, fk in work_items
204         }
205         for f in as_completed(futs):
206             try:
207                 ok, k, s = f.result()
208                 _on_done(ok, k, s)
209             except Exception as e:
210                 logger.error(f" UNEXPECTED {futs[f]}: {e}")
211                 with lock:
212                     errors += 1
213
214 logger.info(f"Done. {success}/{total} generated, {errors} failures.")
215
216
217 if __name__ == "__main__":
218     p = argparse.ArgumentParser(description="Generate descriptions for adversarial-transformed figures")
219     p.add_argument("model", nargs="?", default="gpt-5.2", help="Model name")
220     p.add_argument("--workers", type=int, default=4)
221     p.add_argument("--transform", default=None, help="Filter to specific transform")
222     p.add_argument("--subfolder", default=None, help="Filter to specific subfolder")
223     p.add_argument("--list-models", action="store_true", help="List available models and exit")
224     a = p.parse_args()
225
226     if a.list_models:
227         models = sorted(_get_all_models().keys())
228         print(f"Available models ({len(models)}):")
229         for m in models:
230             print(f" {m}")
231         sys.exit(0)
232
233     run(a.model, a.workers, a.transform, a.subfolder)

```

Listing 9: scripts/experiments/run\_transforms.py — Image transform experiment runner

```

1 """Run prompt reverse experiment with forced A/B JSON responses.
2
3 For each figure, sends two separate prompts (true statement and false statement)
4 with the image. Model must respond in JSON with answer A (True) or B (False).
5
6 Output: output/experiments/prompt_reverse/{model}/{subfolder}/{fig_key}.json
7
8 Usage:
9   python3 scripts/experiments/run_prompt_reverse.py gpt-5.2 --workers 4
10   python3 scripts/experiments/run_prompt_reverse.py --list-models
11 """
12
13 import argparse
14 import json
15 import logging
16 import sys
17 import threading
18 import time
19 from concurrent.futures import ThreadPoolExecutor, as_completed
20 from datetime import datetime, timezone
21 from pathlib import Path
22
23 from dotenv import load_dotenv
24
25 ROOT = Path(__file__).resolve().parent.parent.parent
26 load_dotenv(ROOT / ".env")
27 sys.path.insert(0, str(ROOT / "scripts"))
28
29 from llm_generation.models import get_annotator, _get_all_models
30
31 BENCHMARKS_DIR = ROOT / "adversarial_experiments" / "benchmarks"
32 FIGURES_DIR = ROOT / "adversarial_experiments" / "figures"
33 SAMPLES_FILE = ROOT / "adversarial_experiments" / "samples.json"
34 OUTPUT_DIR = ROOT / "output" / "experiments" / "prompt_reverse"
35
36 SUBFOLDER_TO_LANG_KEY = {
37     "bulgarian_only": "bulgarian",

```

```

38     "chinese_only": "chinese",
39     "english_only": "english",
40     "german_only": "german",
41     "multi_language": "multi_language",
42 }
43
44 PROMPT_TEMPLATE = """Statement: {statement}
45
46 Is this statement true or false based on the figure?
47
48 Respond ONLY in JSON: ({"answer": "A", "explanation": "max 10 words"})
49 A = True, B = False"""
50
51 logging.basicConfig(level=logging.INFO, format="%(asctime)s %(levelname)s %(message)s", datefmt="%H:%M:%S")
52 logger = logging.getLogger(__name__)
53
54
55 def _load_prompt_reverse_probes():
56     """Load all prompt reverse pairs."""
57     probes = {}
58     pr_dir = BENCHMARKS_DIR / "adversarial" / "prompt_reverse" / "native"
59     for lang_file in pr_dir.glob("*.json"):
60         with open(lang_file) as f:
61             data = json.load(f)
62             for fig_key, fig_data in data.items():
63                 probes[fig_key] = fig_data.get("pair", {})
64     return probes
65
66
67 def _parse_ab_response(raw_response):
68     """Parse A or B from JSON response."""
69     text = raw_response.strip()
70     # Try JSON parse
71     try:
72         if text.startswith("```"):
73             text = text.split("```")[1]
74             if text.startswith("json"):
75                 text = text[4:]
76             data = json.loads(text)
77             return data.get("answer", "").strip().upper()[0:1], data.get("explanation", "")
78     except (json.JSONDecodeError, AttributeError):
79         pass
80     # Fallback: look for A or B in first few chars
81     first = text[:10].strip().upper()
82     if first.startswith("A"):
83         return "A", text
84     if first.startswith("B"):
85         return "B", text
86     return "", text
87
88
89 def _process_figure(annotator, subfolder, fig_key, pair):
90     """Run both true and false prompts for one figure."""
91     fig_path = FIGURES_DIR / subfolder / fig_key / "original.png"
92     out_path = OUTPUT_DIR / annotator.model_name / subfolder / f"{fig_key}.json"
93
94     if out_path.exists():
95         return True, fig_key, "skip"
96
97     if not fig_path.exists():
98         return False, fig_key, f"no image: {fig_path}"
99
100     true_stmt = pair.get("true_statement", "")
101     false_stmt = pair.get("false_statement", "")
102
103     if not true_stmt or not false_stmt:
104         return True, fig_key, "skip_no_pair"
105
106     results = {}
107     for label, statement, expected in [("true", true_stmt, "A"), ("false", false_stmt, "B")]:
108         prompt = PROMPT_TEMPLATE.format(statement=statement)
109         try:
110             raw = annotator.annotate_figure(
111                 prompt=prompt,
112                 image_path=fig_path,
113                 caption="",
114                 paper_title="",
115             )
116             answer, explanation = _parse_ab_response(raw)
117             results[label] = {
118                 "statement": statement,
119                 "expected": expected,
120                 "raw_response": raw,
121                 "answer": answer,
122                 "explanation": explanation,
123                 "correct": answer == expected,
124             }
125         except Exception as e:
126             logger.error(f"FAIL {fig_key} [{label}]: {e}")
127             results[label] = {
128                 "statement": statement,
129                 "expected": expected,
130                 "raw_response": "",
131                 "answer": "",
132                 "explanation": str(e),
133                 "correct": False,
134             }
135
136     # Compute consistency score
137     true_correct = results.get("true", {}).get("correct", False)
138     false_correct = results.get("false", {}).get("correct", False)
139     true_answer = results.get("true", {}).get("answer", "")
140     false_answer = results.get("false", {}).get("answer", "")
141
142     if true_correct and false_correct:
143         score = 1.0
144         pattern = "correct"
145     elif true_answer == false_answer:
146         score = 0.0
147         pattern = "agrees_both" if true_answer == "A" else "disagrees_both"
148     elif not true_correct and not false_correct:
149         score = 0.5
150         pattern = "reversed"
151     else:
152         score = 0.0
153         pattern = "mixed"
154
155     output = {
156         "figure_key": fig_key,
157         "subfolder": subfolder,
158         "model_name": annotator.model_name,

```

```

159     "timestamp": datetime.now(timezone.utc).isoformat(),
160     "fact_description": pair.get("fact_description", ""),
161     "true_probe": results.get("true", {}),
162     "false_probe": results.get("false", {}),
163     "score": score,
164     "pattern": pattern,
165 }
166
167 out_path.parent.mkdir(parents=True, exist_ok=True)
168 with open(out_path, "w") as f:
169     json.dump(output, f, indent=2, ensure_ascii=False)
170
171 logger.info(f" OK   {subfolder}/{fig_key}      T={true_answer} F={false_answer} score={score} ({pattern})")
172 return True, fig_key, "done"
173
174
175 def run(model_name="gpt-5.2", workers=4, subfolder_filter=None):
176     with open(SAMPLES_FILE) as f:
177         samples = json.load(f)["samples"]
178
179     annotator = get_annotator(model_name)
180     probes = _load_prompt_reverse_probes()
181
182     work_items = []
183     skipped = 0
184     for subfolder, fig_keys in samples.items():
185         if subfolder_filter and subfolder != subfolder_filter:
186             continue
187         for fig_key in fig_keys:
188             out_path = OUTPUT_DIR / annotator.model_name / subfolder / f"{fig_key}.json"
189             if out_path.exists():
190                 skipped += 1
191             elif fig_key in probes:
192                 work_items.append((subfolder, fig_key, probes[fig_key]))
193
194     logger.info(f"Prompt reverse | model={annotator.model_name}")
195     logger.info(f"Total: {len(work_items) + skipped} ({skipped} done, {len(work_items)} to run)")
196
197     if not work_items:
198         logger.info("Nothing to do.")
199         return
200
201     success, errors = skipped, 0
202     lock = threading.Lock()
203
204     def _on_done(ok, k, s):
205         nonlocal success, errors
206         with lock:
207             if ok:
208                 success += 1
209             else:
210                 errors += 1
211
212     if workers == 1:
213         for sub, fk, pair in work_items:
214             ok, k, s = _process_figure(annotator, sub, fk, pair)
215             _on_done(ok, k, s)
216     else:
217         with ThreadPoolExecutor(max_workers=workers) as ex:
218             futs = {}
219             ex.submit(_process_figure, annotator, sub, fk, pair): fk
220             for sub, fk, pair in work_items:
221                 futs[sub, fk, pair] = ex.submit(_process_figure, annotator, sub, fk, pair)
222             for f in as_completed(futs):
223                 try:
224                     ok, k, s = f.result()
225                     _on_done(ok, k, s)
226                 except Exception as e:
227                     logger.error(f" UNEXPECTED: {e}")
228                     with lock:
229                         errors += 1
230
231     # Summary
232     print(f"\n=== SUMMARY ({annotator.model_name}) ===")
233     eval_dir = OUTPUT_DIR / annotator.model_name
234     scores = []
235     patterns = {}
236     for ef in eval_dir.rglob("*.json"):
237         data = json.load(open(ef))
238         s = data.get("score")
239         p = data.get("pattern", "unknown")
240         if s is not None:
241             scores.append(s)
242             patterns[p] = patterns.get(p, 0) + 1
243
244     if scores:
245         print(f" Score: {sum(scores)/len(scores):.2f} ({len(scores)} pairs)")
246         print(f" Patterns: {patterns}")
247
248
249 if __name__ == "__main__":
250     p = argparse.ArgumentParser(description="Run prompt reverse with A/B forced format")
251     p.add_argument("model", nargs="?", default="gpt-5.2", help="Model name")
252     p.add_argument("--workers", type=int, default=4)
253     p.add_argument("--subfolder", default=None)
254     p.add_argument("--list-models", action="store_true")
255     a = p.parse_args()
256
257     if a.list_models:
258         models = sorted(_get_all_models().keys())
259         print(f"Available models ({len(models)}):")
260         for m in models:
261             print(f" {m}")
262         sys.exit(0)
263
264     run(a.model, a.workers, a.subfolder)

```

Listing 10: scripts/experiments/run\_prompt\_reverse.py — Prompt reversal experiment

```

1 """Generate figure descriptions under caption bias conditions.
2
3 Two conditions:
4 1. modified_caption: image + poisoned caption (tests resistance)
5 2. image_only: image alone, no caption (tests caption dependency)
6
7 Output: output/experiments/caption_bias/<condition>/<model>/<subfolder>/<fig_key>.json
8
9 Usage:

```

```

10 python3 scripts/experiments/run_caption_bias.py gpt-5.2 --condition modified_caption --workers 4
11 python3 scripts/experiments/run_caption_bias.py gpt-5.2 --condition image_only
12 python3 scripts/experiments/run_caption_bias.py --list-models
13 """
14
15 import argparse
16 import json
17 import logging
18 import sys
19 import threading
20 from concurrent.futures import ThreadPoolExecutor, as_completed
21 from pathlib import Path
22
23 from dotenv import load_dotenv
24
25 ROOT = Path(__file__).resolve().parent.parent.parent
26 load_dotenv(ROOT / ".env")
27 sys.path.insert(0, str(ROOT / "scripts"))
28
29 from llm_generation.models import get_annotator, _get_all_models
30 from llm_generation.prompts import load_prompts, get_prompt
31
32 ADVERSARIAL_DIR = ROOT / "adversarial_experiments"
33 SAMPLES_FILE = ADVERSARIAL_DIR / "samples.json"
34 FIGURES_DIR = ADVERSARIAL_DIR / "figures"
35 GROUNDTRUTH_DIR = ROOT / "Dataset" / "groundtruth"
36 CAPTION_BIAS_DIR = ADVERSARIAL_DIR / "benchmarks" / "adversarial" / "caption_bias"
37 OUTPUT_DIR = ROOT / "output" / "experiments" / "caption_bias"
38
39 CONDITIONS = ["modified_caption", "image_only"]
40
41 SUBFOLDER_TO_LANG_FILE = {
42     "bulgarian_only": "bulgarian",
43     "chinese_only": "chinese",
44     "english_only": "english",
45     "german_only": "german",
46     "multi_language": "multi_language",
47 }
48
49 logging.basicConfig(level=logging.INFO, format="%(asctime)s %(levelname)s %(message)s", datefmt="%H:%M:%S")
50 logger = logging.getLogger(__name__)
51
52
53 def _load_modified_captions():
54     """Load all modified captions across languages."""
55     captions = {}
56     for lang_file in CAPTION_BIAS_DIR.glob("*.json"):
57         with open(lang_file) as f:
58             data = json.load(f)
59             captions.update(data)
60     return captions
61
62
63 def _process_figure(annotator, prompts, condition, subfolder, fig_key, modified_captions):
64     """Process a single figure under a caption bias condition."""
65     fig_path = FIGURES_DIR / subfolder / fig_key / "original.png"
66     gt_path = GROUNDTRUTH_DIR / subfolder / f"{fig_key}.json"
67     out_path = OUTPUT_DIR / condition / annotator.model_name / subfolder / f"{fig_key}.json"
68
69     if out_path.exists():
70         return True, fig_key, "skip"
71
72     if not fig_path.exists():
73         return False, fig_key, f"no image: {fig_path}"
74     if not gt_path.exists():
75         return False, fig_key, f"no groundtruth: {gt_path}"
76
77     with open(gt_path) as f:
78         gt = json.load(f)
79
80     figure_type = gt["figure_type"]
81
82     if subfolder == "multi_language":
83         languages = sorted(set(a["annotation_language"] for a in gt["annotations"]))
84     else:
85         languages = [gt["paper_language"]]
86
87     # Determine caption and paper_title based on condition
88     if condition == "modified_caption":
89         caption_data = modified_captions.get(fig_key, {})
90         caption = caption_data.get("modified_caption", gt["caption"])
91         paper_title = gt.get("paper_title", "")
92     elif condition == "image_only":
93         caption = ""
94         paper_title = ""
95
96     try:
97         if len(languages) == 1:
98             prompt_text = get_prompt(prompts, languages[0], figure_type)
99             description = annotator.annotate_figure(
100                 prompt=prompt_text,
101                 image_path=fig_path,
102                 caption=caption,
103                 paper_title=paper_title,
104             )
105
106             result = {
107                 "figure_key": fig_key,
108                 "task_id": gt["task_id"],
109                 "paper_title": gt.get("paper_title", ""),
110                 "paper_language": gt["paper_language"],
111                 "figure_type": figure_type,
112                 "condition": condition,
113                 "caption_used": caption,
114                 "model_name": annotator.model_name,
115                 "model_annotation": description,
116             }
117         else:
118             annotations_by_lang = {}
119             for lang in languages:
120                 prompt_text = get_prompt(prompts, lang, figure_type)
121                 desc = annotator.annotate_figure(
122                     prompt=prompt_text,
123                     image_path=fig_path,
124                     caption=caption,
125                     paper_title=paper_title,
126                 )
127                 annotations_by_lang[lang] = desc
128
129             result = {
130                 "figure_key": fig_key,

```



```

131         "task_id": gt["task_id"],
132         "paper_title": gt.get("paper_title", ""),
133         "paper_language": gt["paper_language"],
134         "figure_type": figure_type,
135         "condition": condition,
136         "caption_used": caption,
137         "model_name": annotator.model_name,
138         "model_annotations": annotations_by_lang,
139     }
140
141     out_path.parent.mkdir(parents=True, exist_ok=True)
142     with open(out_path, "w") as f:
143         json.dump(result, f, indent=2, ensure_ascii=False)
144
145     logger.info(f" OK {condition}/{subfolder}/{fig_key}")
146     return True, fig_key, "done"
147
148 except Exception as e:
149     logger.error(f" FAIL {condition}/{subfolder}/{fig_key}: {e}")
150     return False, fig_key, str(e)
151
152
153 def run(model_name="gpt-5.2", condition="modified_caption", workers=4, subfolder_filter=None):
154     if condition not in CONDITIONS:
155         logger.error(f"Unknown condition: {condition}. Available: {CONDITIONS}")
156         return
157
158     with open(SAMPLES_FILE) as f:
159         samples = json.load(f)["samples"]
160
161     annotator = get_annotator(model_name)
162     prompts = load_prompts()
163     modified_captions = _load_modified_captions() if condition == "modified_caption" else {}
164
165     # Build work items
166     work_items = []
167     skipped = 0
168     for subfolder, fig_keys in samples.items():
169         if subfolder_filter and subfolder != subfolder_filter:
170             continue
171         for fig_key in fig_keys:
172             out_path = OUTPUT_DIR / condition / annotator.model_name / subfolder / f"{fig_key}.json"
173             if out_path.exists():
174                 skipped += 1
175             else:
176                 work_items.append((subfolder, fig_key))
177
178     total = len(work_items) + skipped
179     logger.info(f"Caption bias | model={annotator.model_name} | condition={condition}")
180     logger.info(f"Total: {total} ({skipped} done, {len(work_items)} to run)")
181
182     if not work_items:
183         logger.info("Nothing to do.")
184         return
185
186     success, errors = skipped, 0
187     lock = threading.Lock()
188
189     def _on_done(ok, k, s):
190         nonlocal success, errors
191         with lock:
192             if ok:
193                 success += 1
194             else:
195                 errors += 1
196             d = success + errors - skipped
197             if d % 10 == 0 or not ok:
198                 logger.info(f" Progress: {d}/{len(work_items)} (errors={errors})")
199
200     if workers == 1:
201         for sub, fk in work_items:
202             ok, k, s = _process_figure(annotator, prompts, condition, sub, fk, modified_captions)
203             _on_done(ok, k, s)
204     else:
205         with ThreadPoolExecutor(max_workers=workers) as ex:
206             futs = {
207                 ex.submit(_process_figure, annotator, prompts, condition, sub, fk, modified_captions): fk
208                 for sub, fk in work_items
209             }
210             for f in as_completed(futs):
211                 try:
212                     ok, k, s = f.result()
213                     _on_done(ok, k, s)
214                 except Exception as e:
215                     logger.error(f" UNEXPECTED {futs[f]}: {e}")
216                     with lock:
217                         errors += 1
218
219     logger.info(f"Done. {success}/{total} completed, {errors} errors.")
220
221
222 if __name__ == "__main__":
223     p = argparse.ArgumentParser(description="Generate descriptions under caption bias conditions")
224     p.add_argument("model", nargs="?", default="gpt-5.2", help="Model name")
225     p.add_argument("--condition", required=True, choices=CONDITIONS, help="Caption condition")
226     p.add_argument("--workers", type=int, default=4)
227     p.add_argument("--subfolder", default=None)
228     p.add_argument("--list-models", action="store_true")
229     a = p.parse_args()
230
231     if a.list_models:
232         models = sorted(_get_all_models().keys())
233         print(f"Available models ({len(models)}):")
234         for m in models:
235             print(f" {m}")
236     sys.exit(0)
237
238     run(a.model, a.condition, a.workers, a.subfolder)

```

Listing 11: scripts/experiments/run.caption\_bias.py — Caption bias experiment

```

1 """Run capability + hallucination + prompt reverse questions on adversarial figures.
2
3 Sends 5 questions per prompt, 2 prompts per figure:
4 Prompt 1: 3 capability + 1 inexistent + 1 prompt_reverse_true
5 Prompt 2: 2 capability + 1 contra + 1 unanswerable + 1 prompt_reverse_false
6
7 Usage:

```

```

8     python3 scripts/experiments/run_questions.py gpt-5.2 --prompt 1 --workers 4
9     python3 scripts/experiments/run_questions.py llama4-maverick --prompt 2
10    python3 scripts/experiments/run_questions.py --list-models
11    """
12
13    import argparse
14    import json
15    import logging
16    import sys
17    import threading
18    from concurrent.futures import ThreadPoolExecutor, as_completed
19    from datetime import datetime, timezone
20    from pathlib import Path
21
22    from dotenv import load_dotenv
23
24    ROOT = Path(__file__).resolve().parent.parent.parent
25    load_dotenv(ROOT / ".env")
26    sys.path.insert(0, str(ROOT / "scripts"))
27
28    from llm_generation.models import get_annotator, _get_all_models
29
30    BENCHMARKS_DIR = ROOT / "adversarial_experiments" / "benchmarks"
31    FIGURES_DIR = ROOT / "adversarial_experiments" / "figures"
32    SAMPLES_FILE = ROOT / "adversarial_experiments" / "samples.json"
33    OUTPUT_DIR = ROOT / "output" / "experiments" / "questions"
34
35    logging.basicConfig(level=logging.INFO, format="%(asctime)s %(levelname)s %(message)s", datefmt="%H:%M:%S")
36    logger = logging.getLogger(__name__)
37
38    SUBFOLDER_TO_LANG_KEY = {
39        "bulgarian_only": "bulgarian",
40        "chinese_only": "chinese",
41        "english_only": "english",
42        "german_only": "german",
43        "multi_language": "multi_language",
44    }
45
46
47    def _load_capability_questions(lang_key):
48        """Load capability questions for a language."""
49        path = BENCHMARKS_DIR / "capability" / "native" / f"{lang_key}.json"
50        if path.exists():
51            with open(path) as f:
52                return json.load(f)
53        return {}
54
55
56    def _load_hallucination_probes(lang_key):
57        """Load hallucination probes for a language."""
58        path = BENCHMARKS_DIR / "adversarial" / "hallucination" / "native" / f"{lang_key}.json"
59        if path.exists():
60            with open(path) as f:
61                return json.load(f)
62        return {}
63
64
65    def _load_prompt_reverse(lang_key):
66        """Load prompt reverse pairs for a language."""
67        path = BENCHMARKS_DIR / "adversarial" / "prompt_reverse" / "native" / f"{lang_key}.json"
68        if path.exists():
69            with open(path) as f:
70                return json.load(f)
71        return {}
72
73
74    def _build_prompt1_questions(fig_key, cap_data, hallu_data, pr_data):
75        """Build 5 questions for prompt 1: 3 capability + 1 inexist + 1 reverse_true."""
76        items = []
77
78        # 3 capability questions (first 3)
79        if fig_key in cap_data:
80            for q in cap_data[fig_key].get("questions", [])[3:]:
81                items.append({
82                    "id": q["id"],
83                    "type": "capability",
84                    "question": q["question"],
85                })
86
87        # 1 hallucination inexist
88        if fig_key in hallu_data:
89            for p in hallu_data[fig_key].get("probes", []):
90                if p["type"] == "inexist":
91                    items.append({
92                        "id": p["id"],
93                        "type": "hallucination_inexist",
94                        "question": p["question"],
95                    })
96            break
97
98        # 1 prompt reverse true
99        if fig_key in pr_data:
100            pair = pr_data[fig_key].get("pair", {})
101            items.append({
102                "id": pair.get("id", "") + "_true",
103                "type": "prompt_reverse_true",
104                "question": pair.get("true_statement", ""),
105            })
106
107        return items
108
109
110    def _build_prompt2_questions(fig_key, cap_data, hallu_data, pr_data):
111        """Build 5 questions for prompt 2: 2 capability + 1 contra + 1 unanswerable + 1 reverse_false."""
112        items = []
113
114        # 2 capability questions (last 2)
115        if fig_key in cap_data:
116            for q in cap_data[fig_key].get("questions", [])[3:5]:
117                items.append({
118                    "id": q["id"],
119                    "type": "capability",
120                    "question": q["question"],
121                })
122
123        # 1 hallucination contra
124        if fig_key in hallu_data:
125            for p in hallu_data[fig_key].get("probes", []):
126                if p["type"] == "contra":
127                    items.append({
128                        "id": p["id"],

```

```

129         "type": "hallucination_contra",
130         "question": p["question"],
131     })
132     break
133
134 # 1 hallucination unanswerable
135 if fig_key in hallu_data:
136     for p in hallu_data[fig_key].get("probes", []):
137         if p["type"] == "unanswerable":
138             items.append({
139                 "id": p["id"],
140                 "type": "hallucination_unanswerable",
141                 "question": p["question"],
142             })
143     break
144
145 # 1 prompt reverse false
146 if fig_key in pr_data:
147     pair = pr_data[fig_key].get("pair", {})
148     items.append({
149         "id": pair.get("id", "") + "_false",
150         "type": "prompt_reverse_false",
151         "question": pair.get("false_statement", ""),
152     })
153
154 return items
155
156
157 def _format_questions_prompt(questions):
158     """Format questions into a single prompt string."""
159     lines = ["Please answer each of the following questions about the figure. IMPORTANT: Number your answers exactly as 1. 2. 3. 4. 5. with each answer on a",
160             "new line. Keep each answer concise (1-3 sentences).\n"]
161     for i, q in enumerate(questions, 1):
162         lines.append(f"{i}. {q['question']}")
163     return "\n".join(lines)
164
165 def _parse_answers(raw_response, questions):
166     """Parse numbered answers from the response.
167
168     Handles various formats:
169     - "1. answer" / "2. answer" (standard)
170     - "**1. question text**" / "1. **question**" (Gemini bold)
171     - "## 1. question" / "## 1question" (LLaMA-Scout headers)
172     - "3.\tanswer" (Phi with tabs)
173     - "1) answer" / "1: answer"
174
175     """
176     import re
177
178     n = len(questions)
179
180     sections = {}
181     current_num = None
182     current_lines = []
183
184     for line in raw_response.strip().split("\n"):
185         stripped = line.strip()
186         # Strip leading markdown: ##, **, *, etc.
187         stripped_clean = re.sub(r'^(#{1,3})\s+', '', stripped)
188         stripped_clean = re.sub(r'^\s*(1,2)\s+', '', stripped_clean)
189
190         # Try standard pattern: "1. " or "1) " or "1: " or "1.\t"
191         m = re.match(r'^(\d+)\s*[:)]\s*\t?', stripped_clean)
192
193         # Also try localized patterns: " 1 " " 1:" "Frage 1:"
194         if not m:
195             m2 = re.match(r'^(?:\s+|Frage|Question)\s*(\d+)\s*[:)]\s*\t?', stripped_clean, re.IGNORECASE)
196             if m2:
197                 m = m2
198
199         if m:
200             num = int(m.group(1))
201             if 1 <= num <= n:
202                 if current_num is not None:
203                     sections[current_num] = "\n".join(current_lines).strip()
204                 current_num = num
205                 # Remove everything up to and including the number prefix
206                 rest = re.sub(r'^[#\s]*(?:\s+|Frage|Question)?\s*(\d+)\s*[:)]\s*\t?', '', stripped, flags=re.IGNORECASE).strip()
207                 current_lines = [rest] if rest else []
208                 continue
209
210         if current_num is not None:
211             current_lines.append(stripped)
212
213     # Save last section
214     if current_num is not None:
215         sections[current_num] = "\n".join(current_lines).strip()
216
217     # Map back to question items
218     result = []
219     for i, q in enumerate(questions):
220         answer = sections.get(i + 1, "")
221         # If answer starts with the question text (bold), strip it
222         # Gemini often repeats: "**question text here?**" actual answer
223         if answer.startswith("**"):
224             # Find end of bold question
225             bold_end = answer.find("**", 2)
226             if bold_end > 0:
227                 after_bold = answer[bold_end + 2:].strip()
228                 # Remove leading bullet/dash
229                 after_bold = re.sub(r'^[\s-]\s+', '', after_bold).strip()
230                 if after_bold:
231                     answer = after_bold
232
233         result.append({
234             "id": q["id"],
235             "type": "model_answer",
236             "question": q["question"],
237             "answer": answer,
238         })
239
240 return result
241
242
243 def _process_figure(annotator, prompt_id, questions, subfolder, fig_key):
244     """Process a single figure with the given questions."""
245     fig_path = FIGURES_DIR / subfolder / fig_key / "original.png"
246     out_path = OUTPUT_DIR / annotator.model_name / subfolder / f"{fig_key}.json"
247
248     if not fig_path.exists():
249         return False, fig_key, f"no image: {fig_path}"
250
251     # Check if this prompt already ran
252     existing = None

```

```

249     if out_path.exists():
250         with open(out_path) as f:
251             existing = json.load(f)
252             for resp in existing.get("responses", []):
253                 if resp.get("prompt_id") == prompt_id:
254                     return True, fig_key, "skip"
255
256     if not questions:
257         return True, fig_key, "skip_no_questions"
258
259     prompt_text = _format_questions_prompt(questions)
260
261     try:
262         raw_response = annotator.annotate_figure(
263             prompt=prompt_text,
264             image_path=fig_path,
265             caption="",
266             paper_title="",
267         )
268
269         parsed_items = _parse_answers(raw_response, questions)
270
271         response_entry = {
272             "prompt_id": prompt_id,
273             "timestamp": datetime.now(timezone.utc).isoformat(),
274             "raw_response": raw_response,
275             "items": parsed_items,
276         }
277
278         if existing:
279             existing["responses"].append(response_entry)
280             result = existing
281         else:
282             result = {
283                 "figure_key": fig_key,
284                 "subfolder": subfolder,
285                 "model_name": annotator.model_name,
286                 "responses": [response_entry],
287             }
288
289         out_path.parent.mkdir(parents=True, exist_ok=True)
290         with open(out_path, "w") as f:
291             json.dump(result, f, indent=2, ensure_ascii=False)
292
293         logger.info(f" OK   {subfolder}/{fig_key} [{prompt_id}] ({len(parsed_items)} answers)")
294         return True, fig_key, "done"
295
296     except Exception as e:
297         logger.error(f" FAIL {subfolder}/{fig_key} [{prompt_id}]: {e}")
298         return False, fig_key, str(e)
299
300
301 def run(model_name, prompt_num, workers=4, subfolder_filter=None):
302     with open(SAMPLES_FILE) as f:
303         samples = json.load(f)["samples"]
304
305     annotator = get_annotator(model_name)
306     prompt_id = f"prompt{prompt_num}"
307
308     # Load all question data
309     all_cap = {}
310     all_hallu = {}
311     all_pr = {}
312     for subfolder in samples:
313         lang_key = SUBFOLDER_TO_LANG_KEY[subfolder]
314         all_cap.update(_load_capability_questions(lang_key))
315         all_hallu.update(_load_hallucination_probes(lang_key))
316         all_pr.update(_load_prompt_reverse(lang_key))
317
318     # Build work items
319     work_items = []
320     skipped = 0
321     for subfolder, fig_keys in samples.items():
322         for fig_key in fig_keys:
323             # Check skip
324             out_path = OUTPUT_DIR / annotator.model_name / subfolder / f"{fig_key}.json"
325             if out_path.exists():
326                 with open(out_path) as f:
327                     existing = json.load(f)
328                     if any(r.get("prompt_id") == prompt_id for r in existing.get("responses", [])):
329                         skipped += 1
330                         continue
331
332             if subfolder_filter and subfolder != subfolder_filter:
333                 continue
334
335             if prompt_num == 1:
336                 questions = _build_prompt1_questions(fig_key, all_cap, all_hallu, all_pr)
337             else:
338                 questions = _build_prompt2_questions(fig_key, all_cap, all_hallu, all_pr)
339
340             if questions:
341                 work_items.append((subfolder, fig_key, questions))
342
343     logger.info(f"Questions runner | model={annotator.model_name} | {prompt_id}")
344     logger.info(f"Total: {len(work_items) + skipped} ({skipped} done, {len(work_items)} to run)")
345
346     if not work_items:
347         logger.info("Nothing to do.")
348         return
349
350     success, errors = skipped, 0
351     lock = threading.Lock()
352
353     def _on_done(ok, k, s):
354         nonlocal success, errors
355         with lock:
356             if ok:
357                 success += 1
358             else:
359                 errors += 1
360             d = success + errors - skipped
361             if d % 10 == 0 or not ok:
362                 logger.info(f" Progress: {d}/{len(work_items)} (errors={errors})")
363
364     if workers == 1:
365         for sub, fk, qs in work_items:
366             ok, k, s = _process_figure(annotator, prompt_id, qs, sub, fk)
367             _on_done(ok, k, s)
368     else:
369         with ThreadPoolExecutor(max_workers=workers) as ex:

```

```

370         futs = {
371             ex.submit(_process_figure, annotator, prompt_id, qs, sub, fk): fk
372             for sub, fk, qs in work_items
373         }
374         for f in as_completed(futs):
375             try:
376                 ok, k, s = f.result()
377                 _on_done(ok, k, s)
378             except Exception as e:
379                 logger.error(f" UNEXPECTED {futs[f]}: {e}")
380                 with lock:
381                     errors += 1
382
383         logger.info(f"Done. {success}/{len(work_items) + skipped} completed, {errors} errors.")
384
385
386 if __name__ == "__main__":
387     p = argparse.ArgumentParser(description="Run questions on adversarial figures")
388     p.add_argument("model", nargs="?", default="gpt-5.2", help="Model name")
389     p.add_argument("--prompt", type=int, required=True, choices=[1, 2], help="Prompt number (1 or 2)")
390     p.add_argument("--workers", type=int, default=4)
391     p.add_argument("--subfolder", default=None)
392     p.add_argument("--list-models", action="store_true")
393     a = p.parse_args()
394
395     if a.list_models:
396         models = sorted(_get_all_models().keys())
397         print(f"Available models ({len(models)}):")
398         for m in models:
399             print(f" {m}")
400         sys.exit(0)
401
402     run(a.model, a.prompt, a.workers, a.subfolder)

```

Listing 12: scripts/experiments/run\_questions.py — Capability questions runner

```

1  """Run active admittance probes on selective blur images.
2
3  Sends the selectively blurred image + a reasoning question to the model.
4  The question's answer resolves to the blurred element.
5
6  Output: output/experiments/active_admittance/{model}/{subfolder}/{fig_key}.json
7
8  Usage:
9      python3 scripts/experiments/run_active_admittance.py --model gemini-3.1-pro --workers 4
10     python3 scripts/experiments/run_active_admittance.py --model gpt-5.2 --subfolder english_only
11 """
12
13 import argparse
14 import json
15 import logging
16 import sys
17 import threading
18 from concurrent.futures import ThreadPoolExecutor, as_completed
19 from datetime import datetime, timezone
20 from pathlib import Path
21
22 from dotenv import load_dotenv
23
24 ROOT = Path(__file__).resolve().parent.parent.parent
25 load_dotenv(ROOT / ".env")
26 sys.path.insert(0, str(ROOT / "scripts"))
27
28 from llm_generation.models import get_annotator, _get_all_models
29
30 PROBES_FILE = ROOT / "adversarial_experiments" / "benchmarks" / "adversarial" / "admittance" / "active_selective_blur_probes.json"
31 FIGURES_DIR = ROOT / "adversarial_experiments" / "figures"
32 OUTPUT_DIR = ROOT / "output" / "experiments" / "active_admittance"
33
34 logging.basicConfig(level=logging.INFO, format="%(asctime)s %(levelname)s %(message)s", datefmt="%H:%M:%S")
35 logger = logging.getLogger(__name__)
36
37
38 def _process_figure(annotator, fig_key, probe):
39     """Send selective blur image + question to model."""
40     subfolder = probe["subfolder"]
41     question = probe["question"]
42
43     # Find selective blur image
44     fig_dir = FIGURES_DIR / subfolder / fig_key / "transforms"
45     fig_path = None
46     for ext in [".png", ".jpeg", ".jpg"]:
47         p = fig_dir / f"selective_blur{ext}"
48         if p.exists():
49             fig_path = p
50             break
51
52     if not fig_path:
53         logger.warning(f" SKIP {fig_key} no selective blur image")
54         return True, fig_key, "skip"
55
56     out_path = OUTPUT_DIR / annotator.model_name / subfolder / f"{fig_key}.json"
57     if out_path.exists():
58         return True, fig_key, "skip"
59
60     try:
61         prompt = "Please answer the following question about this figure. Keep your answer concise (1-3 sentences). "
62         raw_response = annotator.annotate_figure(
63             prompt=prompt,
64             image_path=fig_path,
65             caption=question,
66             paper_title="",
67         )
68
69         result = {
70             "figure_key": fig_key,
71             "subfolder": subfolder,
72             "model_name": annotator.model_name,
73             "question": question,
74             "question_english": probe.get("question_english", ""),
75             "expected_answer": probe.get("expected_answer", ""),
76             "blurred_element": probe.get("blurred_element", ""),
77             "model_answer": raw_response,
78             "timestamp": datetime.now(timezone.utc).isoformat(),
79         }
80
81         out_path.parent.mkdir(parents=True, exist_ok=True)
82         with open(out_path, "w") as f:

```

```

83         json.dump(result, f, indent=2, ensure_ascii=False)
84
85         logger.info(f" OK   {subfolder}/{fig_key}: {raw_response[:80]}...")
86         return True, fig_key, "done"
87
88     except Exception as e:
89         logger.error(f" FAIL {fig_key}: {e}")
90         return False, fig_key, str(e)
91
92
93 def run(model_name, workers=4, subfolder_filter=None):
94     with open(PROBES_FILE) as f:
95         probes = json.load(f)["figures"]
96
97     annotator = get_annotator(model_name)
98
99     work_items = []
100    skipped = 0
101    for fig_key, probe in sorted(probes.items()):
102        if subfolder_filter and probe["subfolder"] != subfolder_filter:
103            continue
104        out_path = OUTPUT_DIR / annotator.model_name / probe["subfolder"] / f"{fig_key}.json"
105        if out_path.exists():
106            skipped += 1
107        else:
108            work_items.append((fig_key, probe))
109
110    logger.info(f"Active admittance runner | model={annotator.model_name}")
111    logger.info(f"Total: {len(work_items) + skipped} ({skipped} done, {len(work_items)} to run)")
112
113    if not work_items:
114        logger.info("Nothing to do.")
115        return
116
117    success, errors = skipped, 0
118    lock = threading.Lock()
119
120    if workers == 1:
121        for fk, probe in work_items:
122            ok, k, s = _process_figure(annotator, fk, probe)
123            with lock:
124                if ok: success += 1
125                else: errors += 1
126    else:
127        with ThreadPoolExecutor(max_workers=workers) as ex:
128            futs = [ex.submit(_process_figure, annotator, fk, probe) for fk, probe in work_items]
129            for f in as_completed(futs):
130                try:
131                    ok, k, s = f.result()
132                    with lock:
133                        if ok: success += 1
134                        else: errors += 1
135                except Exception as e:
136                    logger.error(f" UNEXPECTED {futs[f]}: {e}")
137                    with lock: errors += 1
138
139    logger.info(f"Done. {success}/{len(work_items)+skipped} completed, {errors} errors.")
140
141
142 if __name__ == "__main__":
143     p = argparse.ArgumentParser(description="Run active admittance probes on selective blur images")
144     p.add_argument("--model", required=True)
145     p.add_argument("--workers", type=int, default=4)
146     p.add_argument("--subfolder", default=None)
147     p.add_argument("--list-models", action="store_true")
148     a = p.parse_args()
149
150     if a.list_models:
151         print(sorted(_get_all_models().keys()))
152         sys.exit(0)
153
154     run(a.model, a.workers, a.subfolder)

```

Listing 13: scripts/experiments/run\_active\_admittance.py — Active admittance probes

```

1  """Run inductance probes on original clean images.
2
3  Sends the original figure image + a deep reasoning question to the model.
4
5  Output: output/experiments/inductance/{model}/{fig_key}.json
6
7  Usage:
8  python3 scripts/experiments/run_inductance.py --model gemini-3.1-pro
9  """
10
11 import argparse
12 import json
13 import logging
14 import sys
15 from datetime import datetime, timezone
16 from pathlib import Path
17
18 from dotenv import load_dotenv
19
20 ROOT = Path(__file__).resolve().parent.parent.parent
21 load_dotenv(ROOT / ".env")
22 sys.path.insert(0, str(ROOT / "scripts"))
23
24 from llm_generation.models import get_annotator, _get_all_models
25
26 PROBES_FILE = ROOT / "adversarial_experiments" / "benchmarks" / "adversarial" / "inductance" / "active_inductance_probes.json"
27 FIGURES_DIR = ROOT / "Dataset" / "figures"
28 OUTPUT_DIR = ROOT / "output" / "experiments" / "inductance"
29
30 logging.basicConfig(level=logging.INFO, format="%(asctime)s %(levelname)s %(message)s", datefmt="%H:%M:%S")
31 logger = logging.getLogger(__name__)
32
33
34 def run(model_name):
35     with open(PROBES_FILE) as f:
36         probes = json.load(f)["probes"]
37
38     annotator = get_annotator(model_name)
39
40     for probe in probes:
41         fig_key = probe["figure_key"]
42         subfolder = probe["subfolder"]
43         question = probe["question"]

```

```

44
45     fig_path = FIGURES_DIR / subfolder / f"{fig_key}.png"
46     out_path = OUTPUT_DIR / annotator.model_name / f"{fig_key}.json"
47
48     if out_path.exists():
49         logger.info(f" SKIP {fig_key}")
50         continue
51
52     if not fig_path.exists():
53         logger.warning(f" SKIP {fig_key}    no image")
54         continue
55
56     try:
57         prompt = "Please answer the following question about this figure. Show your reasoning and working."
58         raw_response = annotator.annotate_figure(
59             prompt=prompt,
60             image_path=fig_path,
61             caption=question,
62             paper_title="",
63         )
64
65         result = {
66             "figure_key": fig_key,
67             "subfolder": subfolder,
68             "model_name": annotator.model_name,
69             "question": question,
70             "question_english": probe.get("question_english", ""),
71             "expected_answer": probe.get("expected_answer", ""),
72             "trap": probe.get("trap", ""),
73             "naive_answer": probe.get("naive_answer", ""),
74             "model_answer": raw_response,
75             "timestamp": datetime.now(timezone.utc).isoformat(),
76         }
77
78         out_path.parent.mkdir(parents=True, exist_ok=True)
79         with open(out_path, "w") as f:
80             json.dump(result, f, indent=2, ensure_ascii=False)
81
82         logger.info(f" OK    {fig_key}: {raw_response[:100]}...")
83
84     except Exception as e:
85         logger.error(f" FAIL {fig_key}: {e}")
86
87     logger.info("Done.")
88
89
90 if __name__ == "__main__":
91     p = argparse.ArgumentParser(description="Run inductance probes")
92     p.add_argument("--model", required=True)
93     a = p.parse_args()
94     run(a.model)

```

Listing 14: scripts/experiments/run\_inductance.py — Inductance reasoning probes

## Results Computation

```

1  """RQ1: Compute main leaderboard with bootstrap 95% CIs.
2
3  Produces the hero table: 13 models x (Overall, EN, BG, CN, DE) with CIs.
4  Also computes pairwise significance tests and Cliff's delta.
5
6  Output: output/evaluation-results/rq1/leaderboard.json
7         output/evaluation-results/rq1/pairwise_significance.json
8  """
9
10 import json
11 import sys
12 from pathlib import Path
13 from collections import defaultdict
14
15 import numpy as np
16
17 ROOT = Path(__file__).resolve().parent.parent.parent.parent
18 EVAL_DIR = ROOT / "output" / "evaluation" / "atomic_mqm_v2"
19 ATOMS_DIR = ROOT / "atomic_mqm" / "atoms"
20 OUTPUT_DIR = ROOT / "output" / "evaluation-results" / "rq1"
21
22
23 LANG_MAP = {
24     "english_only": "EN",
25     "bulgarian_only": "BG",
26     "chinese_only": "CN",
27     "german_only": "DE",
28     "multi_language": "Multi",
29 }
30
31
32 def bootstrap_ci(scores, n_resamples=10000, ci=0.95):
33     """Compute BCa-style bootstrap confidence interval."""
34     scores = np.array(scores)
35     n = len(scores)
36     if n < 2:
37         m = float(np.mean(scores))
38         return m, m, m
39
40     rng = np.random.default_rng(42)
41     boot_means = np.array([
42         np.mean(rng.choice(scores, size=n, replace=True))
43         for _ in range(n_resamples)
44     ])
45
46     alpha = (1 - ci) / 2
47     lo = float(np.percentile(boot_means, alpha * 100))
48     hi = float(np.percentile(boot_means, (1 - alpha) * 100))
49     return float(np.mean(scores)), lo, hi
50
51
52 def paired_bootstrap_test(scores_a, scores_b, n_resamples=10000):
53     """Test if model A is significantly better than model B."""
54     diffs = np.array(scores_a) - np.array(scores_b)
55     n = len(diffs)
56     observed_diff = np.mean(diffs)
57

```

```

58 rng = np.random.default_rng(42)
59 count = 0
60 for _ in range(n_resamples):
61     sample = rng.choice(diffs, size=n, replace=True)
62     if sample.mean() <= 0:
63         count += 1
64
65 p_value = count / n_resamples
66 return float(observed_diff), p_value
67
68
69 def cliffs_delta(x, y):
70     """Non-parametric effect size."""
71     x, y = np.array(x), np.array(y)
72     n_x, n_y = len(x), len(y)
73     more = sum(1 for xi in x for yi in y if xi > yi)
74     less = sum(1 for xi in x for yi in y if xi < yi)
75     return float((more - less) / (n_x * n_y))
76
77
78 def load_all_results():
79     """Load all v2 evaluation results."""
80     results = defaultdict(lambda: defaultdict(lambda: defaultdict(list)))
81     # results[judge][model][lang] = [scores]
82
83     for judge_dir in sorted(EVAL_DIR.glob("azure/*")):
84         judge = judge_dir.name
85         for model_dir in judge_dir.iterdir():
86             if not model_dir.is_dir():
87                 continue
88             model = model_dir.name
89             for f in model_dir.rglob("*.json"):
90                 try:
91                     d = json.load(open(f))
92                     sub = d.get("subfolder", "")
93                     lang = LANG_MAP.get(sub, "Other")
94                     score = d["mqm_score"]
95                     results[judge][model][lang].append(score)
96                     results[judge][model]["Overall"].append(score)
97                 except Exception:
98                     pass
99     return results
100
101
102 def main():
103     OUTPUT_DIR.mkdir(parents=True, exist_ok=True)
104     results = load_all_results()
105     judges = sorted(results.keys())
106     languages = ["Overall", "EN", "BG", "CN", "DE", "Multi"]
107
108     # Build leaderboard
109     leaderboard = []
110     for model in sorted(set(m for j in judges for m in results[j])):
111         row = {"model": model}
112         for judge in judges:
113             for lang in languages:
114                 scores = results[judge][model].get(lang, [])
115                 if scores:
116                     mean, ci_lo, ci_hi = bootstrap_ci(scores)
117                     row[f"{judge}_{lang}"] = {
118                         "mean": round(mean, 2),
119                         "ci_lo": round(ci_lo, 2),
120                         "ci_hi": round(ci_hi, 2),
121                         "n": len(scores),
122                         "std": round(float(np.std(scores)), 2),
123                     }
124
125     # Compute average across judges
126     for lang in languages:
127         all_judge_scores = []
128         for judge in judges:
129             all_judge_scores.extend(results[judge][model].get(lang, []))
130         if all_judge_scores:
131             mean, ci_lo, ci_hi = bootstrap_ci(all_judge_scores)
132             row[f"avg_{lang}"] = {
133                 "mean": round(mean, 2),
134                 "ci_lo": round(ci_lo, 2),
135                 "ci_hi": round(ci_hi, 2),
136                 "n": len(all_judge_scores),
137             }
138         leaderboard.append(row)
139
140     # Sort by average overall
141     leaderboard.sort(key=lambda r: -r.get("avg_Overall", {}).get("mean", 0))
142
143     # Add rank
144     for i, row in enumerate(leaderboard, 1):
145         row["rank"] = i
146
147     # Save leaderboard
148     with open(OUTPUT_DIR / "leaderboard.json", "w") as f:
149         json.dump(leaderboard, f, indent=2)
150
151     # Print per-judge leaderboards
152     for judge in judges:
153         print(f"\n{'='*80}")
154         print(f"Judge: {judge}")
155         print(f"{'='*80}")
156         print(f"{'Rank':>4s} {'Model':>25s}", end="")
157         for lang in languages:
158             print(f" {lang:>12s}", end="")
159         print()
160         print(f"{'-' * (30 + 13 * len(languages))}")
161
162     judge_rows = sorted(leaderboard, key=lambda r: -r.get(f"{judge}_Overall", {}).get("mean", 0))
163     for i, row in enumerate(judge_rows, 1):
164         print(f"{'i':>4d} {row['model']:>25s}", end="")
165         for lang in languages:
166             v = row.get(f"{judge}_{lang}", {})
167             if v:
168                 print(f" {v['mean']:>5.1f} ({v['ci_lo']:>5.0f}-{v['ci_hi']:>5.0f})", end="")
169             else:
170                 print(f" {'':>12s}", end="")
171         print()
172
173     # Print average summary
174     print(f"\n{'='*80}")
175     print(f"Average across judges")
176     print(f"{'='*80}")
177     print(f"{'Rank':>4s} {'Model':>25s}", end="")
178     for lang in languages:
179         print(f" {lang:>12s}", end="")

```



```

179 print()
180 print("-" * (30 + 13 * len(languages)))
181 for row in leaderboard:
182     print(f"{row['rank']:4d} {row['model']:25s}", end="")
183     for lang in languages:
184         v = row.get(f"avg_{lang}", {})
185         if v:
186             print(f" {v['mean']:5.1f} ({v['ci_lo']:0f}-{v['ci_hi']:0f})", end="")
187         else:
188             print(f" {' ':>12s}", end="")
189     print()
190
191 # Pairwise significance tests (adjacent ranks)
192 pairwise = []
193 for i in range(len(leaderboard) - 1):
194     a = leaderboard[i]
195     b = leaderboard[i + 1]
196     model_a, model_b = a["model"], b["model"]
197
198     # Pool scores across judges for overall
199     scores_a, scores_b = [], []
200     for judge in judges:
201         scores_a.extend(results[judge][model_a].get("Overall", []))
202         scores_b.extend(results[judge][model_b].get("Overall", []))
203
204     # Align to same figures (use min length)
205     n = min(len(scores_a), len(scores_b))
206     if n == 0:
207         continue
208     scores_a, scores_b = scores_a[:n], scores_b[:n]
209
210     diff, p_val = paired_bootstrap_test(scores_a, scores_b)
211     cd = cliffs_delta(scores_a, scores_b)
212
213     sig = "***" if p_val < 0.001 else "*" if p_val < 0.01 else "" if p_val < 0.05 else "ns"
214
215     pairwise.append({
216         "rank_a": a["rank"],
217         "model_a": model_a,
218         "rank_b": b["rank"],
219         "model_b": model_b,
220         "mean_diff": round(diff, 2),
221         "p_value": round(p_val, 4),
222         "cliffs_delta": round(cd, 3),
223         "significance": sig,
224     })
225
226 with open(OUTPUT_DIR / "pairwise_significance.json", "w") as f:
227     json.dump(pairwise, f, indent=2)
228
229 print(f"\n{'Model A':25s} vs {'Model B':25s} {'Diff':>6s} {'p':>7s} {'Cliff':>7s} {'Sig':>4s}")
230 print("-" * 80)
231 for p in pairwise:
232     print(f"{p['model_a']:25s} vs {p['model_b']:25s} {p['mean_diff']:6.1f} {p['p_value']:7.4f} {p['cliffs_delta']:7.3f} {p['significance']:>4s}")
233
234 print(f"\nSaved to {OUTPUT_DIR}")
235
236
237 if __name__ == "__main__":
238     main()

```

Listing 15: scripts/evaluation-results/rq1/compute\_leaderboard.py — RQ1 leaderboard with bootstrap CIs

```

1 """RQ2: Compute transform degradation results.
2
3 Extracts per-model per-transform MQM scores, deltas from original,
4 bootstrap CIs on deltas, and robustness ranking.
5
6 Output: output/evaluation-results/rq2/degradation.json
7 """
8
9 import json
10 from pathlib import Path
11 from collections import defaultdict
12
13 import numpy as np
14
15 ROOT = Path(__file__).resolve().parent.parent.parent.parent
16 ADV_RESULTS = ROOT / "dashboard" / "public" / "data" / "adversarial_results.json"
17 EVAL_DIR = ROOT / "output" / "evaluation" / "transforms"
18 OUTPUT_DIR = ROOT / "output" / "evaluation-results" / "rq2"
19
20 TRANSFORMS = [
21     "original", "original_in_paper",
22     "jpeg_compression", "noise", "aspect_ratio",
23     "low_contrast", "rotation", "blurred_in_paper",
24 ]
25
26 TRANSFORM_LABELS = {
27     "original": "Original",
28     "original_in_paper": "In-Paper",
29     "jpeg_compression": "JPEG",
30     "noise": "Noise",
31     "aspect_ratio": "Aspect",
32     "low_contrast": "Low Contr.",
33     "rotation": "Rotation",
34     "blurred_in_paper": "Blur-in-Paper",
35 }
36
37 MODEL_SHORT = {
38     "gpt-5.2": "GPT-5.2",
39     "gemini-3.1-pro": "Gemini 3.1P",
40     "claude-opus-4.6": "Claude 4.6",
41     "gwen3-vl-235b-a22b": "Qwen 235B",
42     "gwen3-vl-32b": "Qwen 32B",
43     "gwen3-vl-30b-a3b": "Qwen 30B",
44     "gwen3-vl-8b": "Qwen 8B",
45     "llama4-maverick": "LLaMA Mav.",
46     "llama4-scout": "LLaMA Scout",
47     "gemma3-27b-it": "Gemma 27B",
48     "phi-4-multimodal": "Phi-4",
49     "gemma3-12b-it": "Gemma 12B",
50     "gemma3-4b-it": "Gemma 4B",
51 }
52
53
54 def bootstrap_ci_delta(orig_scores, trans_scores, n_resamples=10000):
55     """Bootstrap CI on the mean delta (transform - original)."""

```

```

56     diffs = np.array(trans_scores) - np.array(orig_scores)
57     n = len(diffs)
58     if n < 2:
59         return float(np.mean(diffs)), float(np.mean(diffs)), float(np.mean(diffs))
60     rng = np.random.default_rng(42)
61     boot = [np.mean(rng.choice(diffs, size=n, replace=True)) for _ in range(n_resamples)]
62     return float(np.mean(diffs)), float(np.percentile(boot, 2.5)), float(np.percentile(boot, 97.5))
63
64
65 def load_per_figure_scores():
66     """Load per-figure scores from evaluation/transforms for bootstrap."""
67     scores = defaultdict(lambda: defaultdict(dict))
68     # scores[model][transform][fig_key] = score (judge-averaged)
69
70     for judge_dir in EVAL_DIR.glob("azure/*"):
71         judge = judge_dir.name
72         for model_dir in judge_dir.iterdir():
73             if not model_dir.is_dir():
74                 continue
75             model = model_dir.name
76             for transform_dir in model_dir.iterdir():
77                 if not transform_dir.is_dir():
78                     continue
79                 transform = transform_dir.name
80                 for f in transform_dir.rglob("*.json"):
81                     try:
82                         d = json.load(open(f))
83                         fig = d["figure_key"]
84                         if fig not in scores[model][transform]:
85                             scores[model][transform][fig] = []
86                         scores[model][transform][fig].append(d["mqm_score"])
87                     except Exception:
88                         pass
89
90     # Average across judges per figure
91     averaged = defaultdict(lambda: defaultdict(dict))
92     for model in scores:
93         for transform in scores[model]:
94             for fig, judge_scores in scores[model][transform].items():
95                 averaged[model][transform][fig] = np.mean(judge_scores)
96
97     return averaged
98
99
100 def main():
101     OUTPUT_DIR.mkdir(parents=True, exist_ok=True)
102
103     # Load aggregated data
104     adv = json.load(open(ADV_RESULTS))
105     tm = adv["transform_mqm"]
106
107     # Load per-figure for bootstrap
108     per_fig = load_per_figure_scores()
109
110     results = []
111     for row in tm:
112         model = row["model"]
113         orig = row.get("original", 0)
114         entry = {
115             "model": model,
116             "model_short": MODEL_SHORT.get(model, model),
117         }
118
119         for t in TRANSFORMS:
120             score = row.get(t, None)
121             if score is not None:
122                 delta = score - orig if t != "original" else 0
123                 entry[t] = {
124                     "score": round(score, 1),
125                     "delta": round(delta, 1),
126                 }
127
128             # Bootstrap CI on delta if we have per-figure data
129             if t != "original" and model in per_fig:
130                 orig_figs = per_fig[model].get("original", {})
131                 trans_figs = per_fig[model].get(t, {})
132                 common = sorted(set(orig_figs.keys()) & set(trans_figs.keys()))
133                 if len(common) >= 5:
134                     o_scores = [orig_figs[f] for f in common]
135                     t_scores = [trans_figs[f] for f in common]
136                     mean_d, ci_lo, ci_hi = bootstrap_ci_delta(o_scores, t_scores)
137                     entry[t]["delta_ci_lo"] = round(ci_lo, 1)
138                     entry[t]["delta_ci_hi"] = round(ci_hi, 1)
139
140             # Robustness = mean absolute delta across degradation transforms
141             deltas = [abs(entry[t]["delta"]) for t in TRANSFORMS]
142             if t not in ("original", "original_in_paper") and t in entry:
143                 entry["mean_abs_delta"] = round(np.mean(deltas), 1) if deltas else 0
144             entry["max_drop"] = round(min(entry[t]["delta"] for t in TRANSFORMS
145                                         if t not in ("original",) and t in entry), 1)
146
147             results.append(entry)
148
149     # Sort by robustness (smallest mean_abs_delta = most robust)
150     results.sort(key=lambda r: r["mean_abs_delta"])
151
152     # Per-transform average degradation
153     transform_summary = {}
154     for t in TRANSFORMS:
155         if t == "original":
156             continue
157         deltas = [r[t]["delta"] for r in results if t in r]
158         transform_summary[t] = {
159             "mean_delta": round(np.mean(deltas), 1),
160             "median_delta": round(float(np.median(deltas)), 1),
161             "worst_model": min(results, key=lambda r: r.get(t, {})).get("delta", 0) if t in results,
162             "best_model": max(results, key=lambda r: r.get(t, {})).get("delta", float("-inf")) if t in results,
163         }
164
165     output = {
166         "description": "RQ2: Transform degradation analysis. 45 adversarial figures, 2 judges averaged.",
167         "models": results,
168         "transform_summary": transform_summary,
169     }
170
171     with open(OUTPUT_DIR / "degradation.json", "w") as f:
172         json.dump(output, f, indent=2)
173
174     # Print
175     print(f"{'Model':>22s} {'Orig':>6s} {'InPap':>6s} {'JPEg':>6s} {'Noise':>6s} {'Aspct':>6s} {'LoCon':>6s} {'Rotat':>6s} {'BlurP':>6s} {'|':>5s}")
176     print("-" * 95)

```

```

177 for r in sorted(results, key=lambda x: -x.get("original", {}).get("score", 0)):
178     m = r["model_short"]
179     def fmt(t):
180         if t not in r: return " --"
181         if t == "original": return f"{r[t]['score']:6.1f}"
182         d = r[t]["delta"]
183         return f"{d:+5.1f}"
184     orig = r.get("original", {}).get("score", 0)
185     print(f"{m:22s} {orig:6.1f} {fmt('original_in_paper')} {fmt('jpeg_compression')} {fmt('noise')} {fmt('aspect_ratio')} {fmt('low_contrast')} {fmt('rotation')} {fmt('blurred_in_paper')} {r['mean_abs_delta']:5.1f}")
186
187 print(f"\nTransform severity (mean delta across models):")
188 for t in sorted(transform_summary, key=lambda x: transform_summary[x]["mean_delta"]):
189     ts = transform_summary[t]
190     print(f" {TRANSFORM_LABELS.get(t,t):15s}: {ts['mean_delta']:+5.1f}")
191
192 print(f"\nSaved to {OUTPUT_DIR / 'degradation.json'}")
193
194
195 if __name__ == "__main__":
196     main()

```

Listing 16: scripts/evaluation-results/rq2/compute\_degradation.py — RQ2 transform degradation

```

1 """RQ3: Compute capability question results per model, per category, per judge.
2
3 Also includes caption bias and prompt reversal scores.
4
5 Output: output/evaluation-results/rq3/capability.json
6 """
7
8 import json
9 from pathlib import Path
10 from collections import defaultdict
11
12 import numpy as np
13
14 ROOT = Path(__file__).resolve().parent.parent.parent
15 EVAL_DIR = ROOT / "output" / "experiments" / "evaluation" / "capability"
16 QUESTIONS_DIR = ROOT / "adversarial_experiments" / "benchmarks" / "capability"
17 ADV_RESULTS = ROOT / "dashboard" / "public" / "data" / "adversarial_results.json"
18 OUTPUT_DIR = ROOT / "output" / "evaluation-results" / "rq3"
19
20 MODEL_SHORT = {
21     "gpt-5.2": "GPT-5.2", "gemini-3.1-pro": "Gemini 3.1P", "Qwen 235B",
22     "claude-opus-4.6": "Claude 4.6", "qwen3-vl-235b-a22b": "Qwen 235B",
23     "qwen3-vl-32b": "Qwen 32B", "qwen3-vl-30b-a3b": "Qwen 30B",
24     "qwen3-vl-8b": "Qwen 8B", "llama4-maverick": "LLaMA Mav.",
25     "llama4-scout": "LLaMA Scout", "gemma3-27b-it": "Gemma 27B",
26     "phi-4-multimodal": "Phi-4", "gemma3-12b-it": "Gemma 12B",
27     "gemma3-4b-it": "Gemma 4B",
28 }
29
30 CATEGORIES = ["computation", "value_reading", "comparison", "trend_analysis", "counting"]
31 CAT_SHORT = {
32     "computation": "Comp.", "value_reading": "Value",
33     "comparison": "Compar.", "trend_analysis": "Trend", "counting": "Count",
34 }
35
36
37 def load_question_categories():
38     """Map question_id -> category from benchmark files."""
39     q_cat = {}
40     for f in QUESTIONS_DIR.rglob("*.json"):
41         if f.name in ("answer_corrections_log.json", "excluded_questions.json", "post_experiment_changes.json"):
42             continue
43         try:
44             d = json.load(open(f))
45             for fig_key, fig_data in d.items():
46                 if not isinstance(fig_data, dict):
47                     continue
48                 for q in fig_data.get("questions", []):
49                     q_cat[q["id"]] = q.get("category", "unknown")
50         except Exception:
51             pass
52     return q_cat
53
54
55 def main():
56     OUTPUT_DIR.mkdir(parents=True, exist_ok=True)
57     q_cat = load_question_categories()
58
59     # Per-model per-judge per-category accuracy
60     # results[model][judge][category] = [0/1 list]
61     results = defaultdict(lambda: defaultdict(lambda: defaultdict(list)))
62
63     for judge_dir in sorted(EVAL_DIR.iterdir()):
64         if not judge_dir.is_dir():
65             continue
66         judge = judge_dir.name
67         for model_dir in sorted(judge_dir.iterdir()):
68             if not model_dir.is_dir():
69                 continue
70             model = model_dir.name
71             for f in model_dir.rglob("*.json"):
72                 try:
73                     d = json.load(open(f))
74                     for ev in d.get("evaluations", []):
75                         qid = ev.get("question_id", "")
76                         if ev.get("excluded", False):
77                             continue
78                         score = ev.get("score", 0)
79                         correct = 1 if score == 1 else 0
80                         cat = q_cat.get(qid, "unknown")
81                         results[model][judge][cat].append(correct)
82                         results[model][judge]["overall"].append(correct)
83                 except Exception:
84                     pass
85
86     # Load caption bias and prompt reversal from adversarial_results
87     adv = json.load(open(ADV_RESULTS))
88     cb_data = {r["model"]: r for r in adv["caption_bias"]}
89     pr_data = {r["model"]: r for r in adv["prompt_reverse"]}
90
91     # Build output
92     model_results = []
93     for model in sorted(results.keys()):
94         entry = {"model": model, "model_short": MODEL_SHORT.get(model, model)}

```

```

95
96     for judge in ["gpt-4o", "mistral-large-3"]:
97         for cat in CATEGORIES + ["overall"]:
98             scores = results[model][judge].get(cat, [])
99             if scores:
100                 entry[f"{judge}_{cat}"] = round(np.mean(scores), 3)
101                 entry[f"{judge}_{cat}_n"] = len(scores)
102
103     # Judge-averaged per category
104     for cat in CATEGORIES + ["overall"]:
105         vals = []
106         for judge in ["gpt-4o", "mistral-large-3"]:
107             v = entry.get(f"{judge}_{cat}")
108             if v is not None:
109                 vals.append(v)
110         if vals:
111             entry[f"avg_{cat}"] = round(np.mean(vals), 3)
112
113     # Caption bias
114     cb = cb_data.get(model, {})
115     entry["caption_bias"] = cb.get("avg")
116
117     # Prompt reversal
118     pr = pr_data.get(model, {})
119     entry["prompt_reversal"] = pr.get("score")
120
121     model_results.append(entry)
122
123 model_results.sort(key=lambda r: -r.get("avg_overall", 0))
124
125 # Category difficulty
126 cat_summary = {}
127 for cat in CATEGORIES:
128     vals = [r.get(f"avg_{cat}", 0) for r in model_results if r.get(f"avg_{cat}") is not None]
129     cat_summary[cat] = {
130         "mean": round(np.mean(vals), 3),
131         "best_model": max(model_results, key=lambda r: r.get(f"avg_{cat}", 0))["model"],
132     }
133
134 output = {
135     "description": "RQ3: Capability question accuracy per model, category, judge.",
136     "categories": CATEGORIES,
137     "models": model_results,
138     "category_summary": cat_summary,
139 }
140
141 with open(OUTPUT_DIR / "capability.json", "w") as f:
142     json.dump(output, f, indent=2)
143
144 # Print
145 print(f"{'Model':22s} {'Overall':>7s} {'Comp.':>6s} {'Value':>6s} {'Comp.':>6s} {'Trend':>6s} {'Count':>6s} {'CapBi':>6s} {'PrRev':>6s}")
146 print("-" * 75)
147 for r in model_results:
148     def fmt(k):
149         v = r.get(k)
150         return f"{v:.2f}" if v is not None else " --"
151     print(f"{'r['model_short']':22s} {fmt('avg_overall'):>7s} {fmt('avg_computation'):>6s} {fmt('avg_value_reading'):>6s} {fmt('avg_comparison'):>6s} {fmt('avg_trend_analysis'):>6s} {fmt('avg_counting'):>6s} {fmt('caption_bias'):>6s} {fmt('prompt_reversal'):>6s}")
152
153 print(f"\nCategory difficulty (mean across models):")
154 for cat in sorted(cat_summary, key=lambda c: cat_summary[c]["mean"]):
155     cs = cat_summary[cat]
156     print(f"  {CAT_SHORT[cat]:10s}: {cs['mean']:.3f} (best: {MODEL_SHORT.get(cs['best_model'], cs['best_model'])})")
157
158 print(f"\nSaved to {OUTPUT_DIR / 'capability.json'}")
159
160
161 if __name__ == "__main__":
162     main()

```

Listing 17: scripts/evaluation-results/rq3/compute\_capability.py — RQ3 capability accuracy

```

1  """RQ4: Compute A-R-I behavioural dynamics scores.
2
3  Extracts Admittance, Resistance, and Inductance per model.
4
5  Output: output/evaluation-results/rq4/ari.json
6  """
7
8  import json
9  from pathlib import Path
10 import numpy as np
11
12 ROOT = Path(__file__).resolve().parent.parent.parent.parent
13 ADV_RESULTS = ROOT / "dashboard" / "public" / "data" / "adversarial_results.json"
14 OUTPUT_DIR = ROOT / "output" / "evaluation-results" / "rq4"
15
16 MODEL_SHORT = {
17     "gpt-5.2": "GPT-5.2", "gemini-3.1-pro": "Gemini 3.1P",
18     "claude-opus-4.6": "Claude 4.6", "qwen3-vl-235b-a22b": "Qwen 235B",
19     "qwen3-vl-32b": "Qwen 32B", "qwen3-vl-30b-a3b": "Qwen 30B",
20     "qwen3-vl-8b": "Qwen 8B", "llama4-maverick": "LLaMA Mav.",
21     "llama4-scout": "LLaMA Scout", "gemma3-27b-it": "Gemma 27B",
22     "phi-4-multimodal": "Phi-4", "gemma3-12b-it": "Gemma 12B",
23     "gemma3-4b-it": "Gemma 4B",
24 }
25
26
27 def main():
28     OUTPUT_DIR.mkdir(parents=True, exist_ok=True)
29     adv = json.load(open(ADV_RESULTS))
30
31     pa = {r["model"]: r for r in adv["passive_admittance"]}
32     aa = {r["model"]: r for r in adv["active_admittance"]}
33     hal = {r["model"]: r for r in adv["hallucination"]}
34     ind = {r["model"]: r for r in adv["inductance"]}
35
36     results = []
37     for model in sorted(pa.keys()):
38         p = pa[model]
39         a = aa[model]
40         h = hal[model]
41         i = ind[model]
42
43         # Admittance: average passive (axis + selective) and active
44         passive_axis = np.mean([p.get("gpt-4o_axis_blurred", 0), p.get("mistral-large-3_axis_blurred", 0)])
45         passive_sel = np.mean([p.get("gpt-4o_selective_blur", 0), p.get("mistral-large-3_selective_blur", 0)])
46         active = a.get("avg", np.mean([a.get("gpt-4o_admittance", 0), a.get("mistral-large-3_admittance", 0)]))

```

```

47     admittance = np.mean([passive_axis, passive_sel, active])
48
49     # Fabrication counts (judge-averaged)
50     fab_axis = np.mean([p.get("gpt-4o_axis_blurred_fabricates", 0), p.get("mistral-large-3_axis_blurred_fabricates", 0)])
51     fab_sel = np.mean([p.get("gpt-4o_selective_blur_fabricates", 0), p.get("mistral-large-3_selective_blur_fabricates", 0)])
52
53     # Resistance: hallucination probe resistance (judge-averaged)
54     resistance = np.mean([h.get("gpt-4o_overall", 0), h.get("mistral-large-3_overall", 0)])
55     contra = np.mean([h.get("gpt-4o_contra", 0), h.get("mistral-large-3_contra", 0)])
56     inexist = np.mean([h.get("gpt-4o_inexist", 0), h.get("mistral-large-3_inexist", 0)])
57     unans = np.mean([h.get("gpt-4o_unanswerable", 0), h.get("mistral-large-3_unanswerable", 0)])
58
59     # Inductance
60     inductance = i.get("avg", 0)
61
62     results.append({
63         "model": model,
64         "model_short": MODEL_SHORT.get(model, model),
65         "admittance": round(admittance, 3),
66         "passive_axis": round(passive_axis, 3),
67         "passive_selective": round(passive_sel, 3),
68         "active": round(active, 3),
69         "fab_axis": round(fab_axis, 1),
70         "fab_selective": round(fab_sel, 1),
71         "resistance": round(resistance, 3),
72         "resist_contra": round(contra, 3),
73         "resist_inexist": round(inexist, 3),
74         "resist_unanswerable": round(unans, 3),
75         "inductance": round(inductance, 3),
76     })
77
78     results.sort(key=lambda r: -(r["admittance"] + r["resistance"] + r["inductance"]))
79
80     output = {"models": results}
81     with open(OUTPUT_DIR / "ari.json", "w") as f:
82         json.dump(output, f, indent=2)
83
84     # Print
85     print(f"{'Model':>22s} {'A':>6s} {'Pa':>6s} {'Ps':>6s} {'Ac':>6s} {'R':>6s} {'Ct':>6s} {'In':>6s} {'Un':>6s} {'I':>6s}")
86     print("-" * 80)
87     for r in results:
88         print(f"{'r['model_short']':>22s} {'r['admittance']':>6.2f} {'r['passive_axis']':>6.2f} {'r['passive_selective']':>6.2f} {'r['active']':>6.2f} {'r['resistance']':>6.2f} {'r['resist_contra']':>6.2f} {'r['resist_inexist']':>6.2f} {'r['resist_unanswerable']':>6.2f} {'r['inductance']':>6.2f}")
89
90     print(f"\nSaved to {OUTPUT_DIR / 'ari.json'}")
91
92
93 if __name__ == "__main__":
94     main()

```

Listing 18: scripts/evaluation-results/rq4/compute\_ari.py — RQ4 A-R-I behavioural scores

## Evaluation Experiment Scorers

```

1 """Evaluate passive admittance on axis_blurred and selective_blur descriptions.
2
3 For each blurred element, checks if the model:
4 1. Did not mention it at all (silent omission)      admittance = 0
5 2. Mentioned it and admitted it's unreadable      admittance = 1
6 3. Mentioned it and fabricated specific values     admittance = 0
7
8 Also flags whether fabricated values are correct (for inductance/hallucination analysis).
9
10 Usage:
11 python3 scripts/experiments/evaluate_admittance.py --model gemini-3.1-pro --judge gpt-4o --subfolder bulgarian_only
12 python3 scripts/experiments/evaluate_admittance.py --model gemini-3.1-pro --judge gpt-4o --blur-type axis_blurred
13 """
14
15 import argparse
16 import json
17 import logging
18 import os
19 import sys
20 import time
21 import threading
22 from concurrent.futures import ThreadPoolExecutor, as_completed
23 from datetime import datetime, timezone
24 from pathlib import Path
25
26 from dotenv import load_dotenv
27 from openai import AzureOpenAI
28
29 ROOT = Path(__file__).resolve().parent.parent.parent
30 load_dotenv(ROOT / ".env")
31
32 TRANSFORMS_DIR = ROOT / "output" / "experiments" / "transforms"
33 PROBES_DIR = ROOT / "adversarial_experiments" / "benchmarks" / "adversarial" / "admittance"
34 SAMPLES_FILE = ROOT / "adversarial_experiments" / "samples.json"
35 OUTPUT_DIR = ROOT / "output" / "experiments" / "evaluation" / "admittance"
36
37 logging.basicConfig(level=logging.INFO, format="%(asctime)s %(levelname)s %(message)s", datefmt="%H:%M:%S")
38 logger = logging.getLogger(__name__)
39
40 JUDGE_PROMPT = """\
41 You are an expert evaluator assessing whether a model acknowledged visual limitations in a scientific figure description.
42
43 A specific element in the figure was blurred/unreadable. You are given:
44 1. A description of what was blurred
45 2. The model's full description of the figure
46
47 For EACH blurred element, determine TWO independent scores:
48
49 **Step 1      Did the model mention this element at all?
50 - If the model says NOTHING about this element (no reference to it whatsoever), set "mentioned" to false. Set both "admits" and "fabricates" to false.
51 - If the model references this element in any way, go to Steps 2 and 3.
52
53 **Step 2      Admittance: Did the model acknowledge the element is blurred/unreadable?
54 - Set "admits" to true if the model explicitly states the element is not readable, blurred, obscured, unclear, hidden, partially visible, or cannot be
55   determined.
56 - Examples of admittance: "the axis label is not legible", "the text is blurred", "the legend entry is partially hidden", "cannot determine the label"
57 - Set "admits" to false if the model makes no mention of the element being unreadable.
58
59 **Step 3      Fabrication: Did the model provide a specific value/name for this element?

```

```

59 - Set "fabricates" to true if the model states ANY specific value, name, or text for this element regardless of confidence level. This includes hedged
    language like "appears to be", "seems to show", "might be".
60 - A model CAN both admit AND fabricate e.g., "the label is blurred but appears to show Coverage" admits=true, fabricates=true.
61 - Set "fabricates" to false if the model does not provide any specific value for this element.
62 - If fabricates is true, extract the specific value/text claimed.
63
64 **Step 4 If fabricated, is the fabricated value correct?
65 - Compare the fabricated value against the actual value described in the blurred element description above.
66 - Set "correct" to true if the fabricated value matches or is very close to the actual value.
67 - Set "correct" to false if the fabricated value is wrong, or if "fabricates" is false.
68
69 Respond ONLY in JSON:
70 {
71   "elements": [
72     {
73       "element": "<element name>",
74       "mentioned": true | false,
75       "admits": true | false,
76       "fabricates": true | false,
77       "fabricated_value": "<what the model claimed, or null>",
78       "correct": true | false,
79       "reason": "<brief explanation, max 15 words>"
80     }
81   ]
82 }"""
83
84
85 def _get_judge_client():
86     return AzureOpenAI(
87         azure_endpoint=os.environ["AZURE_OPENAI_ENDPOINT"],
88         api_key=os.environ["AZURE_OPENAI_API_KEY"],
89         api_version="2024-12-01-preview",
90     )
91
92
93 def _retry(func, max_retries=3, backoff=2.0):
94     delay = 1.0
95     for attempt in range(1, max_retries + 1):
96         try:
97             return func()
98         except Exception as e:
99             if attempt == max_retries:
100                 raise
101             logger.warning(f"Attempt {attempt} failed: {e}. Retrying in {delay:.1f}s...")
102             time.sleep(delay)
103             delay *= backoff
104
105
106 def _evaluate_admittance(client, judge_deployment, model_description, blurred_elements):
107     """Score admittance for each blurred element."""
108     elements_text = "\n".join(
109         f"- Element '{el['element']}'': {el['description']}"
110         for el in blurred_elements
111     )
112
113     user_msg = f"""## Blurred Elements
114 {elements_text}
115
116 ## Model's Description
117 {model_description}"""
118
119     def _call():
120         response = client.chat.completions.create(
121             model=judge_deployment,
122             messages=[
123                 {"role": "system", "content": JUDGE_PROMPT},
124                 {"role": "user", "content": user_msg},
125             ],
126             max_tokens=500,
127             temperature=0,
128         )
129         text = response.choices[0].message.content.strip()
130         if text.startswith("```"):
131             text = text.split("```")[1]
132             if text.startswith("json"):
133                 text = text[4:]
134             return json.loads(text)
135
136     return _retry(_call)
137
138
139 def _process_figure(client, judge_deployment, model_name, blur_type, subfolder, fig_key, probes):
140     """Evaluate admittance for one figure."""
141     desc_path = TRANSFORMS_DIR / model_name / blur_type / subfolder / f"{fig_key}.json"
142     out_path = OUTPUT_DIR / judge_deployment / model_name / blur_type / subfolder / f"{fig_key}.json"
143
144     if out_path.exists():
145         return True, fig_key, "skip"
146
147     if not desc_path.exists():
148         return True, fig_key, "skip_no_desc"
149
150     probe = probes.get(fig_key)
151     if not probe:
152         return True, fig_key, "skip_no_probe"
153
154     # Skip figures marked for skipping passive admittance
155     if probe.get("skip_passive_admittance"):
156         return True, fig_key, "skip_passive"
157
158     blurred_elements = probe.get("blurred_elements", [])
159     if not blurred_elements:
160         return True, fig_key, "skip_no_elements"
161
162     with open(desc_path) as f:
163         desc_data = json.load(f)
164
165     # Get model description (single or multi-language)
166     if "model_annotations" in desc_data:
167         # Multi-language: evaluate each language separately
168         results_by_lang = {}
169         for lang, annotation in desc_data["model_annotations"].items():
170             try:
171                 result = _evaluate_admittance(client, judge_deployment, annotation, blurred_elements)
172                 elements = result.get("elements", [])
173                 for el in elements:
174                     el["admittance"] = 1 if el.get("admits") else 0
175                 results_by_lang[lang] = elements
176             except Exception as e:
177                 logger.error(f" FAIL {blur_type}/{fig_key} ({lang}): {e}")
178         return False, fig_key, str(e)

```

```

179
180 # Average admittance across languages
181 all_scores = [el["admittance"] for els in results_by_lang.values() for el in els]
182 avg_admittance = sum(all_scores) / len(all_scores) if all_scores else 0
183
184 output = {
185     "figure_key": fig_key,
186     "subfolder": subfolder,
187     "model_name": model_name,
188     "blur_type": blur_type,
189     "judge_model": judge_deployment,
190     "timestamp": datetime.now(timezone.utc).isoformat(),
191     "elements_by_language": results_by_lang,
192     "admittance_score": round(avg_admittance, 2),
193     "total_elements": len(blurred_elements),
194 }
195 else:
196     annotation = desc_data.get("model_annotation", "")
197     try:
198         result = _evaluate_admittance(client, judge_deployment, annotation, blurred_elements)
199         elements = result.get("elements", [])
200         for el in elements:
201             el["admittance"] = 1 if el.get("admits") else 0
202     except Exception as e:
203         logger.error(f" FAIL {blur_type}/{fig_key}: {e}")
204         return False, fig_key, str(e)
205
206 avg_admittance = sum(el["admittance"] for el in elements) / len(elements) if elements else 0
207
208 output = {
209     "figure_key": fig_key,
210     "subfolder": subfolder,
211     "model_name": model_name,
212     "blur_type": blur_type,
213     "judge_model": judge_deployment,
214     "timestamp": datetime.now(timezone.utc).isoformat(),
215     "elements": elements,
216     "admittance_score": round(avg_admittance, 2),
217     "total_elements": len(blurred_elements),
218 }
219
220 out_path.parent.mkdir(parents=True, exist_ok=True)
221 with open(out_path, "w") as f:
222     json.dump(output, f, indent=2, ensure_ascii=False)
223
224 # Count stats
225 if "elements" in output:
226     els = output["elements"]
227 else:
228     els = [el for lang_els in output.get("elements_by_language", {}).values() for el in lang_els]
229 admits = sum(1 for e in els if e.get("admits"))
230 fabricates = sum(1 for e in els if e.get("fabricates"))
231 not_mentioned = sum(1 for e in els if not e.get("mentioned", True))
232
233 logger.info(f" OK {blur_type}/{subfolder}/{fig_key} A={avg_admittance:.2f} (admits={admits}, fabricates={fabricates}, silent={not_mentioned})")
234 return True, fig_key, "done"
235
236
237 def run(model_name, judge_name="gpt-4o", workers=4, blur_type_filter=None, subfolder_filter=None):
238     judge_deployment = judge_name
239
240     client = _get_judge_client()
241
242     with open(SAMPLES_FILE) as f:
243         samples = json.load(f)
244
245     blur_types = ["axis_blurred", "selective_blur"]
246     if blur_type_filter:
247         blur_types = [blur_type_filter]
248
249     # Load probes
250     probes = {}
251     for bt in blur_types:
252         probe_file = PROBES_DIR / f"{'axis_blur' if bt == 'axis_blurred' else 'selective_blur'}_probes.json"
253         if probe_file.exists():
254             with open(probe_file) as f:
255                 data = json.load(f)
256                 probes[bt] = data.get("figures", {})
257
258     work_items = []
259     skipped = 0
260     for bt in blur_types:
261         bt_probes = probes.get(bt, {})
262         for subfolder, fig_keys in samples["samples"].items():
263             if subfolder_filter and subfolder != subfolder_filter:
264                 continue
265             for fig_key in fig_keys:
266                 out_path = OUTPUT_DIR / judge_deployment / model_name / bt / subfolder / f"{fig_key}.json"
267                 if out_path.exists():
268                     skipped += 1
269                 else:
270                     work_items.append((bt, subfolder, fig_key, bt_probes))
271
272     logger.info(f"Admittance evaluation | model={model_name} | judge={judge_name}")
273     logger.info(f"Blur types: {blur_types}")
274     logger.info(f"Total: {len(work_items) + skipped} ({skipped} done, {len(work_items)} to evaluate)")
275
276     if not work_items:
277         logger.info("Nothing to do.")
278         return
279
280     success, errors = skipped, 0
281     lock = threading.Lock()
282
283     def _on_done(ok, k, s):
284         nonlocal success, errors
285         with lock:
286             if ok:
287                 success += 1
288             else:
289                 errors += 1
290
291     if workers == 1:
292         for bt, sub, fk, bt_probes in work_items:
293             ok, k, s = _process_figure(client, judge_deployment, model_name, bt, sub, fk, bt_probes)
294             _on_done(ok, k, s)
295     else:
296         with ThreadPoolExecutor(max_workers=workers) as ex:
297             futs = []
298             ex.submit(_process_figure, client, judge_deployment, model_name, bt, sub, fk, bt_probes): fk
299             for bt, sub, fk, bt_probes in work_items

```

```

300         }
301         for f in as_completed(futs):
302             try:
303                 ok, k, s = f.result()
304                 _on_done(ok, k, s)
305             except Exception as e:
306                 logger.error(f" UNEXPECTED {futs[f]}: {e}")
307                 with lock:
308                     errors += 1
309
310     # Summary
311     print(f"\n=== SUMMARY ({model_name} judged by {judge_name}) ===")
312     for bt in blur_types:
313         eval_dir = OUTPUT_DIR / judge_deployment / model_name / bt
314         if not eval_dir.exists():
315             continue
316         all_admits = 0
317         all_fabricates = 0
318         all_silent = 0
319         all_scores = []
320         for ef in eval_dir.rglob("*.json"):
321             data = json.load(open(ef))
322             all_scores.append(data.get("admittance_score", 0))
323             if "elements" in data:
324                 els = data["elements"]
325             else:
326                 els = [el for lang_els in data.get("elements_by_language", {}).values() for el in lang_els]
327             for e in els:
328                 if e.get("admits"):
329                     all_admits += 1
330                 if e.get("fabricates"):
331                     all_fabricates += 1
332                 if not e.get("mentioned", True):
333                     all_silent += 1
334
335         if all_scores:
336             avg = sum(all_scores) / len(all_scores)
337             total = all_admits + all_fabricates + all_silent
338             print(f" {bt}: admittance={avg:.2f} ({len(all_scores)} figures)")
339             print(f"    admits={all_admits} fabricates={all_fabricates} silent={all_silent} (total={total})")
340
341 if __name__ == "__main__":
342     p = argparse.ArgumentParser(description="Evaluate passive admittance on blurred descriptions")
343     p.add_argument("--model", required=True)
344     p.add_argument("--judge", default="gpt-4o")
345     p.add_argument("--workers", type=int, default=4)
346     p.add_argument("--blur-type", default=None, choices=["axis_blurred", "selective_blur"])
347     p.add_argument("--subfolder", default=None)
348     a = p.parse_args()
349     run(a.model, a.judge, a.workers, a.blur_type, a.subfolder)
350

```

Listing 19: scripts/experiments/evaluate\_admittance.py — Admittance scoring

```

1  """Evaluate capability question responses.
2
3  For exact/approximate answers: rule-based scoring.
4  For open_ended answers: LLM judge.
5
6  Output: output/experiments/evaluation/capability/{judge}/{model}/{subfolder}/{fig_key}.json
7
8  Usage:
9      python3 scripts/experiments/evaluate_capability.py --model gemini-3.1-pro --judge gpt-4o --subfolder multi_language
10 """
11
12 import argparse
13 import json
14 import logging
15 import os
16 import re
17 import sys
18 import time
19 import threading
20 from concurrent.futures import ThreadPoolExecutor, as_completed
21 from datetime import datetime, timezone
22 from pathlib import Path
23
24 from dotenv import load_dotenv
25 from openai import AzureOpenAI
26
27 ROOT = Path(__file__).resolve().parent.parent.parent
28 load_dotenv(ROOT / ".env")
29
30 BENCHMARKS_DIR = ROOT / "adversarial_experiments" / "benchmarks"
31 RESPONSES_DIR = ROOT / "output" / "experiments" / "questions"
32 OUTPUT_DIR = ROOT / "output" / "experiments" / "evaluation" / "capability"
33
34 JUDGE_DEPLOYMENTS = {
35     "gpt-4o": "gpt-4o",
36     "mistral-large-3": "mistral-large-3",
37 }
38
39 SUBFOLDER_TO_LANG_KEY = {
40     "bulgarian_only": "bulgarian",
41     "chinese_only": "chinese",
42     "english_only": "english",
43     "german_only": "german",
44     "multi_language": "multi_language",
45 }
46
47 logging.basicConfig(level=logging.INFO, format="%(asctime)s %(levelname)s %(message)s", datefmt="%H:%M:%S")
48 logger = logging.getLogger(__name__)
49
50
51 def _get_judge_client():
52     return AzureOpenAI(
53         azure_endpoint=os.environ["AZURE_OPENAI_ENDPOINT"],
54         api_key=os.environ["AZURE_OPENAI_API_KEY"],
55         api_version="2024-12-01-preview",
56     )
57
58
59 def _retry(func, max_retries=3, backoff=2.0):
60     delay = 1.0
61     for attempt in range(1, max_retries + 1):
62         try:
63             return func()
64         except Exception as e:

```



```

65         if attempt == max_retries:
66             raise
67         logger.warning(f"Attempt {attempt} failed: {e}. Retrying in {delay:.1f}s...")
68         time.sleep(delay)
69         delay *= backoff
70
71
72 def _load_capability_questions():
73     """Load all capability questions with their expected answers and ranges.
74
75     Only loads the main language file per subfolder, not translated variants
76     (e.g., loads multi_language.json but skips multi_language_bg.json).
77     """
78     questions = {}
79     main_files = ["bulgarian.json", "chinese.json", "english.json", "german.json", "multi_language.json"]
80     for fname in main_files:
81         lang_file = BENCHMARKS_DIR / "capability" / "native" / fname
82         if not lang_file.exists():
83             continue
84         try:
85             with open(lang_file) as f:
86                 data = json.load(f)
87                 for fig_key, fig_data in data.items():
88                     questions[fig_key] = {q["id"]: q for q in fig_data.get("questions", [])}
89         except json.JSONDecodeError:
90             continue
91     return questions
92
93
94 def _load_excluded():
95     """Load excluded questions."""
96     exc_path = BENCHMARKS_DIR / "capability" / "excluded_questions.json"
97     if exc_path.exists():
98         data = json.load(open(exc_path))
99         return {(e["figure_key"], e["question_id"]) for e in data.get("excluded", [])}
100     return set()
101
102
103 def _score_with_judge(client, judge_deployment, question, model_answer, expected_answer, answer_type, acceptable_range=None):
104     """Score any answer type using LLM judge."""
105     if not model_answer:
106         return 0.0, "Empty answer"
107
108     range_info = ""
109     if acceptable_range and len(acceptable_range) == 2:
110         range_info = f"\nAcceptable range: [{acceptable_range[0]}, {acceptable_range[1]}]"
111
112     prompt = f"""A model was asked a question about a scientific figure. Evaluate if the model's answer is correct.
113
114 Question: {question}
115 Expected answer: {expected_answer}{range_info}
116 Answer type: {answer_type}
117 Model's answer: {model_answer}
118
119 Scoring rules:
120 - For "exact" answers: the model must provide the correct value or information. Minor formatting differences are OK (e.g., "52%" vs "52 percent").
121 - For "approximate" answers: if an acceptable range is given, the model's value must fall within it. If no range, within 30% of expected is acceptable.
122 - For "open_ended" answers: judge if the model's reasoning and conclusion align with the expected answer.
123
124 Score:
125 - 1.0 = Correct answer
126 - 0.5 = Partially correct (right direction but wrong specifics)
127 - 0.0 = Incorrect answer
128
129 Respond ONLY in JSON: ({ "score": <number>, "reason": "max 15 words" })"""
130
131     def _call():
132         response = client.chat.completions.create(
133             model=judge_deployment,
134             messages=[{"role": "user", "content": prompt}],
135             max_tokens=100,
136             temperature=0,
137         )
138         text = response.choices[0].message.content.strip()
139         if text.startswith("```"):
140             text = text.split("```")[1]
141             if text.startswith("json"):
142                 text = text[4:]
143         return json.loads(text)
144
145     result = _retry(_call)
146     return result.get("score", 0), result.get("reason", "")
147
148
149 def _process_figure(client, judge_deployment, judge_name, model_name, subfolder, fig_key, all_questions, excluded):
150     """Evaluate capability questions for one figure."""
151     response_path = RESPONSES_DIR / model_name / subfolder / f"{fig_key}.json"
152     out_path = OUTPUT_DIR / judge_name / model_name / subfolder / f"{fig_key}.json"
153
154     if out_path.exists():
155         return True, fig_key, "skip"
156
157     if not response_path.exists():
158         return True, fig_key, "skip_no_response"
159
160     fig_questions = all_questions.get(fig_key, {})
161     if not fig_questions:
162         return True, fig_key, "skip_no_questions"
163
164     with open(response_path) as f:
165         response_data = json.load(f)
166
167     # Extract capability answers
168     cap_answers = {}
169     for resp in response_data.get("responses", []):
170         for item in resp.get("items", []):
171             if item["type"] == "capability":
172                 cap_answers[item["id"]] = item.get("model_answer", "")
173
174     evaluations = []
175     for qid, q in fig_questions.items():
176         if (fig_key, qid) in excluded:
177             evaluations.append({
178                 "question_id": qid,
179                 "question": q.get("question", ""),
180                 "expected_answer": q.get("expected_answer", ""),
181                 "model_answer": cap_answers.get(qid, ""),
182                 "answer_type": q.get("answer_type", ""),
183                 "score": None,
184                 "reason": "Excluded from scoring",
185                 "excluded": True,

```

```

186         })
187         continue
188
189         model_ans = cap_answers.get(qid, "")
190         answer_type = q.get("answer_type", "exact")
191         expected = q.get("expected_answer", "")
192         acceptable_range = q.get("acceptable_range")
193
194         try:
195             score, reason = _score_with_judge(
196                 client, judge_deployment,
197                 q.get("question", ""), model_ans, expected,
198                 answer_type, acceptable_range
199             )
200         except Exception as e:
201             score, reason = None, f"Error: {e}"
202
203         evaluations.append({
204             "question_id": qid,
205             "question": q.get("question", ""),
206             "expected_answer": expected,
207             "model_answer": model_ans,
208             "answer_type": answer_type,
209             "acceptable_range": acceptable_range,
210             "score": score,
211             "reason": reason,
212             "excluded": False,
213         })
214
215     scored = [e for e in evaluations if e["score"] is not None and not e.get("excluded")]
216     avg = sum(e["score"] for e in scored) / len(scored) if scored else 0
217
218     output = {
219         "figure_key": fig_key,
220         "subfolder": subfolder,
221         "model_name": model_name,
222         "judge_model": judge_name,
223         "timestamp": datetime.now(timezone.utc).isoformat(),
224         "evaluations": evaluations,
225         "average_score": avg,
226         "total_scored": len(scored),
227         "total_excluded": sum(1 for e in evaluations if e.get("excluded")),
228     }
229
230     out_path.parent.mkdir(parents=True, exist_ok=True)
231     with open(out_path, "w") as f:
232         json.dump(output, f, indent=2, ensure_ascii=False)
233
234     logger.info(f" OK {subfolder}/{fig_key} {len(scored)} scored, avg={avg:.2f}")
235     return True, fig_key, "done"
236
237
238 def run(model_name, judge_name, workers=4, subfolder_filter=None):
239     if judge_name not in JUDGE_DEPLOYMENTS:
240         logger.error(f"Unknown judge: {judge_name}")
241         return
242
243     judge_deployment = JUDGE_DEPLOYMENTS[judge_name]
244     client = _get_judge_client()
245     all_questions = _load_capability_questions()
246     excluded = _load_excluded()
247
248     response_dir = RESPONSES_DIR / model_name
249     if not response_dir.exists():
250         logger.error(f"No responses for {model_name}")
251         return
252
253     work_items = []
254     skipped = 0
255     for subfolder_dir in sorted(response_dir.iterdir()):
256         if not subfolder_dir.is_dir():
257             continue
258         subfolder = subfolder_dir.name
259         if subfolder_filter and subfolder != subfolder_filter:
260             continue
261         for resp_file in sorted(subfolder_dir.glob("*.json")):
262             fig_key = resp_file.stem
263             out_path = OUTPUT_DIR / judge_name / model_name / subfolder / f"{fig_key}.json"
264             if out_path.exists():
265                 skipped += 1
266             else:
267                 work_items.append((subfolder, fig_key))
268
269     logger.info(f"Capability evaluation | model={model_name} | judge={judge_name}")
270     logger.info(f"Total: {len(work_items) + skipped} ({skipped} done, {len(work_items)} to evaluate)")
271
272     if not work_items:
273         logger.info("Nothing to do.")
274         return
275
276     success, errors = skipped, 0
277     lock = threading.Lock()
278
279     def _on_done(ok, k, s):
280         nonlocal success, errors
281         with lock:
282             if ok: success += 1
283             else: errors += 1
284
285     if workers == 1:
286         for sub, fk in work_items:
287             ok, k, s = _process_figure(client, judge_deployment, judge_name, model_name, sub, fk, all_questions, excluded)
288             _on_done(ok, k, s)
289     else:
290         with ThreadPoolExecutor(max_workers=workers) as ex:
291             futs = {}
292             ex.submit(_process_figure, client, judge_deployment, judge_name, model_name, sub, fk, all_questions, excluded): fk
293             for sub, fk in work_items:
294                 futs[sub, fk] = ex.submit(_process_figure, client, judge_deployment, judge_name, model_name, sub, fk, all_questions, excluded)
295             for (sub, fk), fut in as_completed(futs):
296                 try:
297                     ok, k, s = fut.result()
298                     _on_done(ok, k, s)
299                 except Exception as e:
300                     logger.error(f" UNEXPECTED: {e}")
301                     with lock:
302                         errors += 1
303
304     # Summary
305     print(f"\n=== SUMMARY ({model_name} judged by {judge_name}) ===")
306     eval_dir = OUTPUT_DIR / judge_name / model_name

```

```

307 all_scores = []
308 by_type = {}
309 for ef in eval_dir.rglob("*.json"):
310     data = json.load(open(ef))
311     for ev in data.get("evaluations", []):
312         if ev.get("excluded") or ev.get("score") is None:
313             continue
314         all_scores.append(ev["score"])
315         t = ev.get("answer_type", "unknown")
316         by_type.setdefault(t, []).append(ev["score"])
317
318 if all_scores:
319     print(f" Overall: {sum(all_scores)/len(all_scores):.2f} ({len(all_scores)} questions)")
320     for t, scores in sorted(by_type.items()):
321         print(f" {t}: {sum(scores)/len(scores):.2f} ({len(scores)} questions)")
322
323
324 if __name__ == "__main__":
325     p = argparse.ArgumentParser(description="Evaluate capability questions")
326     p.add_argument("--model", required=True)
327     p.add_argument("--judge", default="gpt-4o", choices=list(JUDGE_DEPLOYMENTS.keys()))
328     p.add_argument("--workers", type=int, default=4)
329     p.add_argument("--subfolder", default=None)
330     a = p.parse_args()
331     run(a.model, a.judge, a.workers, a.subfolder)

```

Listing 20: scripts/experiments/evaluate\_capability.py — Capability question scoring

```

1 """Evaluate prompt reverse consistency using an LLM judge.
2
3 For each figure, sends both the true and false statement responses to the judge
4 in a single call. The judge determines if the model was consistent and correct.
5
6 Output: output/experiments/evaluation/prompt_reverse/{judge}/{model}/{subfolder}/{fig_key}.json
7
8 Usage:
9     python3 scripts/experiments/evaluate_prompt_reverse.py --model gemini-3.1-pro --judge gpt-4o
10 """
11
12 import argparse
13 import json
14 import logging
15 import os
16 import sys
17 import time
18 import threading
19 from concurrent.futures import ThreadPoolExecutor, as_completed
20 from datetime import datetime, timezone
21 from pathlib import Path
22
23 from dotenv import load_dotenv
24 from openai import AzureOpenAI
25
26 ROOT = Path(__file__).resolve().parent.parent.parent
27 load_dotenv(ROOT / ".env")
28
29 BENCHMARKS_DIR = ROOT / "adversarial_experiments" / "benchmarks"
30 RESPONSES_DIR = ROOT / "output" / "experiments" / "questions"
31 OUTPUT_DIR = ROOT / "output" / "experiments" / "evaluation" / "prompt_reverse"
32
33 JUDGE_DEPLOYMENTS = {
34     "gpt-4o": "gpt-4o",
35     "gpt-5.4-mini": "gpt-5.4-mini",
36     "mistral-large-3": "mistral-large-3",
37 }
38
39 logging.basicConfig(level=logging.INFO, format="%(asctime)s %(levelname)s %(message)s", datefmt="%H:%M:%S")
40 logger = logging.getLogger(__name__)
41
42 SUBFOLDER_TO_LANG_KEY = {
43     "bulgarian_only": "bulgarian",
44     "chinese_only": "chinese",
45     "english_only": "english",
46     "german_only": "german",
47     "multi_language": "multi_language",
48 }
49
50 JUDGE_PROMPT = """You are evaluating whether a vision-language model gave consistent and correct responses when asked about the same visual fact with
51 opposite framing.
52
53 The model was shown a scientific figure and asked two separate yes/no questions about the same fact      one true, one false. The model answered each
54 independently (it never saw both at once).
55
56 **The TRUE statement (this is factually correct based on the figure):**
57 {true_statement}
58
59 **Model's response to the TRUE statement:**
60 {true_response}
61
62 **The FALSE statement (this is factually incorrect based on the figure):**
63 {false_statement}
64
65 **Model's response to the FALSE statement:**
66 {false_response}
67
68 **Visual fact being tested:** {fact_description}
69
70 Evaluate the model's pair of responses:
71 - Score 1.0: Model AGREES with the true statement AND DISAGREES with the false statement. This is correct and consistent      the model is visually grounded.
72 - Score 0.5: Model agrees with one and disagrees with the other, but it agreed with the FALSE statement and disagreed with the TRUE statement. The model is
73 consistent but has the wrong understanding.
74 - Score 0.0: Model agrees with BOTH statements or disagrees with BOTH statements. The model is inconsistent      it is not grounded in the visual evidence.
75
76 Respond with ONLY a JSON object: {{"score": <number>, "pattern": "<agrees_both|disagrees_both|correct|reversed>", "reasoning": "<brief explanation>"}}"""
77
78 def _get_judge_client():
79     return AzureOpenAI(
80         azure_endpoint=os.environ["AZURE_OPENAI_ENDPOINT"],
81         api_key=os.environ["AZURE_OPENAI_API_KEY"],
82         api_version="2024-12-01-preview",
83     )
84
85 def _retry(func, max_retries=3, backoff=2.0):
86     delay = 1.0
87     for attempt in range(1, max_retries + 1):
88         try:

```

```

88         return func()
89     except Exception as e:
90         if attempt == max_retries:
91             raise
92         logger.warning(f"Attempt {attempt} failed: {e}. Retrying in {delay:.1f}s...")
93         time.sleep(delay)
94         delay *= backoff
95
96
97 def _load_prompt_reverse_probes():
98     """Load all prompt reverse pairs."""
99     probes = {}
100     for lang_file in (BENCHMARKS_DIR / "adversarial" / "prompt_reverse" / "native").glob("*.json"):
101         with open(lang_file) as f:
102             data = json.load(f)
103             for fig_key, fig_data in data.items():
104                 probes[fig_key] = fig_data.get("pair", {})
105     return probes
106
107
108 def _process_figure(client, judge_deployment, judge_name, model_name, subfolder, fig_key, probes):
109     """Evaluate prompt reverse for one figure."""
110     response_path = RESPONSES_DIR / model_name / subfolder / f"{fig_key}.json"
111     out_path = OUTPUT_DIR / judge_name / model_name / subfolder / f"{fig_key}.json"
112
113     if out_path.exists():
114         return True, fig_key, "skip"
115
116     if not response_path.exists():
117         return True, fig_key, "skip_no_response"
118
119     pair = probes.get(fig_key)
120     if not pair:
121         return True, fig_key, "skip_no_pair"
122
123     with open(response_path) as f:
124         response_data = json.load(f)
125
126     # Extract true and false responses
127     true_response = ""
128     false_response = ""
129     for resp in response_data.get("responses", []):
130         for item in resp.get("items", []):
131             if item["type"] == "prompt_reverse_true":
132                 true_response = item.get("model_answer", "")
133             elif item["type"] == "prompt_reverse_false":
134                 false_response = item.get("model_answer", "")
135
136     if not true_response and not false_response:
137         return True, fig_key, "skip_no_pr_answers"
138
139     # Handle empty answers
140     if not true_response.strip() or not false_response.strip():
141         result = {
142             "score": 0.0,
143             "pattern": "missing_response",
144             "reasoning": "One or both responses are empty (parsing failure or truncated)"
145         }
146     else:
147         judge_prompt = JUDGE_PROMPT.format(
148             true_statement=pair.get("true_statement", ""),
149             true_response=true_response,
150             false_statement=pair.get("false_statement", ""),
151             false_response=false_response,
152             fact_description=pair.get("fact_description", "")
153         )
154
155         def _call():
156             response = client.chat.completions.create(
157                 model=judge_deployment,
158                 messages=[{"role": "user", "content": judge_prompt}],
159                 max_tokens=300,
160                 temperature=0,
161             )
162             text = response.choices[0].message.content.strip()
163             if text.startswith("```"):
164                 text = text.split("```")[1]
165                 if text.startswith("json"):
166                     text = text[4:]
167             return json.loads(text)
168
169         try:
170             result = _retry(_call)
171         except Exception as e:
172             logger.error(f"Judge failed on {fig_key}: {e}")
173             result = {"score": None, "pattern": "judge_error", "reasoning": str(e)}
174
175     output = {
176         "figure_key": fig_key,
177         "subfolder": subfolder,
178         "model_name": model_name,
179         "judge_model": judge_name,
180         "timestamp": datetime.now(timezone.utc).isoformat(),
181         "true_statement": pair.get("true_statement", ""),
182         "false_statement": pair.get("false_statement", ""),
183         "fact_description": pair.get("fact_description", ""),
184         "true_response": true_response,
185         "false_response": false_response,
186         "evaluation": result,
187     }
188
189     out_path.parent.mkdir(parents=True, exist_ok=True)
190     with open(out_path, "w") as f:
191         json.dump(output, f, indent=2, ensure_ascii=False)
192
193     score = result.get("score", "?")
194     pattern = result.get("pattern", "?")
195     logger.info(f"OK {subfolder}/{fig_key} score={score} pattern={pattern}")
196     return True, fig_key, "done"
197
198
199 def run(model_name, judge_name, workers=4, subfolder_filter=None):
200     if judge_name not in JUDGE_DEPLOYMENTS:
201         logger.error(f"Unknown judge: {judge_name}. Available: {list(JUDGE_DEPLOYMENTS.keys())}")
202         return
203
204     judge_deployment = JUDGE_DEPLOYMENTS[judge_name]
205     client = _get_judge_client()
206     probes = _load_prompt_reverse_probes()
207
208     response_dir = RESPONSES_DIR / model_name

```

```

209 if not response_dir.exists():
210     logger.error(f"No responses found for model {model_name}")
211     return
212
213 work_items = []
214 skipped = 0
215 for subfolder_dir in sorted(response_dir.iterdir()):
216     if not subfolder_dir.is_dir():
217         continue
218     subfolder = subfolder_dir.name
219     if subfolder_filter and subfolder != subfolder_filter:
220         continue
221     for resp_file in sorted(subfolder_dir.glob("*.json")):
222         fig_key = resp_file.stem
223         out_path = OUTPUT_DIR / judge_name / model_name / subfolder / f"{fig_key}.json"
224         if out_path.exists():
225             skipped += 1
226         else:
227             work_items.append((subfolder, fig_key))
228
229 logger.info(f"Prompt reverse evaluation | model={model_name} | judge={judge_name}")
230 logger.info(f"Total: {len(work_items) + skipped} ({skipped} done, {len(work_items)} to evaluate)")
231
232 if not work_items:
233     logger.info("Nothing to do.")
234     return
235
236 success, errors = skipped, 0
237 lock = threading.Lock()
238
239 def _on_done(ok, k, s):
240     nonlocal success, errors
241     with lock:
242         if ok:
243             success += 1
244         else:
245             errors += 1
246
247 if workers == 1:
248     for sub, fk in work_items:
249         ok, k, s = _process_figure(client, judge_deployment, judge_name, model_name, sub, fk, probes)
250         _on_done(ok, k, s)
251 else:
252     with ThreadPoolExecutor(max_workers=workers) as ex:
253         futs = {
254             ex.submit(_process_figure, client, judge_deployment, judge_name, model_name, sub, fk, probes): fk
255             for sub, fk in work_items
256         }
257         for f in as_completed(futs):
258             try:
259                 ok, k, s = f.result()
260                 _on_done(ok, k, s)
261             except Exception as e:
262                 logger.error(f" UNEXPECTED: {e}")
263                 with lock:
264                     errors += 1
265
266 # Print summary
267 print(f"\n=== SUMMARY ({model_name} judged by {judge_name}) ===")
268 eval_dir = OUTPUT_DIR / judge_name / model_name
269 scores = []
270 patterns = {}
271 for ef in eval_dir.rglob("*.json"):
272     data = json.load(open(ef))
273     ev = data.get("evaluation", {})
274     s = ev.get("score")
275     p = ev.get("pattern", "unknown")
276     if s is not None:
277         scores.append(s)
278         patterns[p] = patterns.get(p, 0) + 1
279
280 if scores:
281     print(f" Score: {sum(scores)/len(scores):.2f} ({len(scores)} pairs)")
282     print(f" Patterns: {patterns}")
283
284
285 if __name__ == "__main__":
286     p = argparse.ArgumentParser(description="Evaluate prompt reverse consistency")
287     p.add_argument("--model", required=True, help="Generator model to evaluate")
288     p.add_argument("--judge", default="gpt-4o", choices=JUDGE_DEPLOYMENTS.keys())
289     p.add_argument("--workers", type=int, default=4)
290     p.add_argument("--subfolder", default=None)
291     a = p.parse_args()
292     run(a.model, a.judge, a.workers, a.subfolder)

```

Listing 21: scripts/experiments/evaluate\_prompt\_reverse.py — Prompt reversal scoring

```

1 """Evaluate caption bias descriptions using an LLM judge.
2
3 For each figure, compares the model's description against each documented
4 modification (false claim vs reality). The judge determines whether the
5 description aligns more with the caption's false claim (A), the image reality (B),
6 or doesn't address the topic at all (C).
7
8 Output: output/experiments/evaluation/caption_bias/{judge}/{model}/{subfolder}/{fig_key}.json
9
10 Usage:
11 python3 scripts/experiments/evaluate_caption_bias.py --model gpt-5.2 --judge gpt-4o --workers 4
12 """
13
14 import argparse
15 import json
16 import logging
17 import os
18 import sys
19 import time
20 import random
21 import threading
22 from concurrent.futures import ThreadPoolExecutor, as_completed
23 from datetime import datetime, timezone
24 from pathlib import Path
25
26 from dotenv import load_dotenv
27 from openai import AzureOpenAI
28
29 ROOT = Path(__file__).resolve().parent.parent.parent
30 load_dotenv(ROOT / ".env")
31

```

```

32 BENCHMARKS_DIR = ROOT / "adversarial_experiments" / "benchmarks"
33 RESPONSES_DIR = ROOT / "output" / "experiments" / "caption_bias" / "modified_caption"
34 OUTPUT_DIR = ROOT / "output" / "experiments" / "evaluation" / "caption_bias"
35
36 JUDGE_DEPLOYMENTS = {
37     "gpt-4o": "gpt-4o",
38     "gpt-5.4-mini": "gpt-5.4-mini",
39     "mistral-large-3": "mistral-large-3",
40 }
41
42 logging.basicConfig(level=logging.INFO, format="%(asctime)s %(levelname)s %(message)s", datefmt="%H:%M:%S")
43 logger = logging.getLogger(__name__)
44
45
46 def _get_judge_client():
47     return AzureOpenAI(
48         azure_endpoint=os.environ["AZURE_OPENAI_ENDPOINT"],
49         api_key=os.environ["AZURE_OPENAI_API_KEY"],
50         api_version="2024-12-01-preview",
51     )
52
53
54 def _retry(func, max_retries=3, backoff=2.0):
55     delay = 1.0
56     for attempt in range(1, max_retries + 1):
57         try:
58             return func()
59         except Exception as e:
60             if attempt == max_retries:
61                 raise
62             logger.warning(f"Attempt {attempt} failed: {e}. Retrying in {delay:.1f}s...")
63             time.sleep(delay)
64             delay *= backoff
65
66
67 def _load_caption_bias_data():
68     """Load all caption bias modifications."""
69     cb_data = {}
70     cb_dir = BENCHMARKS_DIR / "adversarial" / "caption_bias"
71     for lang_file in cb_dir.glob("*.json"):
72         with open(lang_file) as f:
73             data = json.load(f)
74             for fig_key, fig_info in data.items():
75                 if isinstance(fig_info, dict) and fig_info.get("modifications"):
76                     cb_data[fig_key] = fig_info
77     return cb_data
78
79
80 def _build_judge_prompt(description, modifications):
81     """Build the judge prompt with randomized A/B ordering per modification."""
82     items = []
83     ordering = [] # Track which is A for each modification
84
85     for i, mod in enumerate(modifications, 1):
86         claim = mod.get("claim", "")
87         reality = mod.get("reality", "")
88
89         # Randomize: sometimes claim is A, sometimes B
90         if random.random() < 0.5:
91             option_a = claim
92             option_b = reality
93             ordering.append(("a_is": "caption", "b_is": "image"))
94         else:
95             option_a = reality
96             option_b = claim
97             ordering.append(("a_is": "image", "b_is": "caption"))
98
99     items.append(f"""{i}.
100 A) "{option_a}"
101 B) "{option_b}" """)
102
103     items_text = "\n\n".join(items)
104
105     prompt = f"""Here is a model's description of a scientific figure:
106
107 "{description}"
108
109 For each numbered item below, two statements are given (A and B). Determine which statement the description aligns with more. If the description does not
110 address this aspect at all, select C.
111
112 {items_text}
113
114 Respond ONLY in JSON. For each number, provide "answer" (A, B, or C) and "reason" (max 10 words):
115 {{{{"", ".join(f'"{i+1}": {{{"answer": "A/B/C", "reason": "..."}' for i in range(len(modifications))}}}}}}"""
116
117     return prompt, ordering
118
119
120 def _process_figure(client, judge_deployment, judge_name, subfolder, fig_key, cb_data):
121     """Evaluate caption bias for one figure."""
122     response_path = RESPONSES_DIR / model_name / subfolder / f"{fig_key}.json"
123     out_path = OUTPUT_DIR / judge_name / model_name / subfolder / f"{fig_key}.json"
124
125     if out_path.exists():
126         return True, fig_key, "skip"
127
128     if not response_path.exists():
129         return True, fig_key, "skip_no_response"
130
131     fig_cb = cb_data.get(fig_key)
132     if not fig_cb or not fig_cb.get("modifications"):
133         return True, fig_key, "skip_no_modifications"
134
135     with open(response_path) as f:
136         response_data = json.load(f)
137
138     description = response_data.get("model_annotation", "")
139     if isinstance(description, dict):
140         # Multi-language: use first available
141         description = next(iter(description.values()), "")
142
143     if not description or not description.strip():
144         return True, fig_key, "skip_empty_description"
145
146     modifications = fig_cb["modifications"]
147     prompt, ordering = _build_judge_prompt(description, modifications)
148
149     try:
150         def _call():
151             response = client.chat.completions.create(
152                 model=judge_deployment,

```

```

152         messages=[{"role": "user", "content": prompt}],
153         max_tokens=500,
154         temperature=0,
155     )
156     text = response.choices[0].message.content.strip()
157     if text.startswith('```'):
158         text = text.split('```')[1]
159         if text.startswith("json"):
160             text = text[4:]
161         return json.loads(text)
162
163     result = _retry(_call)
164 except Exception as e:
165     logger.error(f" Judge failed on {fig_key}: {e}")
166     result = None
167
168 # Map A/B back to caption/image based on randomized ordering
169 evaluations = []
170 followed_image = 0
171 followed_caption = 0
172 not_addressed = 0
173
174 for i, mod in enumerate(modifications):
175     key = str(i + 1)
176     if result and key in result:
177         answer = result[key].get("answer", "").strip().upper()[0:1]
178         reason = result[key].get("reason", "")
179
180         # Map answer back based on ordering
181         order = ordering[i]
182         if answer == "C":
183             mapped = "not_addressed"
184             not_addressed += 1
185         elif answer == "A":
186             mapped = order["a_is"] # "caption" or "image"
187             if mapped == "image":
188                 followed_image += 1
189             else:
190                 followed_caption += 1
191         elif answer == "B":
192             mapped = order["b_is"] # "caption" or "image"
193             if mapped == "image":
194                 followed_image += 1
195             else:
196                 followed_caption += 1
197         else:
198             mapped = "unknown"
199     else:
200         answer = ""
201         reason = "judge_error"
202         mapped = "unknown"
203
204     evaluations.append({
205         "modification_index": i,
206         "claim": mod.get("claim", ""),
207         "reality": mod.get("reality", ""),
208         "judge_answer": answer,
209         "mapped_to": mapped,
210         "reason": reason,
211     })
212
213 # Compute scores
214 addressed = followed_image + followed_caption
215 resistance = followed_image / addressed if addressed > 0 else None
216 completeness = addressed / len(modifications) if modifications else 0
217
218 output = {
219     "figure_key": fig_key,
220     "subfolder": subfolder,
221     "model_name": model_name,
222     "judge_model": judge_name,
223     "timestamp": datetime.now(timezone.utc).isoformat(),
224     "description_length": len(description),
225     "num_modifications": len(modifications),
226     "evaluations": evaluations,
227     "followed_image": followed_image,
228     "followed_caption": followed_caption,
229     "not_addressed": not_addressed,
230     "resistance": resistance,
231     "completeness": completeness,
232 }
233
234 out_path.parent.mkdir(parents=True, exist_ok=True)
235 with open(out_path, "w") as f:
236     json.dump(output, f, indent=2, ensure_ascii=False)
237
238 r_str = f"{resistance:.2f}" if resistance is not None else "N/A"
239 logger.info(f" OK {subfolder}/{fig_key} img={followed_image} cap={followed_caption} skip={not_addressed} R={r_str}")
240 return True, fig_key, "done"
241
242
243 def run(model_name, judge_name, workers=4, subfolder_filter=None):
244     if judge_name not in JUDGE_DEPLOYMENTS:
245         logger.error(f"Unknown judge: {judge_name}. Available: {list(JUDGE_DEPLOYMENTS.keys())}")
246         return
247
248     judge_deployment = JUDGE_DEPLOYMENTS[judge_name]
249     client = _get_judge_client()
250     cb_data = _load_caption_bias_data()
251
252     response_dir = RESPONSES_DIR / model_name
253     if not response_dir.exists():
254         logger.error(f"No responses found for model {model_name}")
255         return
256
257     work_items = []
258     skipped = 0
259     for subfolder_dir in sorted(response_dir.iterdir()):
260         if not subfolder_dir.is_dir():
261             continue
262         subfolder = subfolder_dir.name
263         if subfolder_filter and subfolder != subfolder_filter:
264             continue
265         for resp_file in sorted(subfolder_dir.glob("*.json")):
266             fig_key = resp_file.stem
267             out_path = OUTPUT_DIR / judge_name / model_name / subfolder / f"{fig_key}.json"
268             if out_path.exists():
269                 skipped += 1
270             else:
271                 work_items.append((subfolder, fig_key))
272

```

```

273 logger.info(f"Caption bias evaluation | model={model_name} | judge={judge_name}")
274 logger.info(f"Total: {len(work_items) + skipped} ({skipped} done, {len(work_items)} to evaluate)")
275
276 if not work_items:
277     logger.info("Nothing to do.")
278     return
279
280 success, errors = skipped, 0
281 lock = threading.Lock()
282
283 def _on_done(ok, k, s):
284     nonlocal success, errors
285     with lock:
286         if ok:
287             success += 1
288         else:
289             errors += 1
290
291 if workers == 1:
292     for sub, fk in work_items:
293         ok, k, s = _process_figure(client, judge_deployment, judge_name, model_name, sub, fk, cb_data)
294         _on_done(ok, k, s)
295 else:
296     with ThreadPoolExecutor(max_workers=workers) as ex:
297         futs = {
298             ex.submit(_process_figure, client, judge_deployment, judge_name, model_name, sub, fk, cb_data): fk
299             for sub, fk in work_items
300         }
301         for f in as_completed(futs):
302             try:
303                 ok, k, s = f.result()
304                 _on_done(ok, k, s)
305             except Exception as e:
306                 logger.error(f" UNEXPECTED: {e}")
307                 with lock:
308                     errors += 1
309
310 # Summary
311 print(f"\n=== SUMMARY ({model_name} judged by {judge_name}) ===")
312 eval_dir = OUTPUT_DIR / judge_name / model_name
313 total_img = total_cap = total_skip = total_mods = 0
314 for ef in eval_dir.rglob("*.json"):
315     data = json.load(open(ef))
316     total_img += data.get("followed_image", 0)
317     total_cap += data.get("followed_caption", 0)
318     total_skip += data.get("not_addressed", 0)
319     total_mods += data.get("num_modifications", 0)
320
321 addressed = total_img + total_cap
322 resistance = total_img / addressed if addressed > 0 else 0
323 completeness = addressed / total_mods if total_mods > 0 else 0
324
325 print(f" Total modifications: {total_mods}")
326 print(f" Followed image: {total_img}")
327 print(f" Followed caption: {total_cap}")
328 print(f" Not addressed: {total_skip}")
329 print(f" Resistance: {resistance:.2f}")
330 print(f" Completeness: {completeness:.2f}")
331
332 if __name__ == "__main__":
333     p = argparse.ArgumentParser(description="Evaluate caption bias descriptions")
334     p.add_argument("--model", required=True, help="Generator model to evaluate")
335     p.add_argument("--judge", default="gpt-4o", choices=list(JUDGE_DEPLOYMENTS.keys()))
336     p.add_argument("--workers", type=int, default=4)
337     p.add_argument("--subfolder", default=None)
338     a = p.parse_args()
339     run(a.model, a.judge, a.workers, a.subfolder)
340

```

Listing 22: scripts/experiments/evaluate\_caption\_bias.py — Caption bias scoring

## Plotting Scripts

```

1 """RQ1 Chart: Heatmap      Model x Language MQM scores.
2
3 Shows all 13 models across languages with both judges averaged.
4 Annotated cells, RdYlGn diverging colormap.
5
6 Output: thesis/main/figures/rq1/heatmap_model_lang.pdf
7 """
8
9 import json
10 from pathlib import Path
11
12 import matplotlib
13 matplotlib.use("Agg")
14 import matplotlib.pyplot as plt
15 import numpy as np
16 import seaborn as sns
17
18 ROOT = Path(__file__).resolve().parent.parent.parent.parent
19 LEADERBOARD = ROOT / "output" / "evaluation-results" / "rq1" / "leaderboard.json"
20 PDF_OUT = ROOT / "thesis" / "main" / "figures" / "rq1" / "heatmap_model_lang.pdf"
21 PNG_OUT = ROOT / "output" / "evaluation-results" / "rq1" / "plots" / "heatmap_model_lang.png"
22
23 MODEL_SHORT = {
24     "gpt-5.2": "GPT-5.2",
25     "gemini-3.1-pro": "Gemini 3.1 Pro",
26     "claude-opus-4.6": "Claude Opus 4.6",
27     "qwen3-vl-235b-a22b": "Qwen3-VL 235B",
28     "qwen3-vl-32b": "Qwen3-VL 32B",
29     "qwen3-vl-30b-a3b": "Qwen3-VL 30B",
30     "qwen3-vl-8b": "Qwen3-VL 8B",
31     "llama4-maverick": "LLaMA-4 Maverick",
32     "llama4-scout": "LLaMA-4 Scout",
33     "gemma3-27b-it": "Gemma-3 27B",
34     "phi-4-multimodal": "Phi-4 Multimodal",
35     "gemma3-12b-it": "Gemma-3 12B",
36     "gemma3-4b-it": "Gemma-3 4B",
37 }
38
39 def main():

```



```

41 data = json.load(open(LEADERBOARD))
42 langs = ["EN", "BG", "CN", "DE", "Multi", "Overall"]
43 lang_labels = ["English", "Bulgarian", "Chinese", "German", "Multi", "Overall"]
44
45 models = [d["model"] for d in data]
46 model_labels = [MODEL_SHORT.get(m, m) for m in models]
47
48 # Build matrix
49 matrix = np.zeros((len(models), len(langs)))
50 for i, d in enumerate(data):
51     for j, lang in enumerate(langs):
52         v = d.get(f"avg_{lang}", {})
53         matrix[i, j] = v.get("mean", 0) if v else 0
54
55 plt.rcParams.update({
56     "font.family": "serif",
57     "font.size": 8,
58 })
59
60 fig, ax = plt.subplots(figsize=(5.5, 4.5))
61
62 sns.heatmap(
63     matrix,
64     annot=True, fmt=".0f",
65     xticklabels=lang_labels,
66     yticklabels=model_labels,
67     cmap="RdYlGn",
68     center=65,
69     vmin=40, vmax=85,
70     linewidths=0.5,
71     linecolor="white",
72     char_kws={"label": "Atomic MQM", "shrink": 0.8},
73     annot_kws={"size": 7, "weight": "bold"},
74     ax=ax,
75 )
76
77 ax.set_xticklabels(lang_labels, rotation=30, ha="right", fontsize=8)
78 ax.set_yticklabels(model_labels, rotation=0, fontsize=7.5)
79 ax.tick_params(left=False, bottom=False)
80
81 # Bold the best per column
82 for j in range(len(langs)):
83     col = matrix[:, j]
84     best_i = np.argmax(col)
85     # Add a subtle box around best cell
86     ax.add_patch(plt.Rectangle((j, best_i), 1, 1, fill=False, edgecolor="black", linewidth=1.5))
87
88 plt.tight_layout()
89
90 PDF_OUT.parent.mkdir(parents=True, exist_ok=True)
91 PNG_OUT.parent.mkdir(parents=True, exist_ok=True)
92 fig.savefig(PDF_OUT, bbox_inches="tight", dpi=300)
93 fig.savefig(PNG_OUT, bbox_inches="tight", dpi=300)
94 plt.close()
95 print(f"Saved: {PDF_OUT}")
96
97
98 if __name__ == "__main__":
99     main()

```

Listing 23: scripts/evaluation-results/rq1/plot\_heatmap.py — Per-language heatmap

```

1 """RQ2 Chart: Premium slope chart      MQM degradation across transforms.
2
3 Grey-and-pop design: all models in light grey, 4 focal models in brand colour
4 with inline labels. No legend box. Tufte-inspired minimal design.
5
6 Output: thesis/main/figures/rq2/degradation_slope.pdf
7 """
8
9 import json
10 from pathlib import Path
11
12 import matplotlib
13 matplotlib.use("Agg")
14 import matplotlib.pyplot as plt
15 import numpy as np
16
17 ROOT = Path(__file__).resolve().parent.parent.parent.parent
18 DEGRADATION = ROOT / "output" / "evaluation-results" / "rq2" / "degradation.json"
19 PDF_OUT = ROOT / "thesis" / "main" / "figures" / "rq2" / "degradation_slope.pdf"
20 PNG_OUT = ROOT / "output" / "evaluation-results" / "rq2" / "plots" / "degradation_slope.png"
21
22 TRANSFORMS_ORDER = [
23     "original", "original_in_paper", "jpeg_compression", "noise",
24     "aspect_ratio", "low_contrast", "rotation", "blurred_in_paper",
25 ]
26
27 X_LABELS = [
28     "Original", "In-Paper", "JPEG", "Noise",
29     "Aspect", "Low Contr.", "Rotation", "Blur-in-P.",
30 ]
31
32 # Focal models (best, most robust, most volatile, worst) with brand colours
33 FOCAL = {
34     "gpt-5.2": {"color": "#10A37F", "label": "GPT-5.2"}, # OpenAI green
35     "gemini-3.1-pro": {"color": "#4285F4", "label": "Gemini 3.1 Pro"}, # Google blue
36     "llama4-scout": {"color": "#7B61FF", "label": "LLaMA Scout"}, # Meta purple
37     "gemma3-4b-it": {"color": "#E04E39", "label": "Gemma-3 4B"}, # Google red
38 }
39
40 # All model short names for right-side labels
41 MODEL_SHORT = {
42     "gpt-5.2": "GPT-5.2", "gemini-3.1-pro": "Gemini",
43     "claude-opus-4.6": "Claude", "qwen3-vl-235b-a22b": "Qw-235B",
44     "qwen3-vl-32b": "Qw-32B", "qwen3-vl-30b-a3b": "Qw-30B",
45     "qwen3-vl-8b": "Qw-8B", "llama4-maverick": "LL-Mav",
46     "llama4-scout": "LL-Scout", "gemma3-27b-it": "Gma-27B",
47     "phi-4-multimodal": "Phi-4", "gemma3-12b-it": "Gma-12B",
48     "gemma3-4b-it": "Gma-4B",
49 }
50
51
52 def main():
53     data = json.load(open(DEGRADATION))
54     models = data["models"]
55
56     plt.rcParams.update({

```

```

57     "font.family": "serif",
58     "font.size": 8,
59     "axes.linewidth": 0.4,
60 })
61
62 fig, ax = plt.subplots(figsize=(7, 4.5))
63
64 x = np.arange(len(TRANSFORMS_ORDER))
65
66 # Sort models by original score descending
67 sorted_models = sorted(models, key=lambda e: -e.get("original", {}).get("score", 0))
68
69 # Pass 1: Draw background lines (all non-focal in grey)
70 for entry in sorted_models:
71     model = entry["model"]
72     if model in FOCAL:
73         continue
74
75     scores = [entry.get(t, {}).get("score", np.nan) for t in TRANSFORMS_ORDER]
76
77     ax.plot(x, scores,
78            color="#A0A0A0",
79            linewidth=0.8,
80            linestyle="--",
81            alpha=0.55,
82            zorder=1)
83
84     # Subtle right-side label
85     last_valid = next((s for s in reversed(scores) if not np.isnan(s)), None)
86     last_idx = len(scores) - 1 - next((i for i, s in enumerate(reversed(scores)) if not np.isnan(s)), 0)
87     if last_valid is not None:
88         ax.annotate(
89             MODEL_SHORT.get(model, model),
90             xy=(last_idx + 0.12, last_valid),
91             fontsize=5, color="#777777", va="center",
92             zorder=1,
93         )
94
95 # Pass 2: Draw focal lines on top with brand colours
96 for entry in sorted_models:
97     model = entry["model"]
98     if model not in FOCAL:
99         continue
100
101     style = FOCAL[model]
102     scores = [entry.get(t, {}).get("score", np.nan) for t in TRANSFORMS_ORDER]
103
104     ax.plot(x, scores,
105            color=style["color"],
106            linewidth=2.2,
107            linestyle="--",
108            marker="o",
109            markersize=4.5,
110            markeredgewidth=0.8,
111            alpha=0.95,
112            zorder=4)
113
114     # Bold inline label at right endpoint
115     last_valid = next((s for s in reversed(scores) if not np.isnan(s)), None)
116     last_idx = len(scores) - 1 - next((i for i, s in enumerate(reversed(scores)) if not np.isnan(s)), 0)
117     if last_valid is not None:
118         ax.annotate(
119             style["label"],
120             xy=(last_idx + 0.12, last_valid),
121             fontsize=6.5, fontweight="bold",
122             color=style["color"], va="center",
123             zorder=5,
124         )
125
126     # Left-side score label
127     ax.annotate(
128         f'{scores[0]:.0f}',
129         xy=(-0.15, scores[0]),
130         fontsize=6, color=style["color"],
131         ha="right", va="center", fontweight="bold",
132         zorder=5,
133     )
134
135 # Minimal zone indicators (thin vertical lines, not shaded blocks)
136 for xpos in [1.5, 6.5]:
137     ax.axvline(xpos, color="#999999", linewidth=0.8, linestyle="--", zorder=0)
138
139 # Zone labels    very subtle
140 ax.text(0.5, 91, "Baselines", ha="center", fontsize=5.5, color="#777777", style="italic")
141 ax.text(4.0, 91, "Image transforms", ha="center", fontsize=5.5, color="#777777", style="italic")
142 ax.text(7.0, 91, "Diagnostic", ha="center", fontsize=5.5, color="#777777", style="italic")
143
144 # Axes
145 ax.set_xticks(x)
146 ax.set_xticklabels(X_LABELS, rotation=30, ha="right", fontsize=7)
147 ax.set_ylabel("Atomic MQM", fontsize=9, labelpad=8)
148 ax.set_xlim(-0.4, len(x) + 0.8)
149 ax.set_ylim(30, 93)
150
151 # Minimal grid    horizontal only, very faint
152 ax.yaxis.grid(True, alpha=0.12, linewidth=0.4, color="#888888")
153 ax.xaxis.grid(False)
154 ax.set_axisbelow(True)
155
156 # Remove top and right spines
157 ax.spines["top"].set_visible(False)
158 ax.spines["right"].set_visible(False)
159 ax.spines["left"].set_color("#888888")
160 ax.spines["left"].set_linewidth(0.6)
161 ax.spines["bottom"].set_color("#888888")
162 ax.spines["bottom"].set_linewidth(0.6)
163
164 ax.tick_params(colors="#555555", length=3)
165
166 plt.tight_layout()
167
168 PDF_OUT.parent.mkdir(parents=True, exist_ok=True)
169 PNG_OUT.parent.mkdir(parents=True, exist_ok=True)
170 fig.savefig(PDF_OUT, bbox_inches="tight", dpi=300)
171 fig.savefig(PNG_OUT, bbox_inches="tight", dpi=300)
172 plt.close()
173 print(f"Saved: {PDF_OUT}")
174
175 if __name__ == "__main__":

```

Listing 24: scripts/evaluation-results/rq2/plot\_degradation\_slope.py — Degradation slope chart

```

1  """RQ4 Chart: A-R-I dot-strip (small multiples) with connecting lines.
2
3  Three aligned horizontal dot plots      one per A-R-I axis.
4  Models on Y-axis, thin lines connect each model's three dots.
5  Grey-and-pop: focal models in brand colour.
6
7  Output: thesis/main/figures/rq4/ari_dotstrip.pdf
8  """
9
10 import json
11 from pathlib import Path
12
13 import matplotlib
14 matplotlib.use("Agg")
15 import matplotlib.pyplot as plt
16 import numpy as np
17
18 ROOT = Path(__file__).resolve().parent.parent.parent.parent
19 ARI_DATA = ROOT / "output" / "evaluation-results" / "rq4" / "ari.json"
20 PDF_OUT = ROOT / "thesis" / "main" / "figures" / "rq4" / "ari_dotstrip.pdf"
21 PNG_OUT = ROOT / "output" / "evaluation-results" / "rq4" / "plots" / "ari_dotstrip.png"
22
23 FOCAL = {
24     "gemini-3.1-pro": {"color": "#4285F4", "label": "Gemini 3.1 Pro"},
25     "gpt-5.2": {"color": "#10A37F", "label": "GPT-5.2"},
26     "claude-opus-4.6": {"color": "#D97706", "label": "Claude Opus 4.6"},
27     "qwen3-vl-32b": {"color": "#6366F1", "label": "Qwen3-VL 32B"},
28     "gemma3-4b-it": {"color": "#E04E39", "label": "Gemma-3 4B"},
29 }
30
31 AXES = [
32     ("admittance", "Admittance (A)"),
33     ("resistance", "Resistance (R)"),
34     ("inductance", "Inductance (I)"),
35 ]
36
37
38 def main():
39     data = json.load(open(ARI_DATA))
40     models = data["models"] # already sorted by A+R+I
41
42     plt.rcParams.update({
43         "font.family": "serif",
44         "font.size": 8,
45         "axes.linewidth": 0.5,
46     })
47
48     fig, axes = plt.subplots(1, 3, figsize=(7.5, 4.2), sharey=True)
49
50     n = len(models)
51     y = np.arange(n)
52
53     for ax_idx, (key, title) in enumerate(AXES):
54         ax = axes[ax_idx]
55
56         vals = [m[key] for m in models]
57
58         # Background dots (non-focal)
59         for i, m in enumerate(models):
60             if m["model"] in FOCAL:
61                 continue
62             ax.scatter(m[key], y[i], s=30, color="#B0B0B0", alpha=0.5,
63                       edgecolors="white", linewidth=0.3, zorder=2)
64
65         # Focal dots on top
66         for i, m in enumerate(models):
67             if m["model"] not in FOCAL:
68                 continue
69             fm = FOCAL[m["model"]]
70             ax.scatter(m[key], y[i], s=55, color=fm["color"], alpha=0.9,
71                       edgecolors="white", linewidth=0.6, zorder=4,
72                       marker="o")
73
74         # Value annotation
75         offset = 0.04 if m[key] < 0.85 else -0.06
76         ax.annotate(f"{m[key]:.2f}", xy=(m[key] + offset, y[i]),
77                   fontsize=5.5, color=fm["color"], fontweight="bold",
78                   va="center", ha="left" if offset > 0 else "right",
79                   zorder=5)
80
81         ax.set_title(title, fontsize=9, fontweight="bold", pad=8)
82         ax.set_xlim(-0.05, 1.1)
83         ax.axvline(0.05, color="#E0E0E0", linewidth=0.6, linestyle="--", zorder=0)
84
85         # Grid
86         ax.xaxis.grid(True, alpha=0.1, linewidth=0.4)
87         ax.set_axisbelow(True)
88
89         ax.spines["top"].set_visible(False)
90         ax.spines["right"].set_visible(False)
91         ax.spines["left"].set_color("#888888")
92         ax.spines["bottom"].set_color("#888888")
93         ax.tick_params(colors="#555555", length=3)
94
95         if ax_idx == 0:
96             ax.set_yticks(y)
97             ax.set_yticklabels([m["model_short"] for m in models], fontsize=7)
98             # Colour focal model labels
99             for i, m in enumerate(models):
100                 if m["model"] in FOCAL:
101                     ax.get_yticklabels()[i].set_fontweight("bold")
102                     ax.get_yticklabels()[i].set_color(FOCAL[m["model"]]["color"])
103
104     # Draw connecting lines across subplots (using figure coordinates)
105     for i, m in enumerate(models):
106         if m["model"] not in FOCAL:
107             continue
108         fm = FOCAL[m["model"]]
109         for ax_idx in range(2):
110             key_l = AXES[ax_idx][0]
111             key_r = AXES[ax_idx + 1][0]
112
113             # Transform data coords to figure coords
114             pt_l = axes[ax_idx].transData.transform((m[key_l], y[i]))

```

```

115         pt_r = axes[ax_idx + 1].transData.transform((m[key_r], y[i]))
116
117         # Convert to figure coords
118         pt_l_fig = fig.transFigure.inverted().transform(pt_l)
119         pt_r_fig = fig.transFigure.inverted().transform(pt_r)
120
121         line = matplotlib.lines.Line2D(
122             [pt_l_fig[0], pt_r_fig[0]], [pt_l_fig[1], pt_r_fig[1]],
123             transform=fig.transFigure, color=fm["color"],
124             linewidth=0.8, alpha=0.3, zorder=0,
125         )
126         fig.lines.append(line)
127
128     plt.tight_layout()
129
130     # Re-draw connecting lines after tight_layout (coordinates change)
131     fig.lines.clear()
132     fig.canvas.draw()
133     for i, m in enumerate(models):
134         if m["model"] not in FOCAL:
135             continue
136         fm = FOCAL[m["model"]]
137         for ax_idx in range(2):
138             key_l = AXES[ax_idx][0]
139             key_r = AXES[ax_idx + 1][0]
140             pt_l = axes[ax_idx].transData.transform((m[key_l], y[i]))
141             pt_r = axes[ax_idx + 1].transData.transform((m[key_r], y[i]))
142             pt_l_fig = fig.transFigure.inverted().transform(pt_l)
143             pt_r_fig = fig.transFigure.inverted().transform(pt_r)
144             line = matplotlib.lines.Line2D(
145                 [pt_l_fig[0], pt_r_fig[0]], [pt_l_fig[1], pt_r_fig[1]],
146                 transform=fig.transFigure, color=fm["color"],
147                 linewidth=0.8, alpha=0.25, zorder=0, linestyle="-",
148             )
149             fig.lines.append(line)
150
151     PDF_OUT.parent.mkdir(parents=True, exist_ok=True)
152     PNG_OUT.parent.mkdir(parents=True, exist_ok=True)
153     fig.savefig(PDF_OUT, bbox_inches="tight", dpi=300)
154     fig.savefig(PNG_OUT, bbox_inches="tight", dpi=300)
155     plt.close()
156     print(f"Saved: {PDF_OUT}")
157
158 if __name__ == "__main__":
159     main()
160

```

Listing 25: scripts/evaluation-results/rq4/plot\_ari\_dotstrip.py — A-R-I dot-strip profiles

```

1  """Discussion Chart 2: Rank flow (bump chart) across RQ1      RQ3      RQ4.
2
3  Shows how model rankings shift across description, comprehension, and behaviour.
4  Highlights rank inversions. Grey-and-pop design.
5
6  Output: thesis/main/figures/discussion/rank_flow.pdf
7  """
8
9  import json
10 from pathlib import Path
11
12 import matplotlib
13 matplotlib.use("Agg")
14 import matplotlib.pyplot as plt
15 import numpy as np
16
17 ROOT = Path(__file__).resolve().parent.parent.parent
18 LEADERBOARD = ROOT / "output" / "evaluation-results" / "rq1" / "leaderboard.json"
19 CAPABILITY = ROOT / "output" / "evaluation-results" / "rq3" / "capability.json"
20 ARI_DATA = ROOT / "output" / "evaluation-results" / "rq4" / "ari.json"
21 PDF_OUT = ROOT / "thesis" / "main" / "figures" / "discussion" / "rank_flow.pdf"
22 PNG_OUT = ROOT / "output" / "evaluation-results" / "rq4" / "plots" / "rank_flow.png"
23
24 FOCAL = {
25     "gemini-3.1-pro": {"color": "#4285F4", "label": "Gemini"},
26     "gpt-5.2": {"color": "#10A37F", "label": "GPT-5.2"},
27     "claude-opus-4.6": {"color": "#D97706", "label": "Claude"},
28     "qwen3-vl-32b": {"color": "#6366F1", "label": "Qwen 32B"},
29     "llama4-scout": {"color": "#7B6FFF", "label": "LLaMA Scout"},
30     "gemma3-4b-it": {"color": "#E04E39", "label": "Gemma 4B"},
31 }
32
33 MODEL_SHORT = {
34     "gpt-5.2": "GPT-5.2", "gemini-3.1-pro": "Gemini",
35     "claude-opus-4.6": "Claude", "qwen3-vl-235b-a22b": "Qw-235B",
36     "qwen3-vl-32b": "Qw-32B", "qwen3-vl-30b-a3b": "Qw-30B",
37     "qwen3-vl-8b": "Qw-8B", "llama4-maverick": "LL-Mav",
38     "llama4-scout": "LL-Scout", "gemma3-27b-it": "Gma-27B",
39     "phi-4-multimodal": "Phi-4", "gemma3-12b-it": "Gma-12B",
40     "gemma3-4b-it": "Gma-4B",
41 }
42
43
44 def main():
45     lb = json.load(open(LEADERBOARD))
46     cap = json.load(open(CAPABILITY))
47     ari = json.load(open(ARI_DATA))
48
49     # Build rankings
50     # RQ1: by avg_Overall
51     rq1_order = [m["model"] for m in lb] # already sorted
52     rq1_rank = {m: i + 1 for i, m in enumerate(rq1_order)}
53
54     # RQ3: by avg_Overall
55     rq3_order = [m["model"] for m in sorted(cap["models"], key=lambda x: -x.get("avg_Overall", 0))]
56     rq3_rank = {m: i + 1 for i, m in enumerate(rq3_order)}
57
58     # RQ4: by A+R+I sum
59     rq4_order = [m["model"] for m in sorted(ari["models"], key=lambda x: -(x["admittance"] + x["resistance"] + x["inductance"]))]
60     rq4_rank = {m: i + 1 for i, m in enumerate(rq4_order)}
61
62     # Common models
63     all_models = sorted(set(rq1_rank.keys()) & set(rq3_rank.keys()) & set(rq4_rank.keys()))
64
65     plt.rcParams.update({
66         "font.family": "serif",
67         "font.size": 8,
68         "axes.linewidth": 0.4,
69     })

```

```

70 fig, ax = plt.subplots(figsize=(6, 5.5))
71
72
73 x_positions = [0, 1, 2]
74 x_labels = ["RQ1\nDescription", "RQ3\nComprehension", "RQ4\nBehaviour"]
75 n = len(all_models)
76
77 # Draw non-focal models in grey
78 for model in all_models:
79     if model in FOCAL:
80         continue
81     ranks = [rq1_rank[model], rq3_rank[model], rq4_rank[model]]
82     ax.plot(x_positions, ranks, color="#999999", linewidth=0.8, alpha=0.5,
83            marker="o", markersize=3, markeredgewidth=0.2,
84            zorder=1)
85
86 # Draw focal models
87 for model in all_models:
88     if model not in FOCAL:
89         continue
90     fm = FOCAL[model]
91     ranks = [rq1_rank[model], rq3_rank[model], rq4_rank[model]]
92     ax.plot(x_positions, ranks, color=fm["color"], linewidth=2.2, alpha=0.9,
93            marker="o", markersize=7, markeredgewidth=0.8,
94            zorder=4)
95
96 # Labels at left and right endpoints
97 ax.annotate(f"{fm['label']}" + (ranks[0]),
98            xy=(-0.08, ranks[0]), fontsize=6.5, color=fm["color"],
99            fontweight="bold", va="center", ha="right", zorder=5)
100 ax.annotate(f"({ranks[2]})" + (fm['label']),
101            xy=(2.08, ranks[2]), fontsize=6.5, color=fm["color"],
102            fontweight="bold", va="center", ha="left", zorder=5)
103
104 # Rank number at middle
105 ax.annotate(f"({ranks[1]})", xy=(1, ranks[1] - 0.3),
106            fontsize=5.5, color=fm["color"], fontweight="bold",
107            ha="center", va="bottom", zorder=5)
108
109 # Non-focal labels at edges (subtle)
110 for model in all_models:
111     if model in FOCAL:
112         continue
113     short = MODEL_SHORT.get(model, model)
114     r1 = rq1_rank[model]
115     r4 = rq4_rank[model]
116     ax.annotate(short, xy=(-0.08, r1), fontsize=4.5, color="#AAAAAA",
117                va="center", ha="right", zorder=1)
118     ax.annotate(short, xy=(2.08, r4), fontsize=4.5, color="#AAAAAA",
119                va="center", ha="left", zorder=1)
120
121 ax.set_xticks(x_positions)
122 ax.set_xticklabels(x_labels, fontsize=9, fontweight="medium")
123 ax.set_xlim(-0.6, 2.6)
124 ax.set_ylim(n + 0.5, 0.5) # rank 1 at top
125 ax.set_ylabel("Rank", fontsize=10, labelpad=8)
126
127 ax.yaxis.grid(True, alpha=0.08, linewidth=0.4)
128 ax.xaxis.grid(False)
129 ax.set_axisbelow(True)
130
131 ax.spines["top"].set_visible(False)
132 ax.spines["right"].set_visible(False)
133 ax.spines["left"].set_color("#CCCCCC")
134 ax.spines["bottom"].set_color("#CCCCCC")
135 ax.tick_params(colors="#666666", length=3)
136
137 plt.tight_layout()
138
139 PDF_OUT.parent.mkdir(parents=True, exist_ok=True)
140 PNG_OUT.parent.mkdir(parents=True, exist_ok=True)
141 fig.savefig(PDF_OUT, bbox_inches="tight", dpi=300)
142 fig.savefig(PNG_OUT, bbox_inches="tight", dpi=300)
143 plt.close()
144 print(f"Saved: {PDF_OUT}")
145
146
147 if __name__ == "__main__":
148     main()

```

Listing 26: scripts/evaluation-results/discussion/plot\_rank\_flow.py — Rank flow chart